SKALA

K-뉴딜 Back-end 개발

Java 이해하기

2026.07

본 문서는 SK(주)C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

목차

1일차

- 프로그램 기초
- 자바둘러보기
- 개발환경구성
- 자바변수와 자료형
- 네이밍 규칙wk
- 클래스와 객체
- 주석달기
- 자바의 제어문
- 자바 패키지
- 자바 필수 클래스
- 객체지향 프로그램
- Java 내부 구조

2일차

- 상속
- 추상화.인터페이스
- 추상 클래스
- 다형성
- Collection Type
- Generic Type
- Lambda Expression
- Stream API
- Java Util 클래스
- Reflection
- Annotation
- 개념토론

3일차 (오전)

- 객체 생성 패턴
- SOLID
- 네크워크 기초
- Socket
- Simple HTTP

프로그램 기초

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

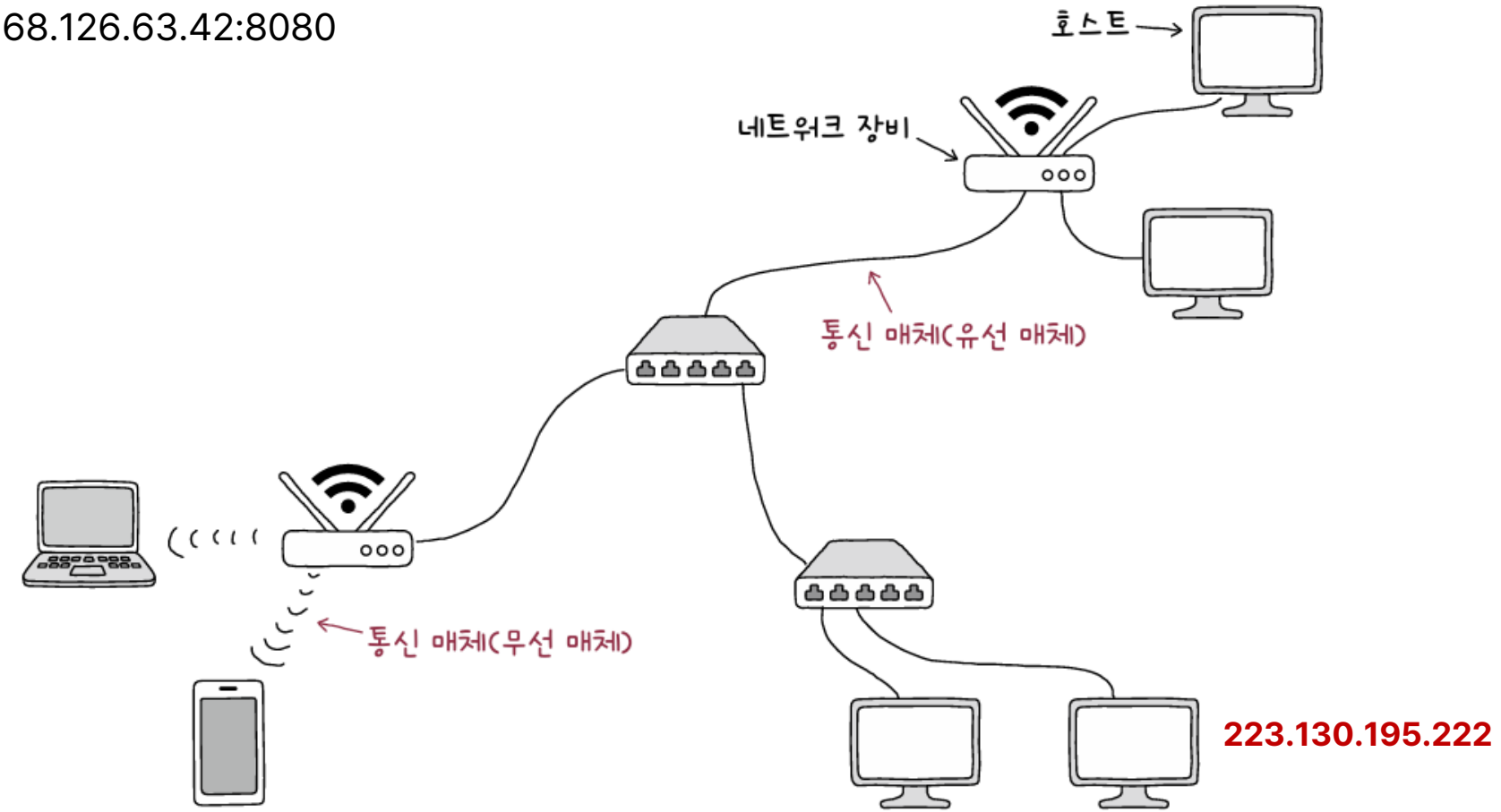
SK 주식회사 C&C

IP 와 Port

네트워크 환경에서 프로세스 통신을 식별하기 위한 식별자 : IP + PORT

- **IP 주소:** 인터넷에서 특정 컴퓨터(Host)를 식별하기 위한 논리적 주소
- **Port**: IP 주소를 가진 Host 내에 실행중인 서비스/프로세스를 구분하는 번호

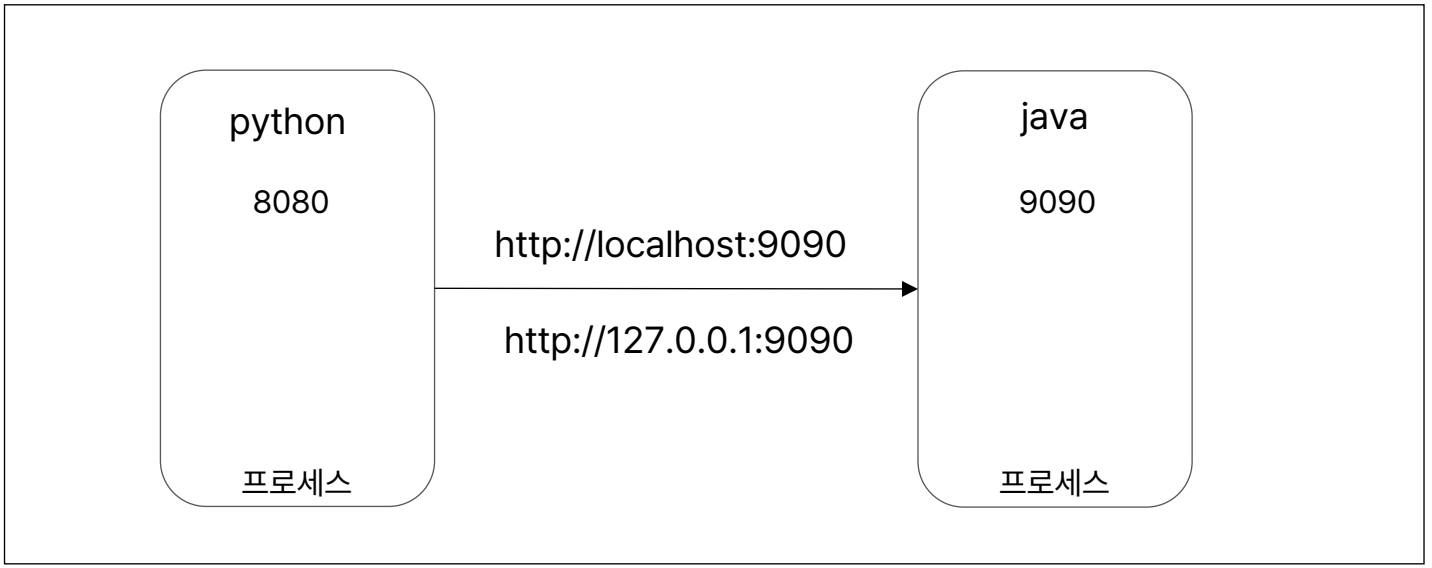
http://168.126.63.42:8080



Localhost vs 127.0.0.1

동일 Host 내에서 프로세스 간 통신을 위한 domain Name (localhost) , IP (127.0.0.1)

로컬 PC IP: 168.126.63.42



Domain 정보 관리 (로컬)

```
cat /etc/hosts
127.0.0.1       localhost
```

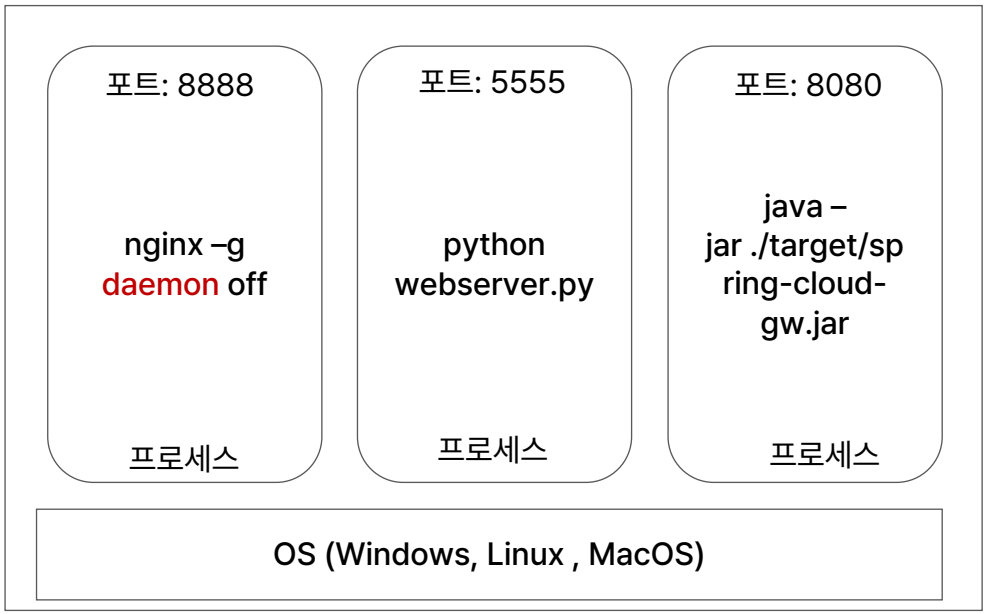
무선/유선 IP 확인

```
ipconfig getifaddr en0   # 무선

ipconfig getifaddr en1   # 유선
```

Process 와 Port

- 현재 **Memory**에 로드되어 **CPU**를 할당받고 실행 중인 프로그램

```
python  webserver.py
npm run dev
java –jar ./target/ideoperators-0.0.1-SNAPSHOT.jar
```



외부 프로세스간 통신

- IP + Port

  예) 172.8.32.9:8080

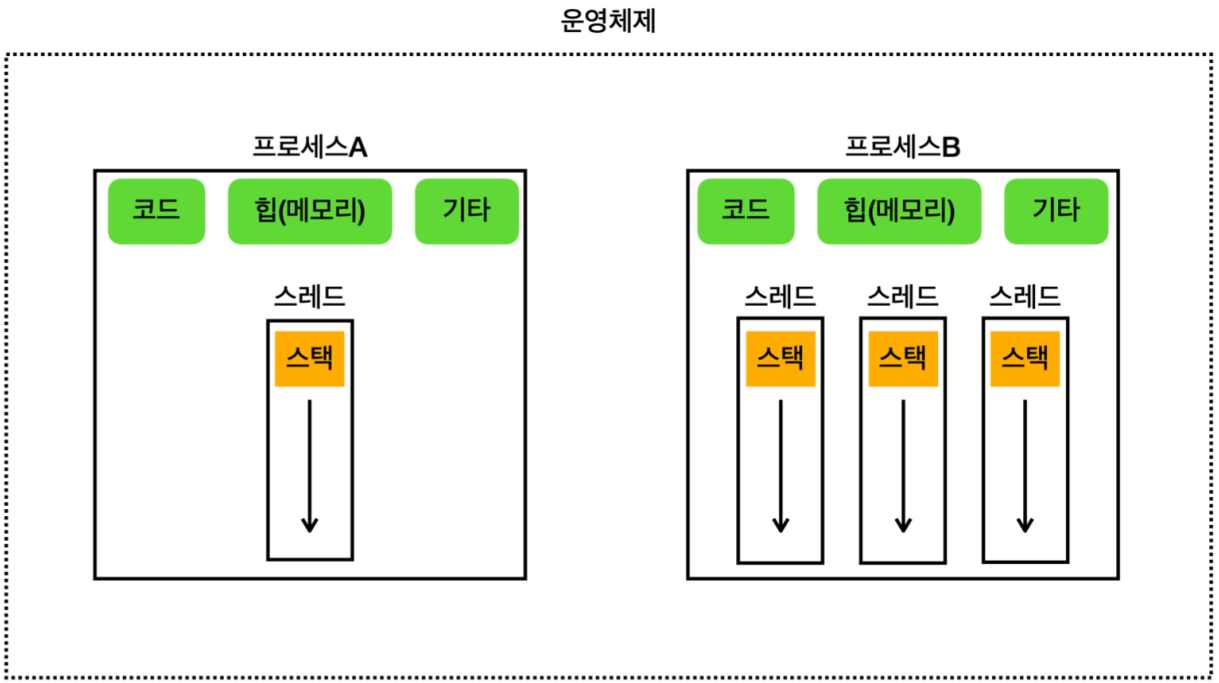
내부 프로세스 간 통신

- 127.0.0.1 + Port

  예) 127.0.0.1:8080 , localhost:8080

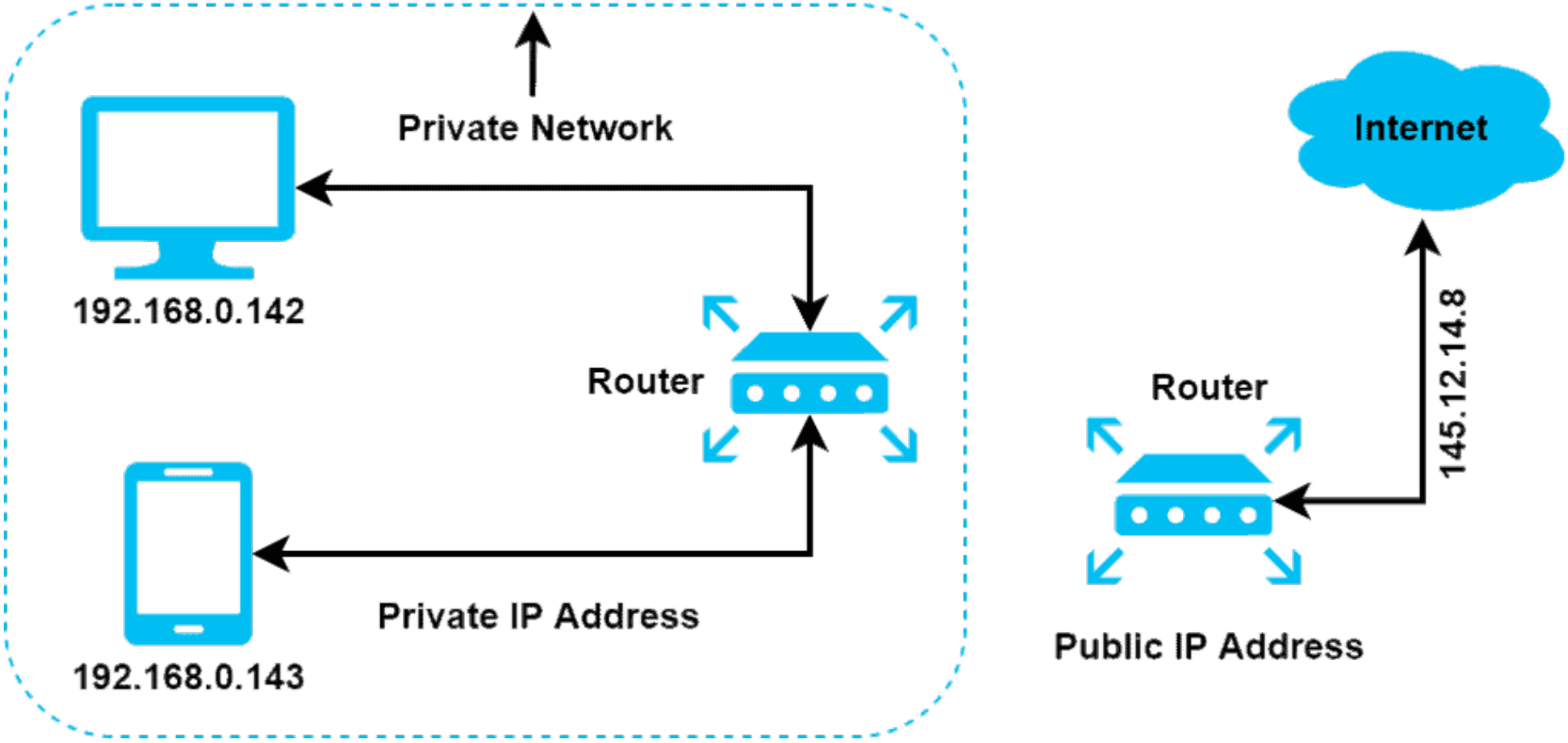
Process vs Thread

항목	프로세스 (Process)	스레드 (Thread)
정의	실행 중인 프로그램의 인스턴스	프로세스 내에서 실행되는 작업 단위
메모리 공간	독립적인 메모리 공간 사용	프로세스의 메모리 공간을 공유
생성 비용	상대적으로 크고 무겁다	생성 비용이 낮고 가볍다
통신 방법	IPC (파이프, 소켓 등) 필요	공유 메모리를 통해 간단하게 통신 가능
충돌 영향	다른 프로세스에 영향 없음	다른 스레드에 영향을 줄 수 있음
예시	웹 브라우저, 게임, IDE 등 독립 실행 환경	웹 페이지 렌더링, 백그라운드 다운로드 등



Public IP 와 Private IP

특정 private Network 내에서만 유일한 식별자 private IP (대부분 Private IP)



내부에서 사용하는 고유 번호 리스트 관리

Gateway, Router, Load Balancer

Router는 연결

네트워크를 연결/확장, 길을 만드는 기능

인터넷 자체가 Router들의 집합

Layer 3

Gateway 는 관문

서비스/Application으로 진입하기 위한 진입점

프로토콜의 변환

외부 IP 를 내부 IP로 변환

JWT 인증, Filter, CORS, Header 변경

Layer 7 Routing (Service Routing)

+

Service LoadBalaner (Optional)

Load Balancer는 분배

요청 트래픽을 여러 노드에 분배/분산

Layer 4, Layer 7

NLB, ALB ??

프론트 엔드 (Front-end)

- **사용자가 실제로 눈으로 보고, 손으로 조작하는 웹 화면을 만드는 영역**
- **사용자와 직접 맞닿는 인터페이스(UI) 와 사용 경험(UX) 을 설계하고 구현하는 일**

역할	설명
구조 만들기 (HTML)	웹페이지의 뼈대 - 예: 제목, 단락, 버튼, 이미지 위치 등
꾸미기 (CSS)	색깔, 글꼴, 여백, 애니메이션 등 디자인 요소
동작 넣기 (JavaScript)	클릭, 입력, 드래그 등 사용자 행동에 반응하도록 기능을 추가
데이터 불러오기 (API 연동)	백엔드 서버에 데이터를 요청하고, 받은 정보를 화면에 노출
사용자 경험 개선 (UX)	더 편리하고 빠르게 사용할 수 있도록 화면 흐름과 반응을 최적화

- **프론트 엔드에서 자주 쓰는 기술**

구분	예시
기본 언어	HTML, CSS, JavaScript
프레임워크/라이브러리	React, Vue.js (더 효율적으로 화면 개발)
패키지 관리자	npm, yarn (필요한 라이브러리 설치 도구)
빌드 도구	Webpack, Vite (코드를 빠르게 묶고 최적화)

End: 종단, 경계지점, 맞닿는 지점 (Endpoint)

백 엔드 (Back-end)

- **웹서비스의 보이지 않는 뒷단에서 모든 기능이 제대로 작동하도록 만드는 영역**
- **프론트 엔드의 요청을 받아서 데이터를 처리하고 결과를 돌려주는 역할**

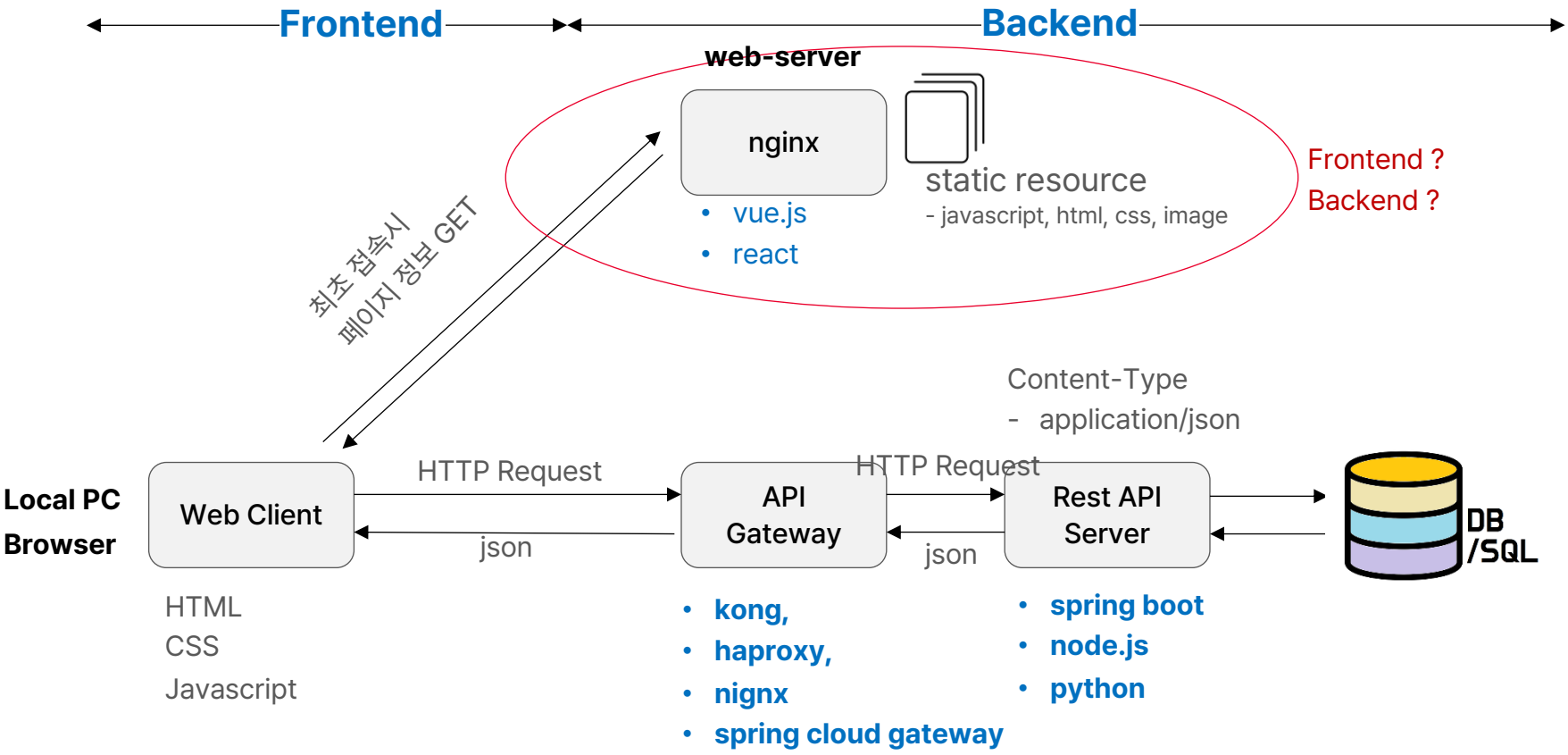
역할	설명
서버 운영	사용자의 요청을 받는 서버(컴퓨터 프로그램)를 관리하고 실행
DB연동, 데이터 처리	사용자가 입력한 정보를 계산·검증·분석하고 필요한 처리를 수행
API 제공	프론트엔드가 데이터를 주고받을 수 있도록 API를 만들어 제공
보안 및 권한 관리	로그인, 암호화, 접근 제한 등 사용자 정보 보호 기능을 담당
성능 관리 및 로깅	서버가 빠르고 안정적으로 동작하도록 최적화하고, 오류를 기록

- **백 엔드에서 자주 쓰는 기술**

구분	예시
프로그래밍 언어	Java, Python, JavaScript(Node.js), Go, Kotlin
프레임워크	Spring Boot(Java), Django(Python), Express(Node.js), FastAPI(Python)
데이터베이스(DB)	MySQL, PostgreSQL, MongoDB, Redis
API 형식	REST API, GraphQL
서버/배포 환경	AWS, Docker, Nginx, GitHub Actions 등

현대적 Frontend / Backend 구조

- 최초 한 번 HTML을 로드한 뒤, 이후 화면 전환은 페이지 새로고침 없이 JavaScript로 화면을 동적으로 변경하는 웹 애플리케이션 방식
- Client side에서 화면에 필요한 변수 데이터 생성 및 처리 작업(JS)



JSON Format

- JSON은 데이터 표현 및 저장을 위한 형식으로, 주로 데이터 교환 및 API 응답 등에 사용

JSON (JavaScript Object Notation)

정의:

- JSON은 데이터를 키-값 쌍 형식의 경량 데이터 교환 형식
- 언어에 구애받지 않고 대부분의 프로그래밍 언어에서 사용
- 사람이 읽기 쉽고, 기계가 처리하기도 간단

기본 구조:

- **객체(Object):** {}로 묶이고, 키-값 쌍으로 구성
- **배열(Array):** []로 묶이고, 값들의 리스트를 포함
- **값(Value):** 문자열, 숫자, 객체, 배열, true, false, null 지원

example.json

```
{
  "name": "John Doe",
  "age": 30,
  "isMarried": false,
  "children": ["Anna", "Ben"],
  "address": {
    "city": "Seoul",
    "zipCode": "12345"
  }
}
```

YAML Format

- YAML은 데이터 표현 및 저장을 위한 형식으로, 주로 구성 파일에 사용

YAML (YAML Ain't Markup Language)

정의:

- YAML은 사람이 읽기 쉽도록 설계된 데이터 직렬화 형식
- 들여쓰기를 통해 데이터 계층을 표현
- JSON보다 간결하고 가독성이 높음
- 주로 구성 파일(config)에서 많이 사용

기본 구조:

- 키-값 쌍(Key-Value Pair): ":"으로 구분
- 리스트(List): "-"로 시작
- 들여쓰기: 계층 구조를 나타내는 데 사용
- 주석: #로 작성

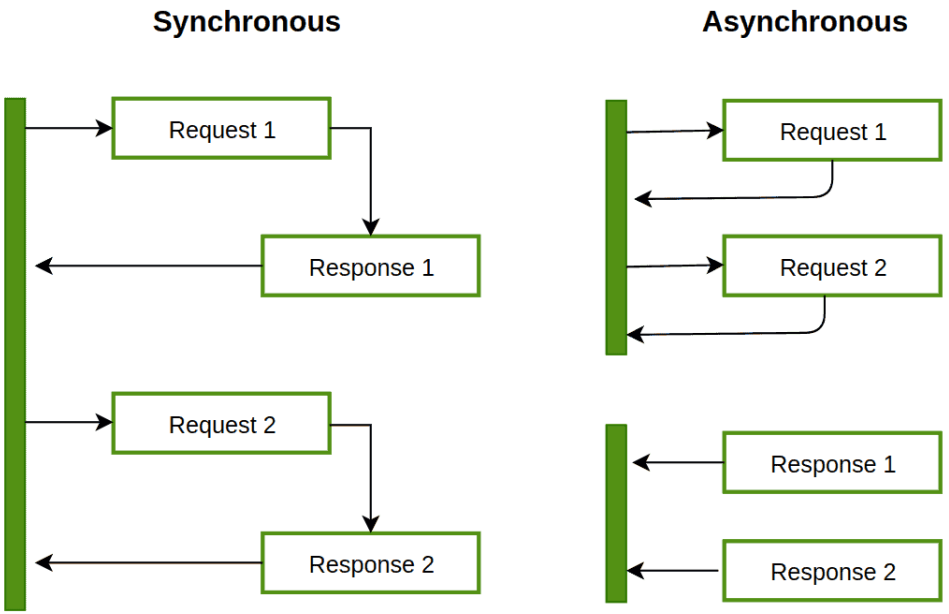
example.yaml

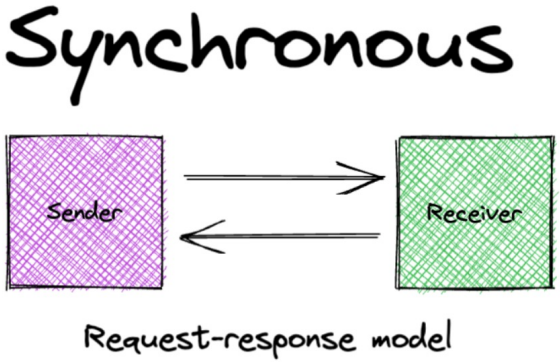
```yaml
name: John Doe
age: 30
isMarried: false
children:
  - Anna
  - Ben
address:
  city: Seoul
  zipCode: "12345"
```

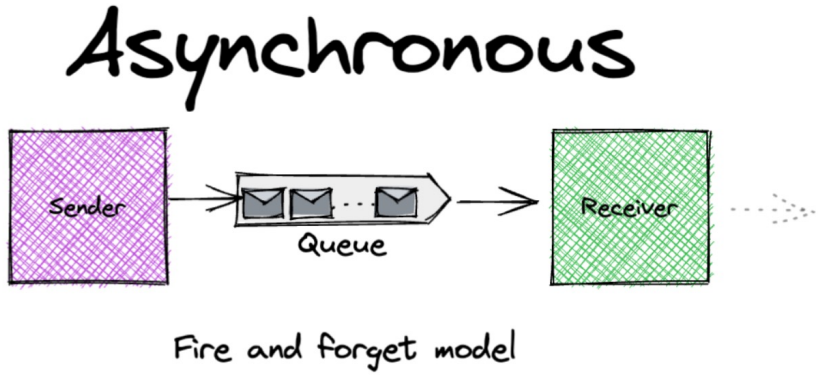
JSON vs YAML

특징	JSON 설명	YAML 설명
형식	{}와 []를 사용한 구조적 표현	들여쓰기 기반의 구조적 표현
가독성	사람이 읽을 수 있으나 형식이 딱딱함	사람이 읽기 매우 쉬움
주석 지원	지원하지 않음	#로 주석 작성 가능
사용 사례	API 응답, 데이터 교환, 브라우저 내 데이터 처리	구성 파일, 설정 파일
파싱 속도	빠름	JSON보다 느릴 수 있음
유연성	고정된 형식	더 유연하며 덜 엄격함

동기와 비동기 (Sync vs Async) (1/2)







동기와 비동기 (Sync vs Async) (2/2)

구분	동기식(Synchronous)	비동기식(Asynchronous)
처리 방식	요청 → 응답까지 기다림	요청 → 기다리지 않고 다음 작업 수행, 결과는 나중에 수신
흐름 제어	직렬적(Blocking) 처리	병렬적(Non-blocking) 처리
응답 시간	요청 수 많을수록 느려짐	병렬 처리 가능하여 시스템 응답성 향상 가능
구현 복잡도	단순함 (순차적 코드 흐름)	복잡함 (콜백 지옥, 상태 관리 필요)
신뢰성	높은 일관성 (순서대로 처리됨)	처리 순서 불확실성 (재시도, 실패 감지 필요)
리소스 사용	자원 점유 시간 길어짐	자원 효율적으로 사용 가능
사용 적합 상황	트랜잭션이 중요한 금융/결제 시스템 등	대규모 사용자 이벤트 처리, 알림, 대기열 기반 작업 등
예시	함수 호출 후 리턴값 기다림, HTTP 요청/응답	이벤트 리스너, 콜백, Promise, 메시지 큐

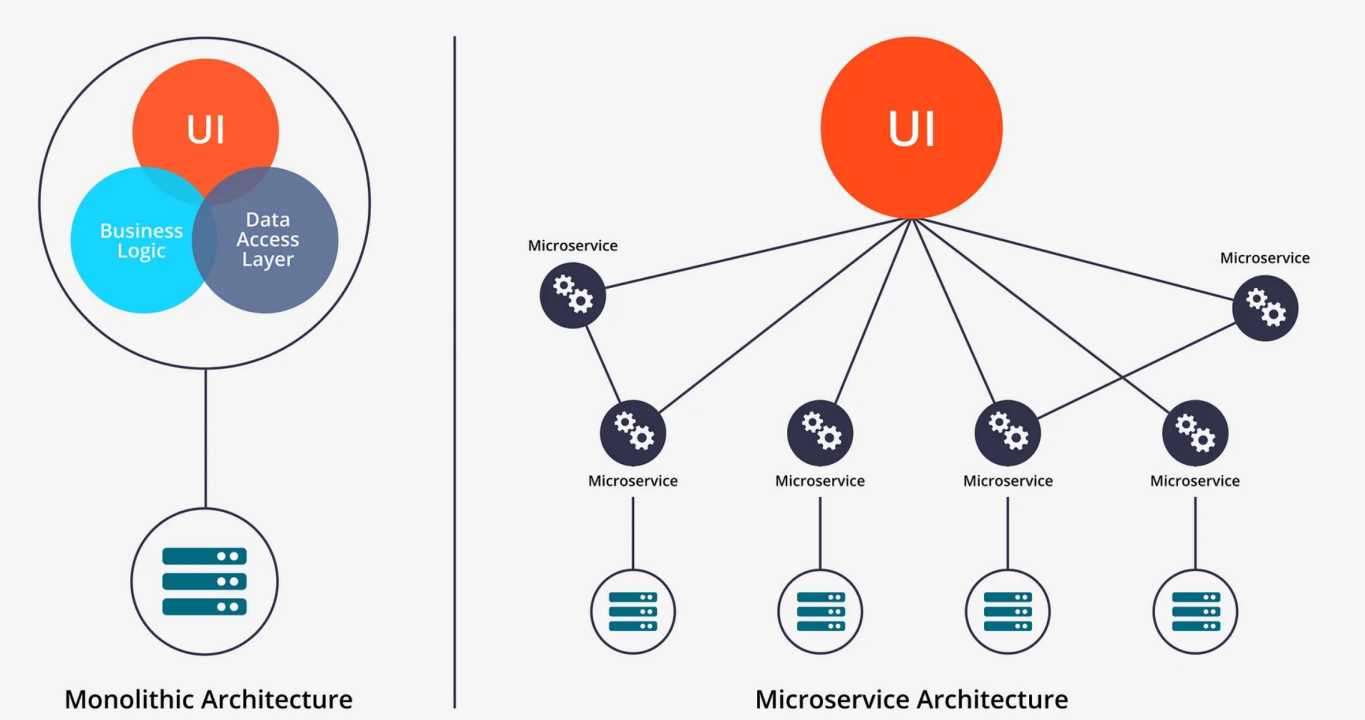
좋은 소프트웨어란?

관점	정의
응집도 (Cohesion)	• 하나의 모듈/서비스가 하나의 역할/책임에 집중하는 정도 (선택과 집중) • 높을수록 변경/이해/재사용이 쉬움.
복잡도 (Complexity)	• 업무 로직, 아키텍처의 복잡도 • 낮을수록 유지보수와 확장 용이.
단독 실행 (Standalone Execution)	• 시스템의 일부를 독립적으로 실행하거나 테스트할 수 있는 능력. • 높을수록 분산 개발/배포/장애 복구가 유리함.
결합도 (Coupling)	• 구성 요소 간 의존성, 낮을수록 (Loosely-Coupled) 변경/확장이 쉬움. • 높으면 (Tightly-Coupled) 전체 시스템에 영향 줌.

소프트웨어 아키텍처 비교

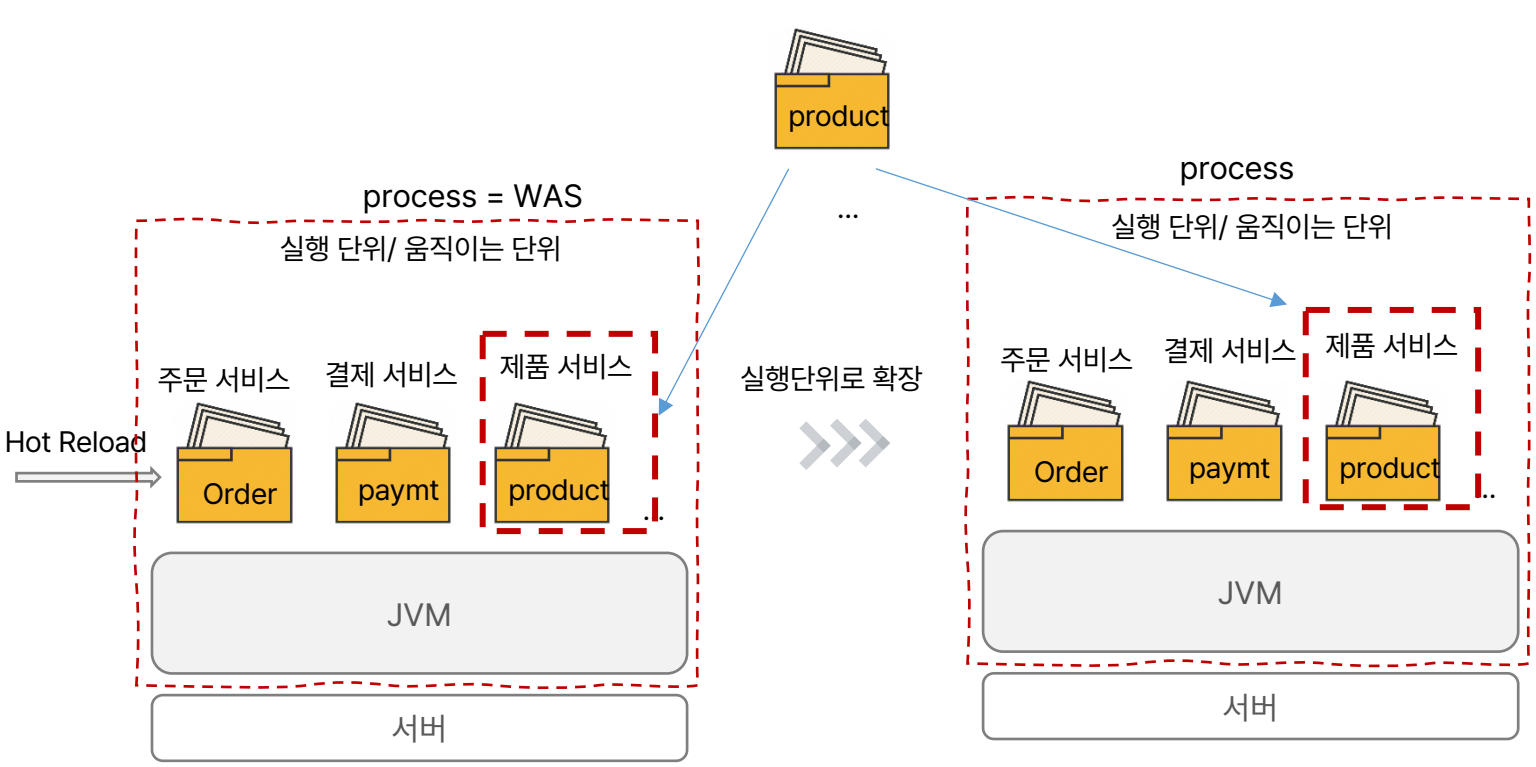
기준	Monolith	MSA (Microservice Architecture)
응집도	역할과 책임의 혼재 (응집도 ↓)	기능별 책임 분리 (응집도 ↑)
복잡도	로직의 복잡도 ↑ , 아키텍처 복잡도 ↓	로직의 복잡도 ↓, 아키텍처 복잡도 ↑
단독 실행	개별 실행이 어려움, 테스트 복잡도 증가	개별 실행 용이, 테스트 복잡도 낮음
결합도	높음 (Tightly-Coupled)→ 하나 변경 시 전체 영향	낮음 (Loosely-Coupled)→ 독립적 배포·확장 가능

❖Monolith는 하나의 프로그램이므로 **응집도**가 더 높지 않을까? → 하나의 모듈이 얼마나 일관된 책임을 가지고 있는가의 측면



Monolith (덩어리)

- 실행 단위는 개발, 배포, 운영 단위



코드 묶음
(class, bean)

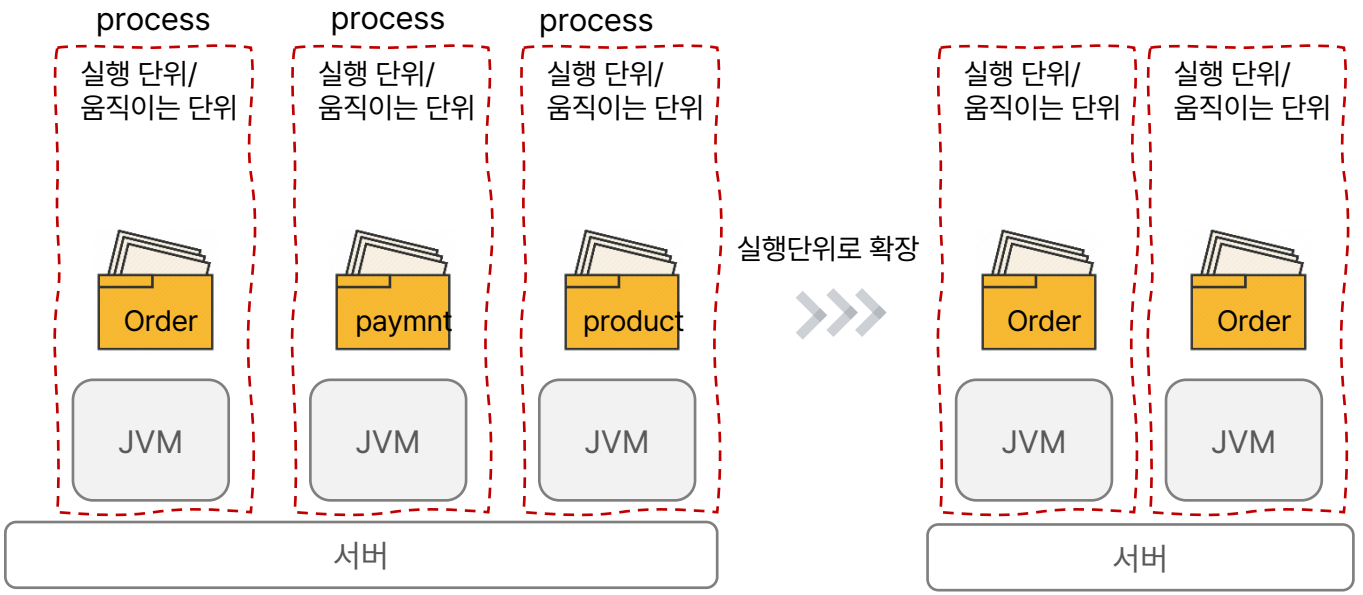
나쁜 소프트웨어

- 내부는 **낮은 응집도**
- 내부는 **높은 복잡도**
- 개발/배포는 **War, Jar**
  실행단위는 **WAS**
- 서비스간 **긴밀한 결합**

❖ "나쁘다"는 의미 ? 잘못되었다는 의미는 아니며, 개발/운영 관점에서 관리하기 어렵다는 의미로 이해

마이크로 서비스 (알갱이)

- 실행 단위는 개발, 배포, 운영 단위 - 작아지면 관리가 편하다.



좋은 소프트웨어

- 내부는 **높음 응집도**
- 내부는 **낮은 복잡도**
- 개발/배포/실행단위 **동일**
- 서비스간 **긴밀한 결합**

❖ 참고 :마이크로는 기준 단위의 10^{-6} : "나노도 아니고 밀리도 아닌가? 적당하게 작다" 라는 의미

KDT Back-end 개발 – Java, SpringBoot, Rest API 활용/구현

자바 둘러보기

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

Java의 역사

Java 등장 이전 주요 어려움 (1990년대 초반)

- 플랫폼 종속성 문제 (OS 하드웨어 의존)
- 메모리 관리의 어려움
- 메모리 직접 접근 및 OS 권한을 가지는 프로그램으로 인한 보안 취약
- 네트워크 /분산 프로그램 개발의 어려움

JAVA의 등장

하드웨어와 OS중심 개발에서
" **플랫폼 독립적, 네트워크 중심, 엔터프라이즈 중심 개발**"로 패러다임 전환을 위해 등장

1. 플랫폼 독립성 해결

- write once, run anywhere
- 소스코드 -> 바이트 코드
- JVM
- 윈도우, Linux, Unix

2. 메모리 관리 자동화

- Garbage Collection (GC) 도입
- 개발자가 메모리 해제 신경 쓸 필요 없음
- 메모리 누수 감소
- 서버 장기 실행에 안정적

3. 강력한 보안 모델

- 포인트 없음
- 샌드 박스 실행 모델
- 클래스 로딩 시 검증

4. 네트워크를 기본 전제로 설계

- java.net 패키지 제공
- socket, http, URL 추상화
- 예외 기반 오류 처리
- 웹서버, WAS, 분산 시스템으로 확장

5. 대규모 엔터프라이즈 개발에 적합

- 강제적 OOP
- 명확한 타입 시스템
- 패키지 구조
- 멀티스레딩 표준화
- 비동기 처리 기반의 대용량 처리 지원

Java의 역사

- Sun Microsystems의 제임스 고슬링(James Gosling)과 팀이 가전 제품용 언어 개발 시작
  (2010년 Oracle이 SUN 인수하여, Java의 상품권, 저작권, 브랜드가 Oracle로 이전)
- 1995년 Java 1.0 공개 , Write Once, Run Anywhere
- 1998년 Java 2 (J2SE 1.2) - Swing UI 도입, JVM 안정성 강화
- 2018년 Java 11 - Oracle이 LTS (Long Term Support) 정책 발표: 6개월마다 새 버전, 3년마다 LTS
- 2025년 기준
  - 최신 일반 버전 (GA) : Java SE 25,
  - LTS 버전: Java SE 21

SE (standard Edition) : 자바 언어의 기준이 되는 표준 플랫폼
EE (Enterprise Edition): WAS 같은 기업용 서버의 표준 사양 (Jakarta EE로 이전)

GA (General Availability): 정식 릴리즈. 검증된 안정 버전으로 새로운 기능이나 변경 사항을 포함. 최신 기술을 빠르게 경험하고 싶은 개발자용
일반적으로 다음버전까지 6개월 단기 지원 (신규 기능이라도 deprecated 될 수 있음)

LTS (Long Term Support): 장기지원 버전. 몇년간 안정적이고 지속적인 보안/버그 패치, 호환성 검증된 버전이며, 기업, 프로덕션 환경용

Interpreter 와 Compiler

소스 코드를 기계가 이해할 수 있는 기계어로 변환 파일을 여기 저기서 바로 실행

소스 코드를 한 줄(또는 일정 단위)씩 읽어서 해석 (Interpret) → 기계어 변환 → 실행

Compiler	Interpreter
프로그램 전체를 먼저 변환	실행하면서 조금씩 변환
실행 전에 완료	실행 중 계속 수행
한 번만 컴파일	실행할 때마다 해석 및 변환

ByteCode : .pyc, .class

JIT (Just In Time)

Compiler의 기기 의존 관계 → 컨테이너로 해결

[참고] 다양한 언어

언어	방식	Bytecode	JIT	실행 방식
C	컴파일	X	X	Native 실행
C++	컴파일	X	X	Native 실행
Rust	컴파일	X	X	Native 실행
Go	컴파일	X	X	Native 실행
Java	혼합 (.class)	O	O	Bytecode → JIT → Native
Python	혼합 (.pyc)	O	X 외부 C/C++ Lib 호출	Bytecode → Python VM
JavaScript	혼합 (실행시 Bytecode)	O	O	Bytecode → JIT → Native

언어별 Position

역할	언어
엔터프라이즈 비즈니스 로직	Java, kotlin
클라우드 인프라 / 플랫폼	Go
시스템 핵심 / 고성능	Rust
AI / 데이터	Python

Java의 특징

- **객체 지향적인 언어 :** 모든 것을 객체로 표현하고, Object 클래스에서 모든 클래스가 파생
- **플랫폼 독립적 :** 자바 가상 머신(JVM) 위에서 동작하기 때문에 OS에 종속적이지 않음
- **메모리 관리 :** Garbage collector가 존재하기 때문에 개발자가 직접 메모리 할당/해제를 해주지 않아도 됨
- **풍부한 라이브러리** : 방대한 표준 라이브러리(Java API)를 제공 - 네트워크, 데이터베이스 연결, GUI 개발 등

Java 프로그램의 구조

- 클래스(Class)를 기본 단위로 구성
- 가장 간단한 자바 프로그램도 최소 하나 이상의 클래스를 포함

```
// 패키지 선언 (선택 사항)
package edu.skala;

// 클래스 선언
public class HelloSkala {

    // main 메서드 (프로그램의 시작점)
    public static void main(String[] args) {
        // 실행할 코드 작성
        System.out.println("Hello, Skala!");
    }

    // 다른 메서드들 (선택 사항)
    public void anotherMethod() {
        // ...
    }
}
```

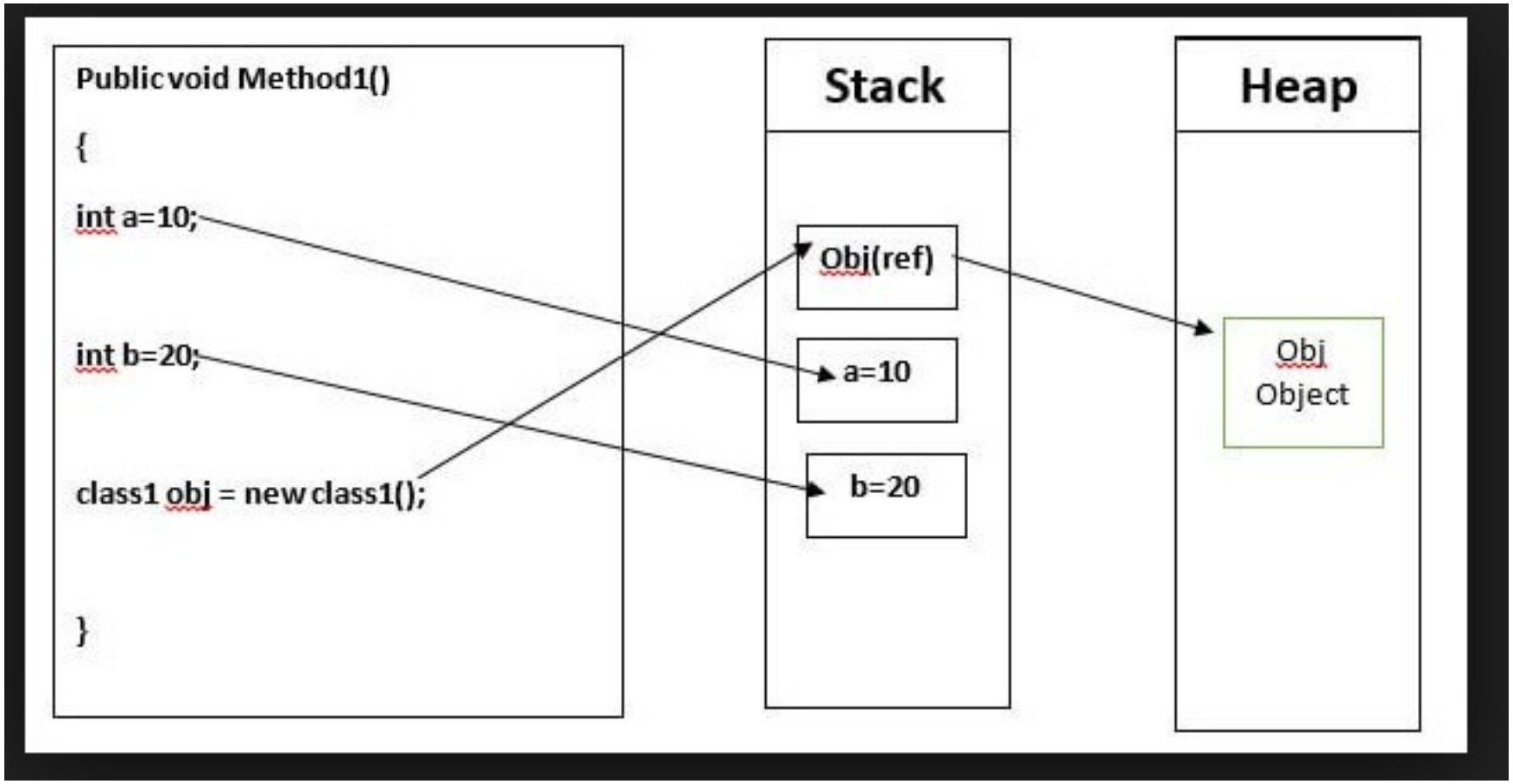
Java 메모리 구조

- **Stack 영역**
  - 메서드 호출 정보와 지역 변수(Local Variables), 기본형(Primitive type) 변수가 저장
  - 메서드가 호출될 때마다 하나의 프레임(Frame)이 스택에 쌓이고, 메서드 실행이 끝나면 해당 프레임이 제거됨.
  - 스레드마다 별도의 스택 영역을 가짐.

- **Heap 영역**
  - 객체(Object)와 배열(Array)이 저장되는 공간
  - 프로그램 실행 중 생성되는 거의 모든 데이터가 이곳에 저장됨.
  - 여러 스레드가 공유하는 영역
  - 가비지 컬렉터(GC)가 주로 활동하는 영역으로, 더 이상 사용되지 않는 객체를 찾아내 메모리에서 제거

```
void method() {
    int a = 10;               // Stack (지역 변수)
    String s = "Hello";       // Stack (참조), 문자열은 Heap의 상수 풀
    Person p = new Person();  // p는 Stack, 객체는 Heap
}
```

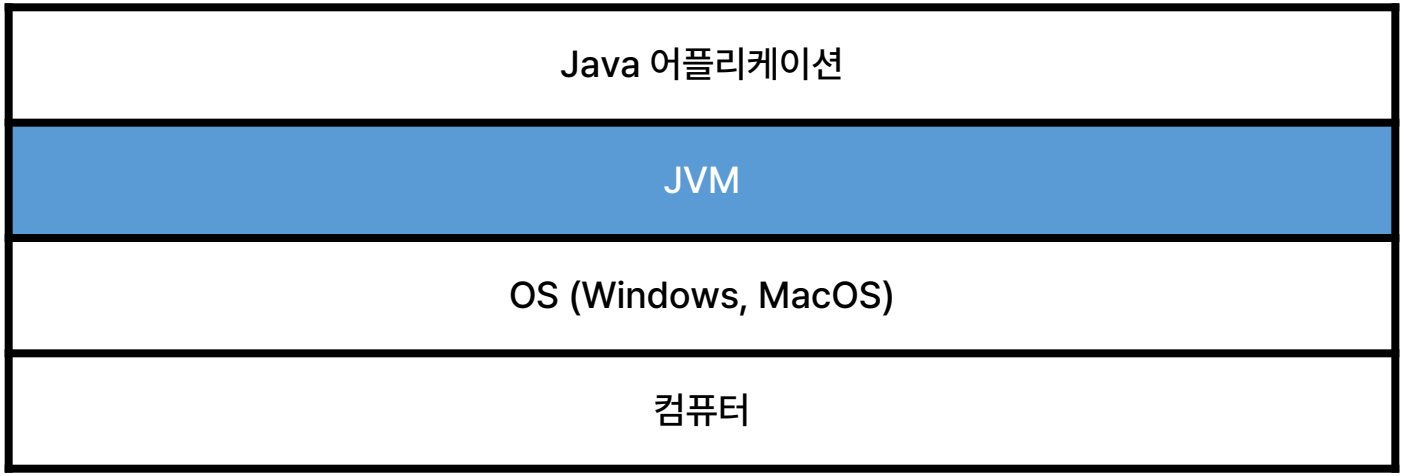
[참고] 메모리 Stack vs Heap



자바와 JVM

JVM = Java Vitrual Machine = 자바를 실행하기 위한 가상 머신

Java 어플리케이션은 JVM 위에서 동작하기 때문에 OS에 종속적이지 않음



Java 파일과 컴파일

- **.java 파일** : 소스 코드 파일 - 개발자가 작성하는 텍스트 기반의 Java 프로그램 소스 코드
- **.class 파일** : 바이트코드 파일 (컴파일 결과물) - JVM이 읽고 실행할 수 있는 중간 언어로 변환 (플랫폼 독립적)
- **JVM (Java Virtual Machine)**: 실행 엔진 – 바이트코드(class 파일)을 실행하는 가상 머신

Skala.java

사람 친화적

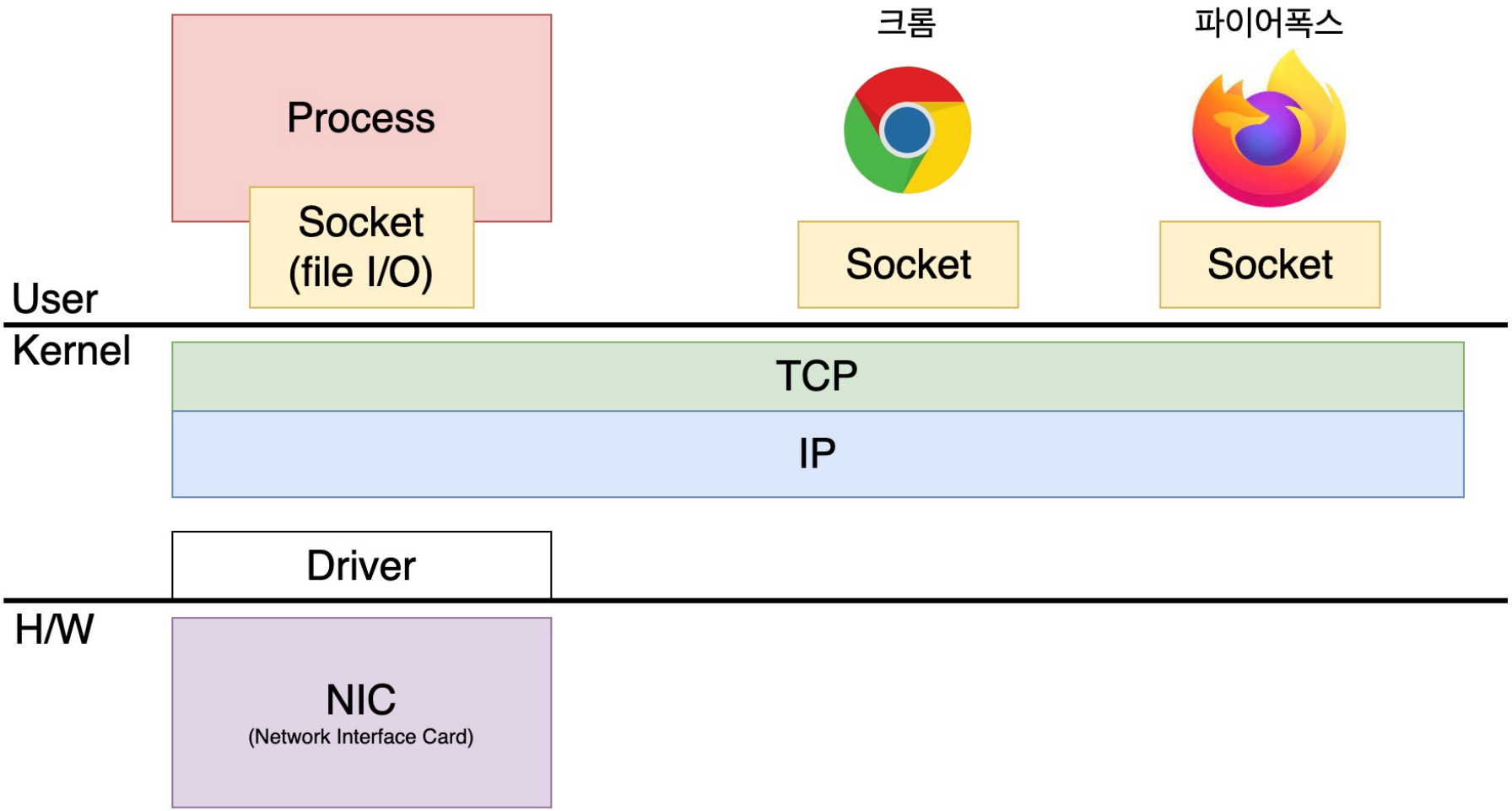
Skala.class

JVM 친화적

Program

CPU 친화적

[참고] IP vs Port 의미



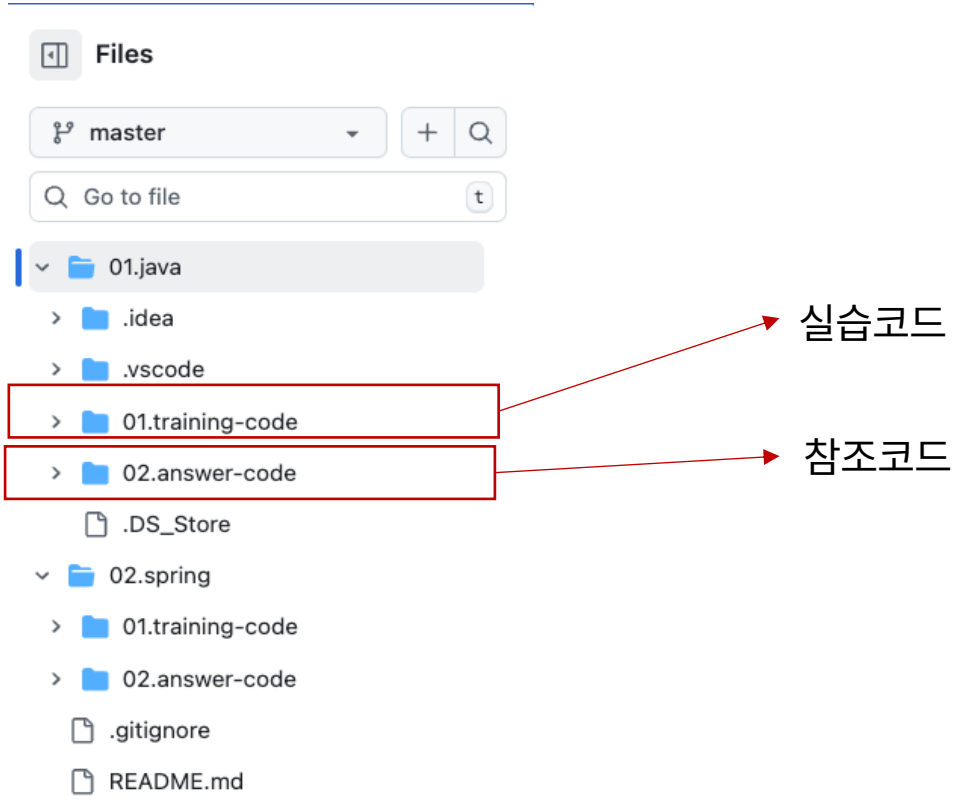
개발환경 구성

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

실습 코드 다운로드

```
cd ~
mkdir workspace
cd workspace
git clone  https://github.com/himang10/skala-java-springboot.git
cd skala-java-spring
```



확인 사항

git clone 시 skala-java-spring 디렉토리 내에
.git 이 존재

개인 repository를 만들어서 함께 git에 push 하는 경우에는
기존 .git 을 삭제 필요

필수 프로그램 설치

교육을 위한 프로그래 설치

arm64 버전 (macbook silicon chip 전용)

```
brew install -y jq  curl maven gradle node git kubectl
```

Java 21 프로그램 설치

교육을 위한 프로그래 설치

❖ arm64 버전 (macbook silicon chip 전용)

```
brew install openjdk@21
java -version

export JAVA_HOME=/opt/homebrew/opt/openjdk@21
```

```
echo "export JAVA_HOME=/opt/homebrew/opt/openjdk@21" >> ~/.zshrc
source   ~/.zshrc
```

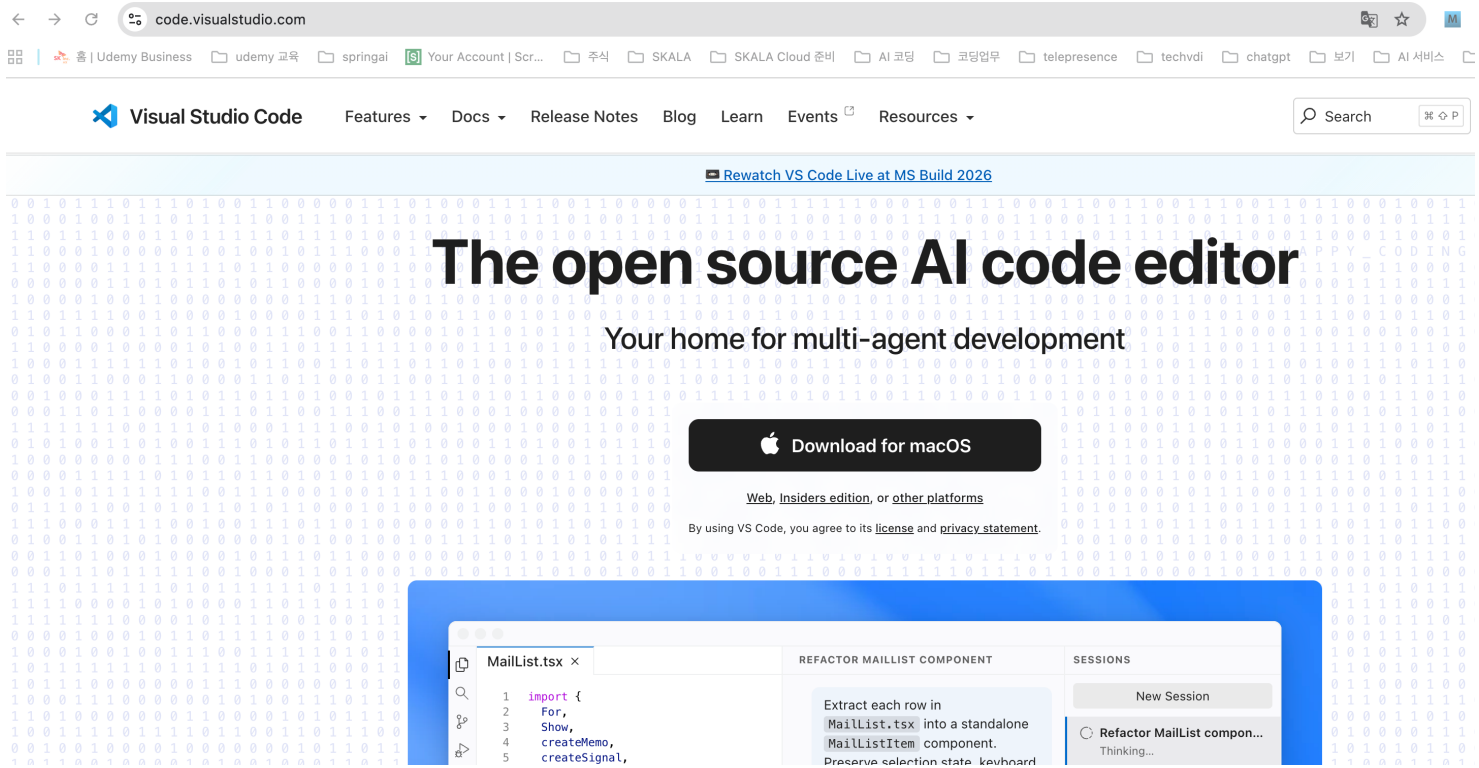
IDE(Integrated Development Environment)

- **개발자가 코딩, 컴파일, 디버깅을 개별 프로그램으로 진행하던 시절(?) → 하나의 프로그램에서 통합 제공**
- **주요 IDE (2001년 이후)**
  - Visual Studio Code (VS Code)
  - IntelliJ (Java 개발)
  - Visual Studio (C#, .NET 개발)
  - Eclipse (Java 개발)
  - PyCharm (Python 개발)

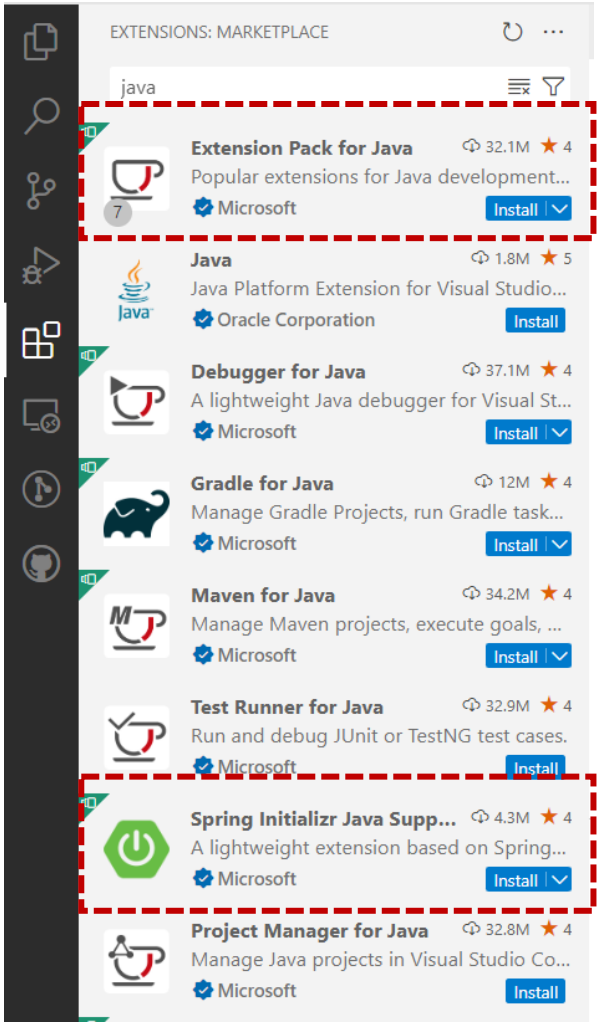
- **AI 통합**
  - VS Code 기반의 GitHub Copilot, ChatGTP Codex, Claude Code

VS Code

- Visual Studio Code(VS Code)는 마이크로소프트에서 개발한 오픈 소스 기반의 코드 편집기
- 다양한 프로그래밍 언어를 지원하며 가볍고 빠르면서도 강력한 기능들을 제공
- https://code.visualstudio.com 접속 후 "Download for macOS" 로 설치



VS Code Java Extension 추가



좌측 Extensions 메뉴에서 “java” 필터 후 설치

- Extension Pack for Java
- Spring Initializr Java Support

Shell Command : code 설정

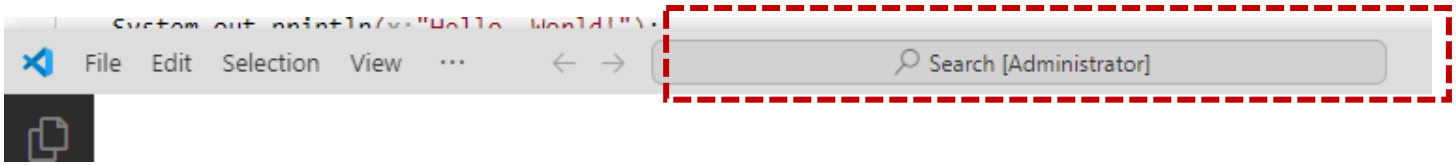
VSCODE 실행 후

1. 단축 키 : Command + Shift + P를 눌러 Command Palette 를 열어줌
2. Shell Command 입력창에 code 입력
3. Shell Command: Install ‘code’ command in Path를 선택

현재 디렉토리에서 vscode 실행 방법

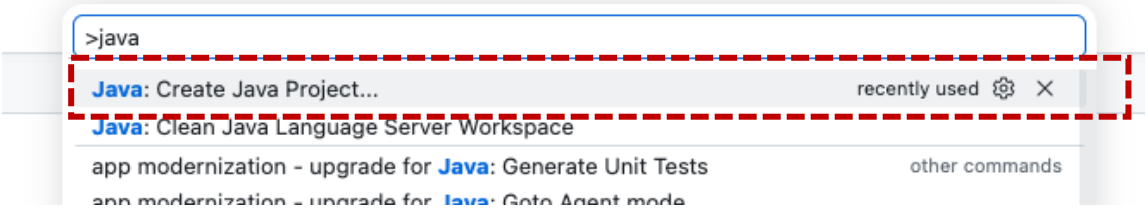
```
> code .
```

Java 프로젝트 생성

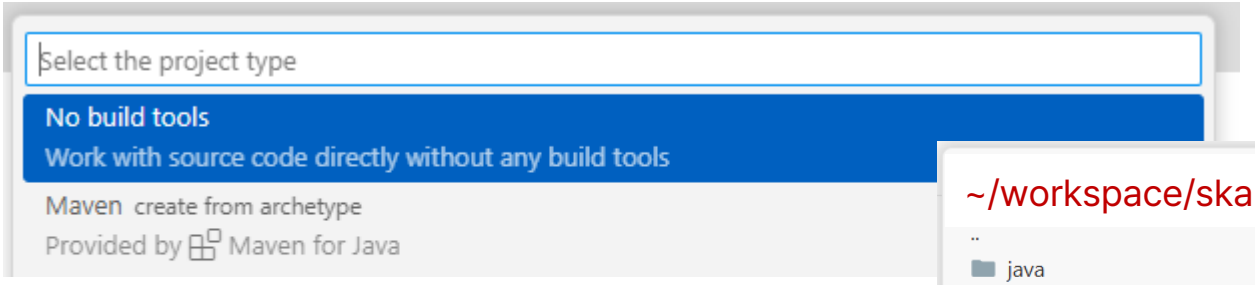


Search에 “>java” 입력 후 프로젝트 생성 선택

- Java: Create Java Project...



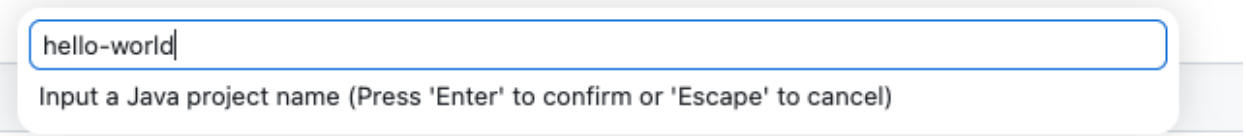
No build tools 선택



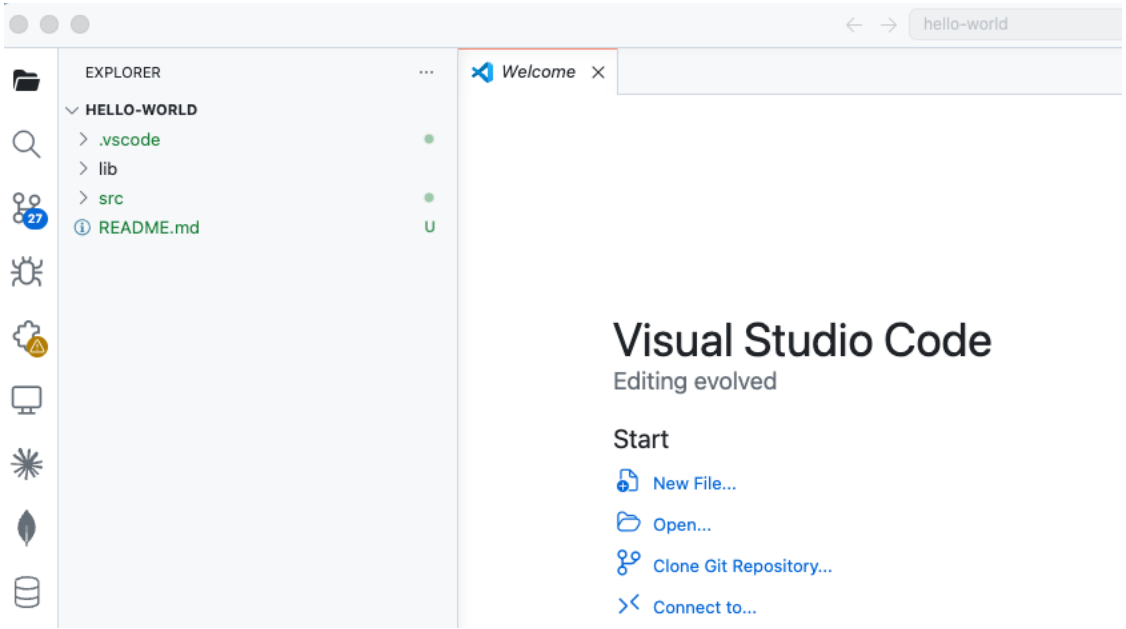
프로젝트 폴더 상위 폴더(java)를 선택

Java 프로젝트 생성

프로젝트 이름을 입력 : hello-world

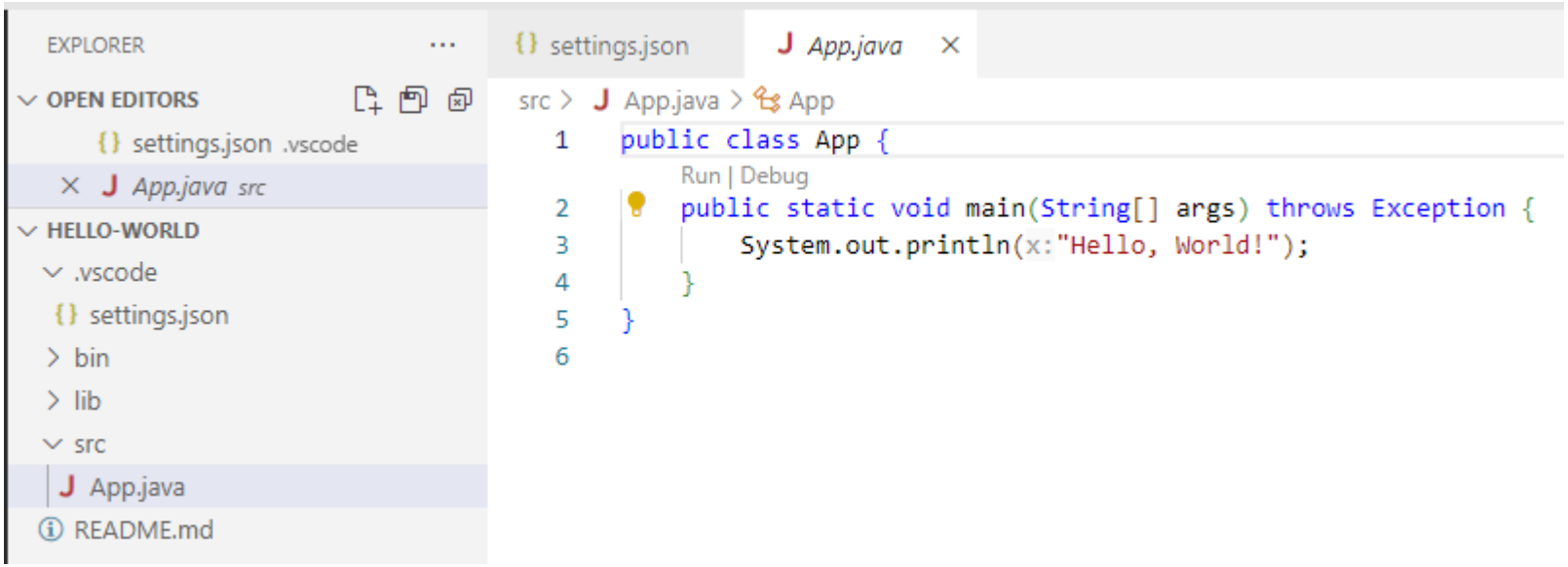


hello-world project 생성



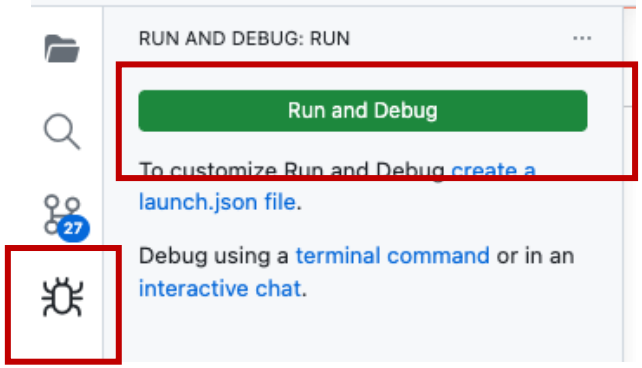
소스 코드 보기: Hello World

1. src/App.java 선택



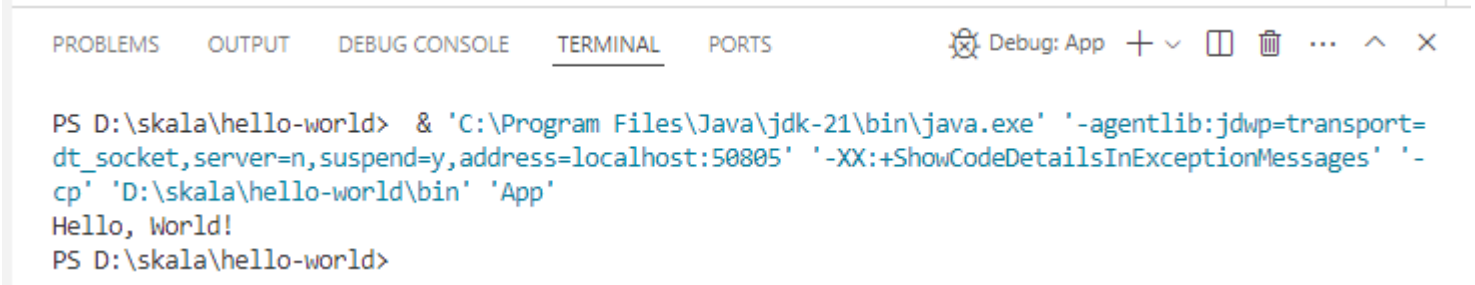
코드 실행하기: Hello World

2. Run And Debug 메뉴에서 “Run and Debug” 버튼 클릭



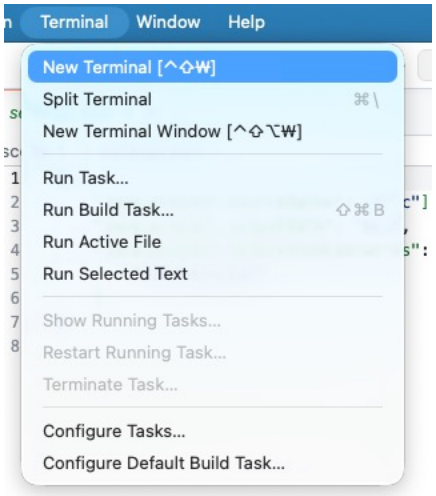
클릭

3. 하단에 “터미널”이 열리면서 “Hello, World” 출력



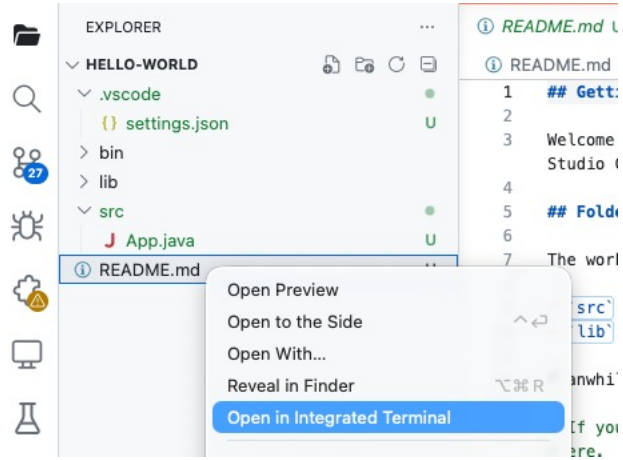
CLI로 실행해보기

- New Terminal 오픈



또는 README.md 오른쪽 마우스 버튼 클릭

"Open In Integrated Terminal 선택 "



- 하단 Terminal에서 다음 명령 시행

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

(*|skala-2025:harbor)
~/mydev/skala/skala-cloud/skala-java-spring/mycode/hello-world/ [skala/step4.1*] javac -d bin src/App.java
(*|skala-2025:harbor)
~/mydev/skala/skala-cloud/skala-java-spring/mycode/hello-world/ [skala/step4.1*] java -cp bin App
Hello, World!
(*|skala-2025:harbor)
~/mydev/skala/skala-cloud/skala-java-spring/mycode/hello-world/ [skala/step4.1*]
```

javac: java Compiler

java: java 실행

parameter tage

-d: class 저장 디렉토리

-cp : Class Path

프로젝트 설정 정보 (src, bin 폴더의 구분)

프로젝트 설정 정보 확인: src, bin, lib

Java 프로그램의 구조

- 클래스(Class)를 기본 단위로 구성
- 가장 간단한 자바 프로그램도 최소 하나 이상의 클래스를 포함

```
// 클래스 선언
public class HelloSkala {

    // main 메서드 (프로그램의 시작점)
    public static void main(String[] args) {
        // 실행할 코드 작성
        System.out.println("Hello, Skala!");
    }
}
```

구성 요소	역할
클래스 (class)	프로그램 단위 구성 블록
main() 메서드	자바 프로그램 실행의 시작점
출력문 (System.out.println)	텍스트를 콘솔에 출력

[실습] App.java 수정

- App.java에 "스칼라에 오신 것을" + 1초 휴식 + "환영합니다." 가 화면에 프린트되도록 수정

```
public static void main(String[] args) throws Exception {
        //System.out.println("스칼라에 오신것을 환여합니다");
        System.out.print("스칼라에 오신것을 ");
        Thread.sleep(1000);
        System.out.println("환영합니다");
    }
```

- RUN and Debug로 실행해보기
- Terminal을 한개 열어서 CLI로 컴파일 및 실행해보기

[참고] Thread 클래스 : 멀티스레딩 프로그래밍에서 스레드 생성 및 제어를 지원

- **Thread.sleep(long millis)** : 지정된 시간 동안 스레드를 일시 중단 (유틸리티)
- Thread t = new Thread(() -> System.out.println("Hello Thread")); t.start();

자바 변수와 자료형

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

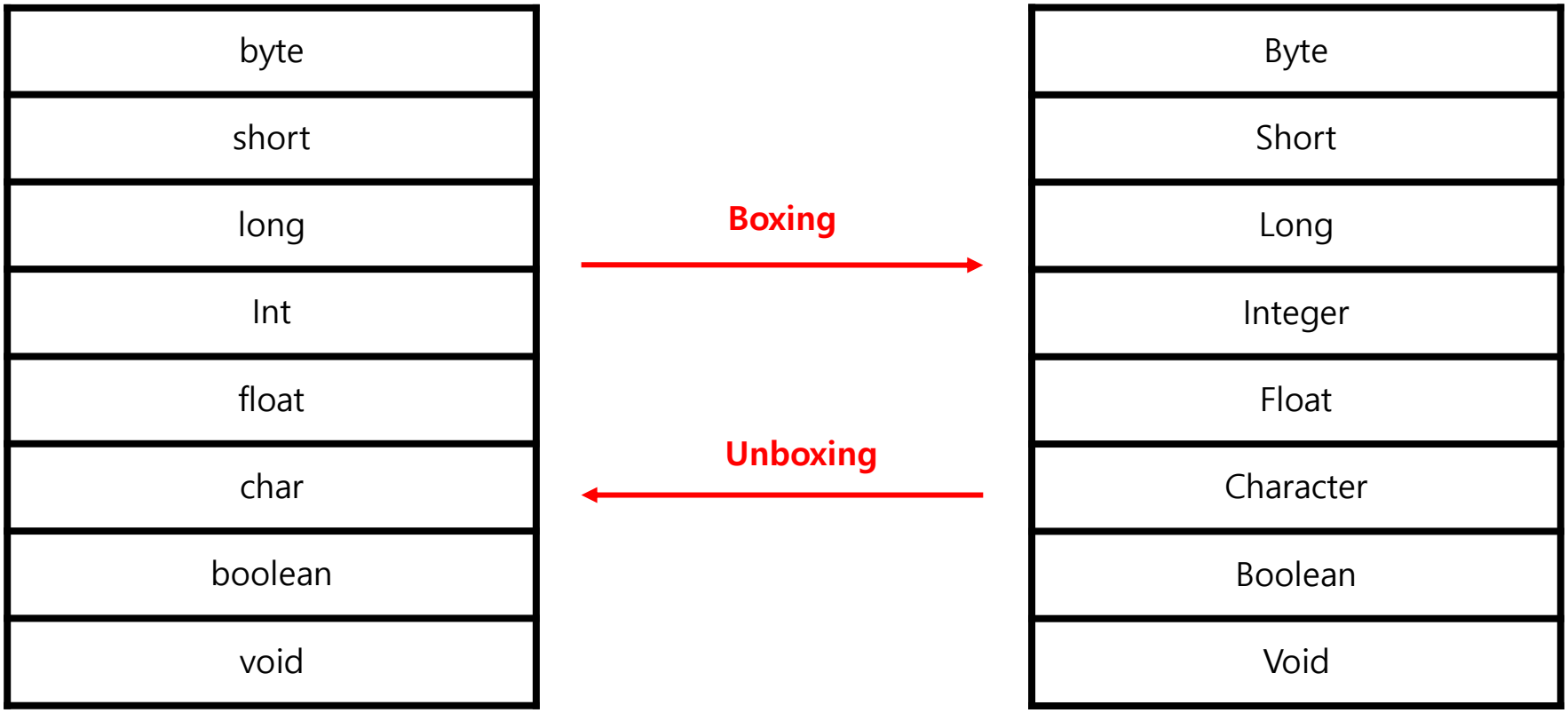
자바 변수와 자료형

- 기본형(Primitive)와 참조형(Reference)로 구분

분류	자료형	크기 (byte)	기본값	설명	예시 코드
기본형	byte	1	0	정수형, -128 ~ 127	byte b = 100;
	short	2	0	정수형, -32,768 ~ 32,767	short s = 30000;
	int	4	0	기본 정수형	int i = 100000;
	long	8	0L	큰 범위의 정수형, 접미사 L 필요	long l = 10000000000L;
	float	4	0.0f	실수형, 접미사 f 필요	float f = 3.14f;
	double	8	0.0	기본 실수형	double d = 3.141592;
	char	2	'₩u0000'	문자형, 유니코드 사용	char c = 'A';
	boolean	1 (논리적)	false	true 또는 false	boolean flag = true;
참조형	String	-	null	문자열 객체 (클래스)	String msg = "Hello";
	배열 ([])	-	null	같은 타입의 집합	int[] arr = {1, 2, 3};
	사용자 정의 클래스	-	null	객체 생성 필요	Student s = new Student();

기본형과 참조형 래퍼(Wrapper) 클래스

- 기본 타입을 래퍼 클래스로 변환하는 과정을 boxing이라고 하며, 반대를 unboxing이라고 함
- 기본형과 래퍼 클래스는 자동으로 boxing 및 unboxing 처리됨.




```
int primitive = 10;
Integer wrapper = primitive;
```

```
Integer wrapper = 100;
int primitive = wrapper;
```

[참고] 기본형 저장 위치

기본형은 객체가 아니기때문에 실행 컨텍스트에 따라 저장하는 위치가 달라질 수 있음

```
void foo() {
    int a = 10;
    long b = 20L;
}
```

- Stack Frame의 Local Vafriables 영역

```
class A {
    int x;
}
A a = new A();
```

- new A()는 Heap
- x attribute는 Heap
- a는 stack

```
int c = a + b;
```

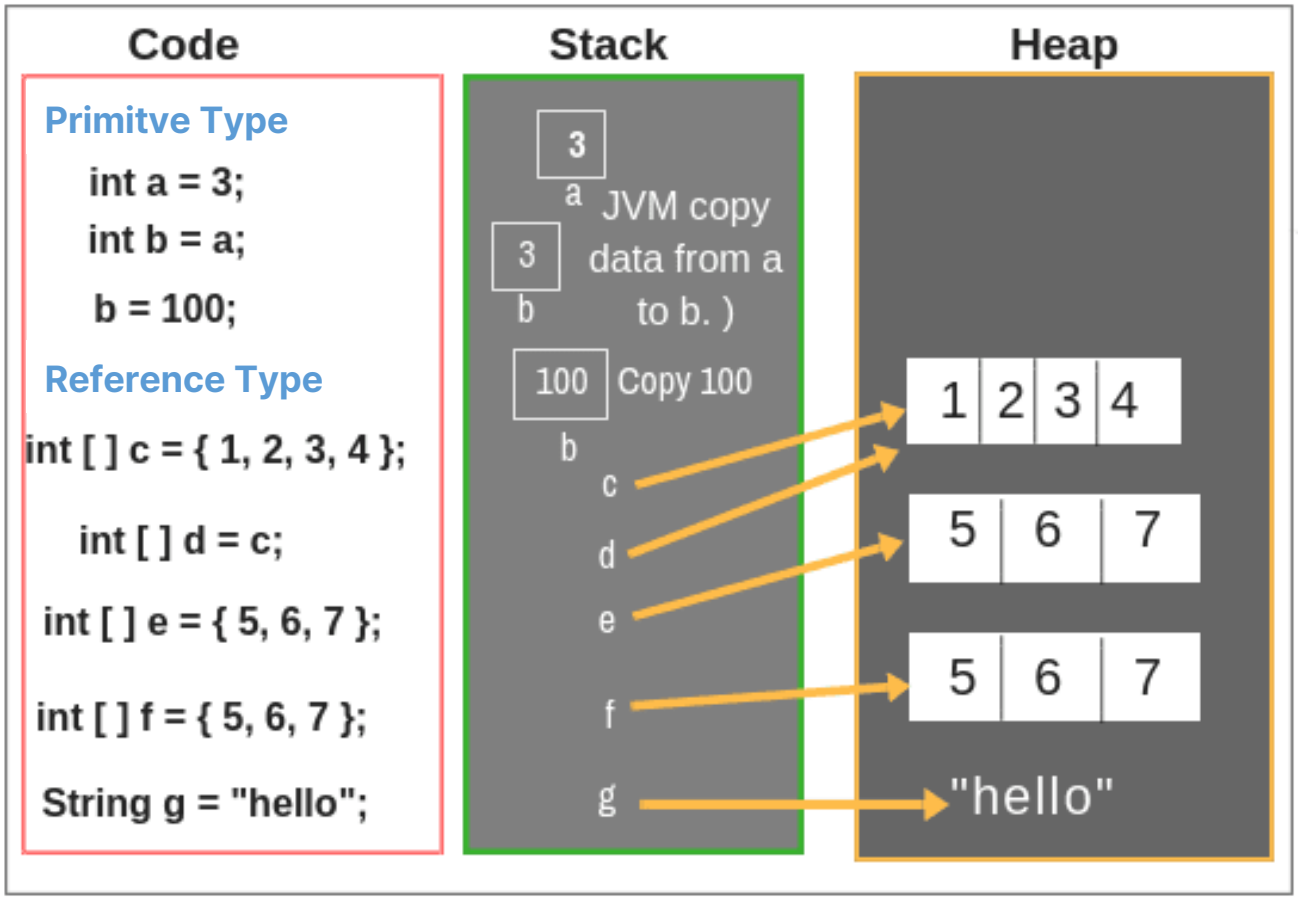
- a, b는 Operand Stack 으로 복사되어 계산
- c는 Local Variables

```
class A {
    static int count;
}
```

- 클래스 메타 데이터는 Metaspace,
  java.lang.Class 객체는 Heap
- static int count는 Class 객체 내 Heap

[참고] 기본형과 참조형의 저장 방식

데이터 타입	저장 위치	메모리 구조 설명
Primitive 타입	스택(Stack)	고정된 크기의 값이므로 스택에 저장 가능
Reference 타입	힙(Heap)	동적 크기를 가지므로 힙에 객체가 저장됨
Reference 변수 자체	스택(Stack)	참조값(주소)이 저장됨



[참고] 참조형 변수의 불변 객체(Immutable Object)

- 한 번 생성된 후 **내부 상태(필드 값)가 절대 변경되지 않는 객체**
- 값을 변경하는 것처럼 보이는 연산은 **새로운 객체를 생성**해서 반환
- Java의 Wrapper Class인 Integer, Float, Double, Long, Character, Boolean, String 모두 불변 객체

Call by Value

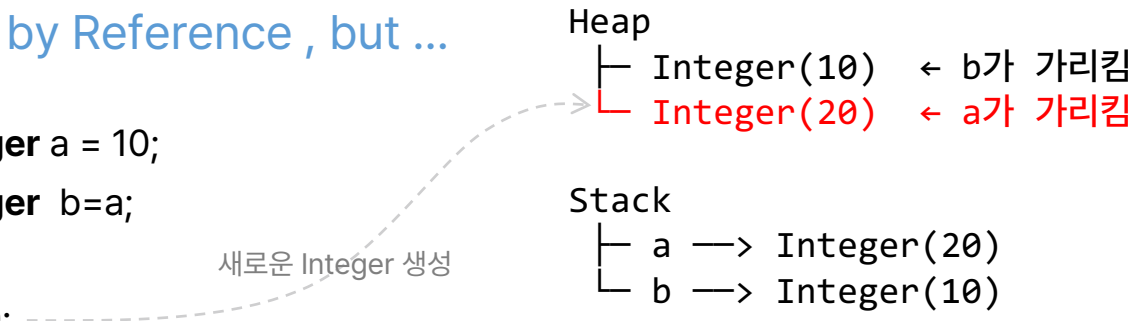
```
int a = 10;
int  b=a;

a=20;
System.out.printf("a=%d b=%d", a, b);
```

결과

a=20 b=10

Call by Reference , but ...

```
Integer a = 10;
Integer  b=a;

a=20;
System.out.printf("a=%d b=%d", a, b);
```



결과는?

a=20 b=20 or **a=20 b=10**

자바의 키워드

- 언어의 문법을 구성하는 예약어(reserved words)나 변수 및 메서드 이름 등으로 사용

제어문/흐름	클래스/객체 관련	자료형/리터럴	접근/제어	기타 키워드
if	class	int	public	package
else	interface	double	private	import
switch	enum	boolean	protected	return
case	extends	char	static	new
default	implements	byte	final	this
while	instanceof	short	abstract	super
do	new	long	synchronized	throws
for	super	float	transient	throw
break	this	void	volatile	try
continue	null (리터럴)			catch
return				finally
yield (Java 14+)				assert
record (Java 16+)				goto*
				const*

❖ **yield는 switch expression 여러줄 return 시 return 대신 사용**
❖ **record는 불변 데이터 전용 클래스 (DTO 작성시 반복코드 제거)**
❖ **goto 와 const는 Java에서 사용되지 않지만, 예약어로 지정되어 있어 사용 불가**

[실습] NumberBox.java

참고: 01.training-code/01.snippet/NumberBox.java

- 기본형 int a와 b는 100으로 입력
- 래퍼 클래스 Integer A와 B도 100으로 입력되었을 때 서로 비교(==, equals())

```
public class NumberBox {
    public static void main(String[] args) {
        int a = 100;
        int b = 100;

        Integer A = 100;
        Integer B = 100;

        // 기본형 비교 (값 비교)
        System.out.println("a == b: " + (a == b));

        // 래퍼 클래스 비교 (주소 비교)
        System.out.println("A == B: " + (A == B));

        // 래퍼 클래스 equals()로 비교 (값 비교)
        System.out.println("A.equals(B): " + A.equals(B));
    }
}
```

실행

```
> javac -d bin NumberBox.java
> java -cp bin NumberBox
```

```
a == b: true
A == B: false
A.equals(B): true
```

자바 연산자

분류	연산자 예시	설명
산술	+, -, *, /, %	덧셈, 뺄셈, 곱셈, 나눗셈, 나머지 연산
비교	==, !=, <, >, <=, >=	두 값의 크기나 동등성 비교
논리	&&, `	
대입	=, +=, -=, *=, /=, %=	값을 변수에 저장하거나 연산 후 대입
증감	++, --	값 1 증가 또는 감소 (전위, 후위 모두 가능)
삼항	조건 ? 참값 : 거짓값	조건에 따라 값을 선택하는 연산
비트	&, `	, ^, ~, <<, >>, >>>`
문자열 연결	+	문자열을 이어 붙일 때 사용 (String str = "Hello" + " World";)

자바 연산자 – 산술 연산

```
int a = 10;
int b = 3;

System.out.println(a + b); // 13
System.out.println(a % b); // 1
```

자바 연산자 – 비교 연산

```
int a = 10;
int b = 3;

System.out.println(a > b);   // true
System.out.println(a == 10); // true
```

자바 연산자 – 논리 연산

```
boolean x = true;
boolean y = false;

System.out.println(x && y); // false
System.out.println(x || y); // true
System.out.println(!x);     // false
```

❖ **논리 연산자 && 와 || 의 단축 평가**

&&: 앞의 조건이 false면 뒤 조건을 보지 않음

||: 앞의 조건이 true면 뒤 조건을 보지 않음

자바 연산자 – 대입 및 복합 대입 연산

```
int num = 5;
num += 2; // num = num + 2;


System.out.println(num); // 7
```

자바 연산자 – 증감 연산 (전위/후위 차이)

```
int i = 5;


System.out.println(++i);
System.out.println(i++);
System.out.println(i);
```

각 단계별 프린트 값은 ?

자바 연산자 – 삼항 연산

```
int score = 85;


String result = (score >= 80) ? "합격" : "불합격";
System.out.println(result);
```

프린트 결과는 ?

자바 연산자 – 문자열 연결

```
String name = "스칼라";


System.out.println("안녕하세요, " + name + "님!");
```

프린트 결과는 ?

문자열 포맷터 – String.format(), printf()

서식	설명
%s	문자열
%d	10진수 정수
%f	실수 (소수점 기본 6자리)
%c	문자
%b	boolean 값
%%	% 자체

```
String name = "스칼라";
int age = 30;
String formatted = String.format("이름: %s, 나이: %d", name, age);
System.out.println(formatted);  // 이름: 스칼라, 나이: 30
```

```
double pi = 3.141592;
System.out.println(String.format("원주율: %.2f", pi));  // 원주율: 3.14
```

```
System.out.printf("|%10s|\n", "Java");  // 오른쪽 정렬 (총 10자리)
System.out.printf("|%-10s|\n", "Java"); // 왼쪽 정렬

|      Java|
|Java      |
```

```
System.out.printf("%05d\n", 42);    // 00042
```

[실습] FormatterExample

01.training/01.snipset/FormatterExample.java

- String.format()을 직접 작성하고 테스트해보자

FormatterExample.java

```
public class FormatterExample {
    public static void main(String[] args) {
        String name = "스칼라";
        int age = 30;
        String formatted = String.format("이름: %s, 나이: %d", name, age);
        System.out.println(formatted);  // 이름: 스칼라, 나이: 30

        double pi = 3.141592;
        System.out.println(String.format("원주율: %.2f", pi));  // 원주율: 3.14
        System.out.printf("|%10s|\n", "Java");  // 오른쪽 정렬 (총 10자리)
        System.out.printf("|%-10s|\n", "Java"); // 왼쪽 정렬

    }
}
```

실행하기

```
> java FormatterExample.java

이름: 스칼라, 나이: 30
원주율: 3.14
|      Java|
|Java      |
```

문자열 포맷터 – StringBuilder

- 문자열을 빠르고 효율적으로 조립(append) 할 수 있는 클래스
- 반복적이거나 동적으로 문자열을 생성할 때 매우 유용하며, 포맷 효과를 수동으로 구성

```
String name = "스칼라";
int age = 25;

StringBuilder sb = new StringBuilder();
sb.append("이름: ").append(name)
  .append(", 나이: ").append(age);

System.out.println(sb.toString());    // 출력: 이름: 스칼라, 나이: 25
```

```
StringBuilder sb = new StringBuilder();
sb.append(String.format("%-10s", "스칼라"))  // 왼쪽 정렬
  .append(String.format("%5d", 25));         // 오른쪽 정렬

System.out.println(sb.toString());           // 출력 결과: 스칼라         25
```

Method Chaing , Fluent API 방식

[실습] StringBuilderExample

01.training-code/01.snipset/StringBuilderExample.java

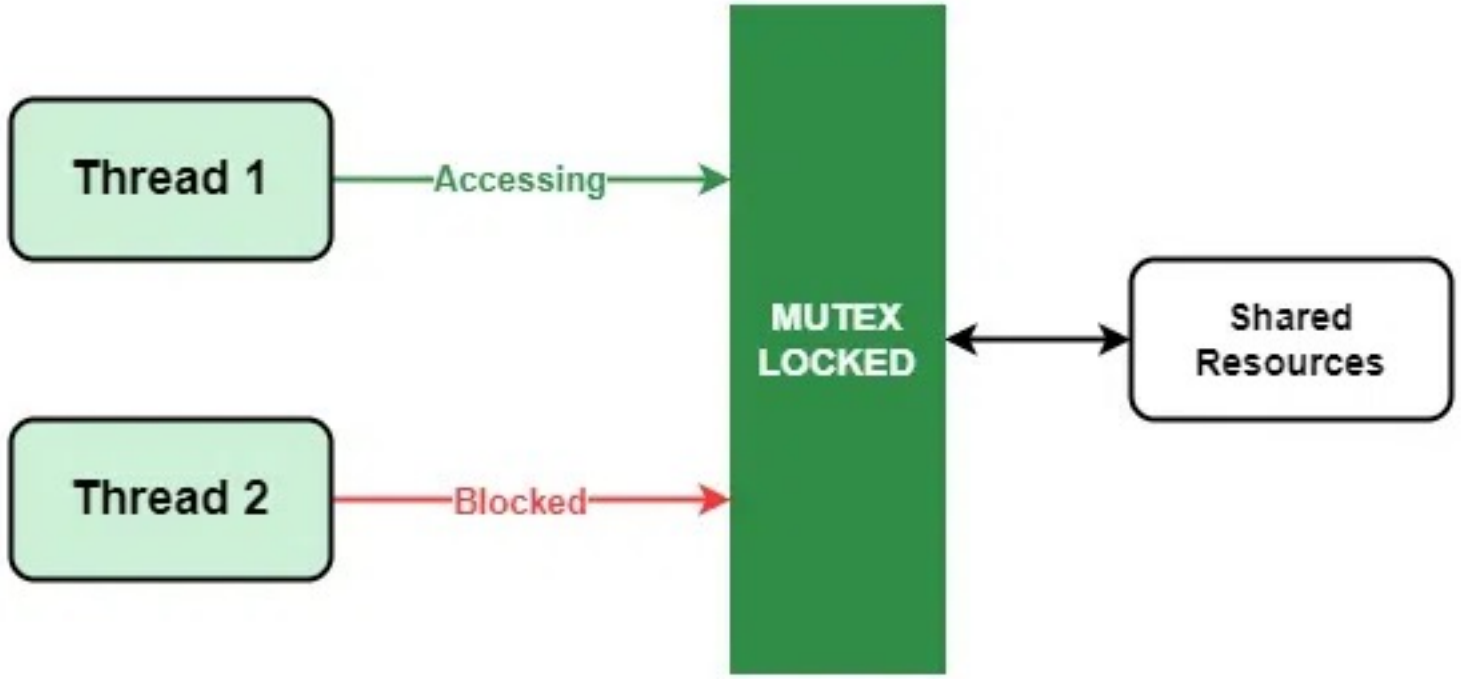
- 앞서 교육했던 샘플을 직접 작성해서 동작 상태를 확인해보자
- 실행을 통해 String이 어떻게 구성되는지 확인해보자

StringBuilderExample.java

```
public class StringBuilderExample {
    public static void main(String[] args) {
        String name = "스칼라";
        int age = 25;

        StringBuilder sb1 = new StringBuilder();
        sb1.append("이름: ").append(name)
        .append(", 나이: ").append(age);

        System.out.println(sb1.toString());


        StringBuilder sb2 = new StringBuilder();
        sb2.append(String.format("%-10s", "스칼라"))  // 왼쪽 정렬
        .append(String.format("%5d", 25));            // 오른쪽 정렬

        System.out.println(sb2.toString());          // 출력 결과: 스칼라       25

    }
}
```

Mutex (상호 배제, Mutual Extension)

- Multi-Thread 환경에서 “공유 자원에는 한 번에 하나의 Thread 만 접근해야 한다”는 개념을 구현



StringBuilder vs StringBuffer

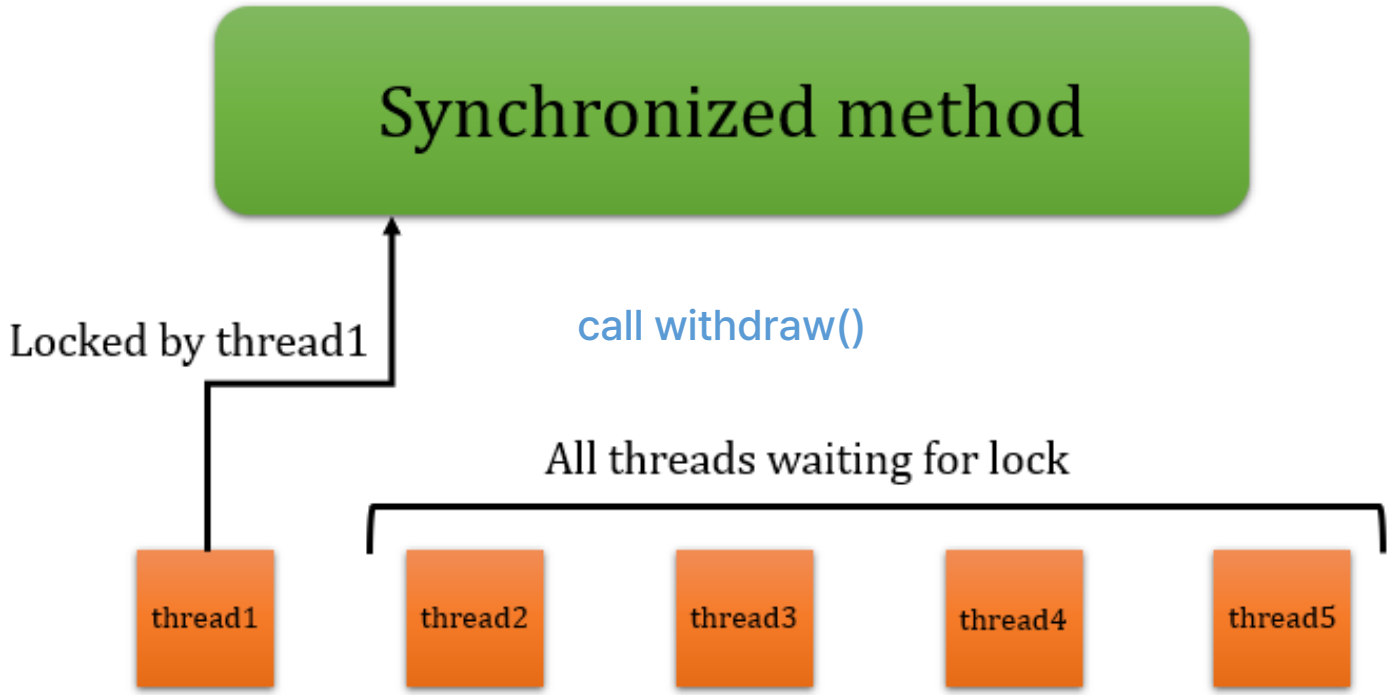
- 두가지는 모두 사용법은 완전히 동일
- StringBuffer는 내부적으로 메서드가 synchronized 처리되어 여러 스레드가 동시에 접근해도 안전
  - StringBuilder는 StringBuffer보다 최대 2~3배 빠름 (동기화 오버헤드 없음)
  - 하지만 멀티스레드 환경에서는 동기화가 없는 StringBuilder는 데이터 손상 위험 존재

항목	StringBuilder	StringBuffer
스레드 안전성	스레드 안전하지 않음	**스레드 안전함 (synchronized)**
성능	빠름 (단일 스레드 환경에서 권장)	느림 (동기화 오버헤드 있음)
사용 시점	단일 스레드 또는 동기화 불필요한 경우 사용	멀티 스레드 환경에서 동시 접근이 필요한 경우
도입 시기	Java 5 이후	Java 초창기부터
대체 관계	StringBuffer의 비동기화 버전	StringBuilder보다 안정적이나 느림
기본 메서드	append(), insert(), delete(), toString() 등	동일

Mutex Lock 구현제 Synchronized

- synchronized 키워드는 멀티 쓰레드 환경에서 동기화를 보장하기 위한 기능
- 공유 자원에 여러 쓰레드가 동시에 접근할때 데이터 불일치를 차단

```
public synchronized void withdraw(int amount) {
   …
}
```



[실습] SynchronizedExample.java

01.training-code/01.snipset/SynchronizedExample.java

synchronized 기능을 이용해서 Thread t1 과 Thread t2 가 동시 withdraw 접속 시 순차 처리 확인하기

```
class BankAccount {
    private int balance = 1000; // 잔액

    public synchronized void withdraw(int amount) {
        if (balance >= amount) {
            System.out.println(Thread.currentThread().getName() + " 출금 시도: " + amount);
            try {
                Thread.sleep(3000); // 출금 처리 지연(동시성 문제 시뮬레이션)
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
            balance -= amount;
            System.out.println(Thread.currentThread().getName() + " 출금 완료. 남은 잔액: " + balance);
        } else {
            System.out.println(Thread.currentThread().getName() + " 출금 실패. 잔액 부족");
        }
    }
}

public class SynchronizedExample {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();

        // 두 개의 스레드가 동시에 출금
        Thread t1 = new Thread(() -> account.withdraw(800), "스레드-1");
        Thread t2 = new Thread(() -> account.withdraw(800), "스레드-2");

        t1.start();
        t2.start();                 //t2.join() 사용 ?
    }
}
```

네이밍 컨벤션

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

네이밍 컨벤션

"컴퓨터 과학의 가장 어려운 문제는 *캐시 무효화, *Off-by-One 오류, 그리고 적절한 이름 짓기다."

- Phil Karlton

- ❖ 캐시와 실제 데이터간의 일관성 문제, 캐시 유지/삭제 시점 문제 등
- ❖ 반복문, 인텍스, 범위 설정 시 1만큼 잘못 계산 오류 (배열 크기 5인데 인덱스를 0 ~5까지 접근하는 실수)

네이밍 컨벤션

- 코드의 품질은 작명에서 시작
- 짧고 명확하며 일관성 있는 네이밍이 중요
- 좋은 이름은 개발자의 사고를 명확히 하고, 코드의 의미를 드러내는 가장 강력한 도구

관점	설명
의사 전달	잘 지은 이름은 주석 없이도 코드의 역할을 자연스럽게 설명합니다.
유지보수	시간이 지나도 이름만 보고 코드의 의도를 쉽게 이해할 수 있습니다.
협업	팀원들이 코드를 읽고 빠르게 파악할 수 있어 협업 효율이 올라갑니다.
디버깅/확장	적절한 네이밍은 추적, 검색, 재사용 시 핵심 단서 역할을 합니다.

나쁜 예시	좋은 예시	설명
int d;	int daysUntilDeadline;	의미 없는 이름 대신 명확한 의미
getData()	getCustomerList()	반환 값의 성격을 명확히 표현
doStuff()	calculateTotalPrice()	기능을 명확하게 표현

네이밍 컨벤션

- 프로그래밍 언어에서 사용하는 Naming Convention 종류

명칭	표기 방식	예시	설명
Camel Case (카멜 표기법)	첫 단어 소문자, 이후 단어는 대문자	myVariableName	Java 변수, 메서드명에 사용
Pascal Case (파스칼 표기법)	모든 단어의 첫 글자를 대문자	MyClassName	Java 클래스, 인터페이스명에 사용
Snake Case (스네이크 표기법)	소문자 단어를 _로 구분	my_variable_name	Python 변수, 데이터베이스 필드명 등
Kebab Case (케밥 표기법)	소문자 단어를 -로 구분	my-variable-name	URL, 파일명 등에서 사용
Upper Snake Case (=Screaming Snake Case)	대문자 단어를 _로 구분	MAX_VALUE	Java 상수(final static)에 사용

참고) kebab (꼬치구이), Screaming (소리를 지르는?)

왜 "-"는 프로그램에서 사용하지 않고, underbar(_)를 사용하나?

Java에서 사용하는 네이밍 규칙

- 클래스/타입명에 PascalCase를, 변수/메서드에는 camelCase를 사용
- 상수(constant)는 항상 대문자 + 언더스코어(SCREAMING_SNAKE_CASE) 로 작성
- 패키지 이름은 모두 소문자로 작성하며 보통 도메인 역순으로 표기

항목	사용 예	네이밍 방식	예시
클래스 이름	클래스 정의 시	PascalCase	HelloWorld, StockManager
인터페이스 이름	인터페이스 정의 시	PascalCase	Runnable, Comparable
메서드 이름	메서드 정의 시	camelCase	calculateBMI(), getName()
변수 이름	지역/멤버 변수	camelCase	studentName, stockCount
상수 이름	static final 상수	Upper Snake Case	MAX_SIZE, DEFAULT_PORT

Java에서 사용되는 CRUD 메서드 명 (동사 + 명사)

- Java에서 메서드 명으로 자주 사용되는 CRUD (조회, 추가, 수정, 삭제) 관련 일반적인 동사 목록
- 동사 + 명사 : getUser() - 사용자를 get한다, loadProfile : 프로파일을 Load한다. ...

동작 구분	대표 메서드 접두어	의미 / 설명	예시 메서드 이름
조회 (Read)	get	단일 항목 조회 (by ID 등)	getUserById()
	find	조건 검색, 존재 여부 포함	findUserByEmail()
	search	다중 조건 검색 (보통 리스트)	searchUsersByKeyword()
	load	지연 로딩 시 사용 (JPA 등)	loadProfile()
추가 (Create)	add	목록에 항목 추가	addItemToCart()
	create	새 객체 생성 및 저장	createUser()
	save	DB에 저장 (create 또는 update 포함)	saveProduct()
수정 (Update)	update	기존 데이터 수정	updatePassword()
	modify	특정 필드 일부 수정	modifyUserRole()
삭제 (Delete)	delete	객체 또는 항목 삭제	deleteUserById()
	remove	컬렉션/리스트 등에서 제거	removeItemFromList()

클래스 명

자유롭게 작성할 수 있지만 일반적으로 사용하는 규칙이 존재함

- 클래스 명은 명사
- 대문자로 시작 (UpperCameCase)
- 두 개 이상의 단어가 조합될 때는 각 단어의 첫 번째 글자는 대문자

예시)

- OrderHistory.java
- UserJoimServeice.java

메소드 명

동사 + 명사를 메소드 명을 구성하는 것이 일반적인 방식

- 동사로 시작함
- 소문자로 시작
- 두 개 이상의 단어가 조합될 땐 첫 글자만 소문자 (lowerCameCase)

예시)

- vallidateAndRun ( );
- findOrderHistories ( );
- execute()

  ※ 동사만 있는 경우는 클래스명 또는 인터페이스 명 자체로 명사 역할이 명확한 경우 동사만으로 구성 가능

변수 명

변수명은 일반적으로 명사로 짓는 것을 권고

변수명을 읽었을 때 어떤 의미인지 파악이 가능해야 함 (AI Tool 등록시 변수명만으로 파악 가능)

예시)

- username (사용자 이름)
- totalPrice (총 금액)
- createdAt (생성일시)
- maxRetryCount (최대 재시도 횟수)

고려 사항)

- for 반복문에서 순서를 나타내기 위한 정수 변수명은 i, j, k 등을 활용
- 변수명에 특수문자를 사용할 수는 있지만 지양
- 변수명에 a, b2 등 의미를 알기 힘든 단어는 지양

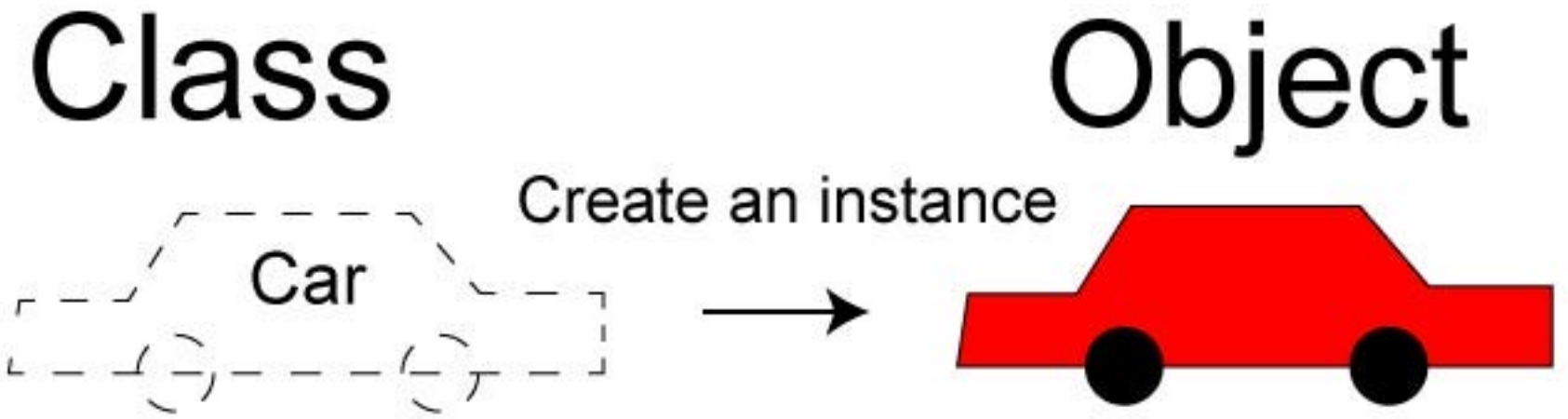
Java 클래스와 객체

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

Java에서 객체를 만드는 방법

객체는 클래스의 인스턴스이며, 모든 객체는 java.lang.Object를 상속



Properties	**Methods** - behaviors
color	start()
price	backward()
km	forward()
model	stop()

Property values	**Methods**
color: red	start()
price: 23,000	backward()
km: 1,200	forward()
model: Audi	stop()

메모리 공간 할당

위치 : 주소

Java 핵심 용어

개념	설명	Java 예제 코드
클래스 (Class)	객체를 만들기 위한 설계도. 속성과 동작을 정의.	class Student
객체 (Object)	클래스를 기반으로 생성된 실체(인스턴스).	Student s1 = new Student();
속성 (Field)	객체가 가지는 상태/정보 (변수).	String name; double grade;
메서드 (Method)	객체가 수행하는 동작 (함수).	void study();
생성자 (Constructor)	객체 생성 시 초기화 메서드. 클래스 이름과 동일.	Student(String name, int id)

Java – 클래스 (Class)

Java에서 클래스는 객체를 생성하기 위한 설계도로 속성과 동작을 포함하며 생성자를 가짐

구성 요소	설명	예시
필드(Field)	객체의 상태를 저장하는 변수	String name; double price;
메서드(Method)	객체의 동작을 정의하는 함수	void updatePrice(double price) {...}
생성자(Constructor)	객체 생성시 초기화(클래스 이름과 동일)	public Stock(String name, double price) {...}
this	클래스기반 생성된 객체 참조	this.name = name;
super	부모 클래스의 멤버에 접근	super(name, price);

```
public class Stock {
    String name;
    double price;

    public Stock(String name, double price) {
        this.name = name;   // this: 현재 객체
        this.price = price;
    }

    public void updatePrice(double newPrice) {
        this.price = newPrice;
    }

    public void printInfo() {..}
}
```

Java – 객체 (Object)

Class로부터 생성된 실체(Instance)로 속성(필드)과 동작(메서드)을 갖는 메모리 상의 실제 존재

구성 요소	설명	예시
인스턴스 생성	객체를 메모리에 생성	Stock s = new Stock("스칼라 AI", 17000);
필드	객체가 가지고 있는 상태/정보	s.name = "스칼라 AI";
메서드	객체가 수행할 수 있는 기능/행동	s.updatePrice(17000);
생성자	객체가 생성될 때 초기값 설정	new Stock("스칼라 AI", 17000);

```
public class Main {
    public static void main(String[] args) {
        // 객체 생성 (인스턴스화)
        Stock scalaEdu = new Stock("스칼라 에듀", 15000);
        Stock scalaAI = new Stock("스칼라 AI", 17500);

        // 객체 상태 변경
        scalaEdu.updatePrice(15800);
        scalaEdu.printInfo();
    }
}
```

[실습] 클래스와 객체 실습

01.training/01.snipset/stock/Main.java, Stock.java

- 01.training-code/01.snippet/stock/Stock.java, Main.java를 완성해보자
- 이를 통해 클래스와 객체에 대해 이해해보자

```
public class Main {
    public static void main(String[] args) {
        // 객체 생성 (인스턴스화)
        Stock scalaEdu = new Stock("스칼라 에듀", 15000);
        Stock scalaAI = new Stock("스칼라 AI", 17500);

        // 객체 상태 변경
        scalaEdu.updatePrice(15800);
        scalaEdu.printInfo();
    }
}
```

Java – 필드 (Field)

- 클래스 내부에 선언된 변수로, 객체의 고유한 "속성" 또는 "상태"를 저장
- 필드는 객체가 생성될 때 메모리에 할당되며, 각 객체는 독립적인 자신의 필드 값을 가짐

종류	설명	예시
인스턴스 필드	객체마다 독립적인 값을 가짐	String name;
정적 필드 (static)	클래스 전체에서 공유되는 값	static int count;
상수 필드 (final)	한 번 값이 정해지면 변경 불가	final String STOCK_TYPE = "우선주";

this 이해하기

- This는 클래스를 객체로 Heap에 생성될때 객체를 가르키는 주소

```
this 는 클래스 자기 자신을 의미함


class User {
  private String name;

  User(String name) {
    this.name = name;
  }
}
```

Java – 메서드 (Method)

- 메서드는 객체의 동작(기능)을 정의하는 코드 블록

```
[접근제어자] [반환형] [메서드이름](매개변수 목록) {
    // 메서드 본문
    return 값; // 반환형이 void가 아닌 경우
}
```

예시)

```
public String myFunction(String name) {
   …
   return “return value”;
}
```

구성 요소	설명	예시
접근제어자	외부에서 메서드 접근 가능 여부	public, private, protected, (default)
반환형	메서드 실행 후 반환하는 값의 타입	int, String, void 등
메서드명	동작을 나타내는 이름 (소문자로 시작)	getPrice, calculateTax
매개변수(Parameter)	메서드에 전달되는 입력 값들	(int a, int b)
return문	결과를 반환하고 메서드 종료	return a + b;

Java – 메서드 (Method)

```
public class StockUtils {

    // 1. 반환값도 없고, 매개변수도 없음
    public void printWelcomeMessage() {
        System.out.println("주식 정보 시스템에 오신 것을 환영합니다.");
    }

    // 2. 매개변수만 있고, 반환값 없음
    public void printStockPrice(String stockName, double price) {
        System.out.println(stockName + "의 현재 가격은 " + price + "원입니다.");
    }

    // 3. 반환값만 있고, 매개변수 없음
    public String getMarketStatus() {
        return "장 마감";
    }

    // 4. 매개변수도 있고, 반환값도 있음
    public double calculatePriceChange(double yesterday, double today) {
        return today - yesterday;
    }

    // 5. 오버로딩(Overloading): 이름은 같지만 매개변수가 다름
    public void printStockPrice(String stockName, double price) {
        System.out.println("종목명: [" + stockName + "]");
        System.out.println("가격:   [" + price + "]");
    }
}
```

Java – 메서드: 헷갈리는 용어, 매개 변수와 인자

매개 변수 = parameters = 메소드에 전달된 입력값을 저장하는 변수
인자, 전달 인자 = arguments = 메소드를 호출할 때 전달하는 입력값

```
int sum(int a, int b) { // a 와 b 는 매개 변수 (parameters)
  return a + b;
}


sum(1, 2) // 1 과 2 는 인자, 전달인자 (arguments)
```

static 클래스, 변수 , 메소드 (1/2)

static 키워드는 객체가 아니라 클래스 자체에 속하는 변수나 메소드를 정의할때 사용

```
public class StaticTest {

    static class Example {
        public static final int INITIAL_COUNT = 10;    //Metaspace의 상수 풀에 존재
        private static int count = 0;                   // Heap 내 java.lang.Class의 Index 정보
        private int instanceId;

        Example() {
            count++;
            instanceId = count + INITIAL_COUNT;
        }

        public static int getLastInstanceId() {
            return count + INITIAL_COUNT;
        }

        public static int getCount() {
            return count;
        }

        public int getInstanceId() {
            return instanceId;
        }
    }
```

static 클래스, 변수 , 메소드 (2/2)

static 키워드는 객체가 아니라 클래스 자체에 속하는 변수나 메소드를 정의할때 사용

```
public class StaticTest {

// { 앞의 코드 }


    public static void main(String[] args) {
        Example e1 = new Example();
        Example e2 = new Example();
        Example e3 = new Example();

        System.out.println("총 생성된 객체 수(count): " + Example.getCount());
        System.out.println("마지막 생성된 instanceId: " + Example.getLastInstanceId());

        System.out.println("e1의 instanceId: " + e1.getInstanceId());
        System.out.println("e2의 instanceId: " + e2.getInstanceId());
        System.out.println("e3의 instanceId: " + e3.getInstanceId());
    }
}
```

[실습] static 이해하기

01.training/01.snippet/static/StaticTest.java

1. 앞의 StaticTest.java 코드를 작성해보자
2. Static 동작 방식을 이해해보자

주석 달기

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

Java의 주석(comment) 방식

구분	사용 문법	용도	예시
한 줄 주석	//로 시작	간단한 설명, TODO, 디버깅 등	// 변수 초기화
여러 줄 주석	/* ... */로 감싸기	여러 줄 설명, 블록 주석 등	/* 여러 줄 주석입니다 */
문서 주석	/** ... */로 시작 (Javadoc)	메서드, 클래스 문서 자동화	/** 이름을 반환합니다 **/

```
public class Example {

    // 한 줄 주석: 사용자 이름 출력
    public static void main(String[] args) {
        /* 여러 줄 주석:
           변수 선언과 초기화 */
        String name = "Skala";

        /**
         * 사용자 이름을 출력합니다.
         * @param name 사용자 이름
         */
        System.out.println("Hello, " + name);
    }
}
```

Javadoc 주요 태그

태그	설명
@param	메서드 매개변수 설명
@return	반환값 설명
@throws	예외 설명
@author	작성자
@since	버전 표시

```
/**
 * 주어진 두 수의 합을 반환합니다.
 * @param a 첫 번째 정수
 * @param b 두 번째 정수
 * @return 두 정수의 합
 */
public int add(int a, int b) {
    return a + b;
}
```

[실습] Javadoc 만들어보기

02.snippet/JavaDocsExample.java

```
javadoc –d doc  JavaDocsExample.java
```

```
/**
 * 간단한 계산기 클래스.
 * <p>
 * 두 개의 정수에 대해 덧셈과 뺄셈 기능을 제공합니다.
 * </p>
 *
 * @author 홍길동
 * @version 1.0
 * @since 2025-08-13
 */
public class Calculator {

    /**
     * 마지막 계산 결과를 저장합니다.
     */
    private int lastResult;

    /**
     * 기본 생성자.
     * 초기 lastResult 값은 0입니다.
     */
    public Calculator() {
        this.lastResult = 0;
    }

    /**
     * 두 정수를 더합니다.
     *
     * @param a 첫 번째 정수
     * @param b 두 번째 정수
     * @return 덧셈 결과
     */
    public int add(int a, int b) {
        lastResult = a + b;
        return lastResult;
    }

    /**
     * 두 정수를 뺍니다.
     *
     * @param a 첫 번째 정수
     * @param b 두 번째 정수
     * @return 뺄셈 결과 (a - b)
     */
    public int subtract(int a, int b) {
        lastResult = a - b;
        return lastResult;
    }
}
```

VSCODE의 Live Server를 이용하여 Index.html을 확인해보자

Java 주석 작성 가이드

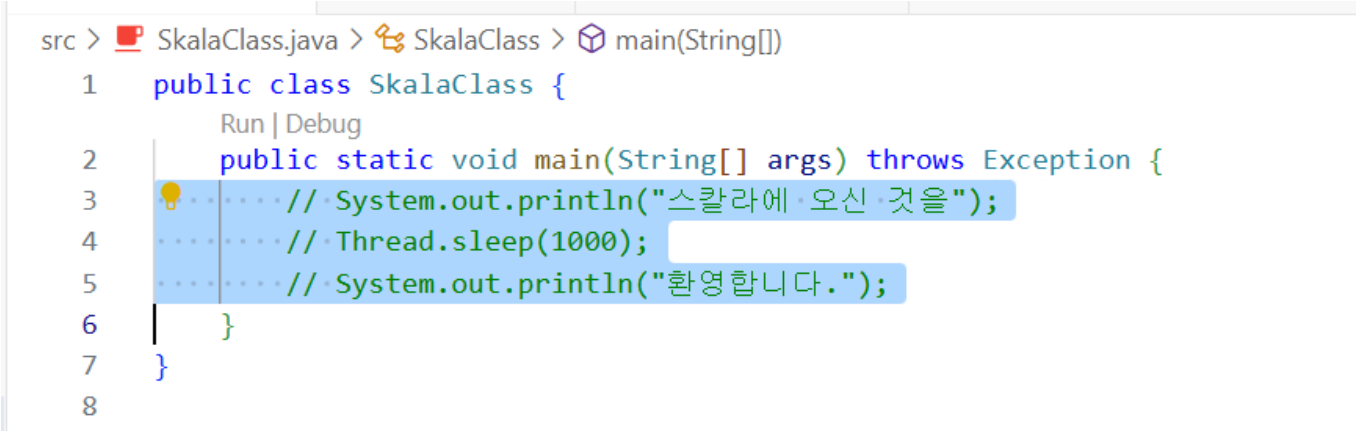
가이드 항목	설명
필요한 경우만 작성	코드 자체로 의미가 명확할 경우 불필요한 주석은 피함
주석 내용은 왜(Why)와 무엇(What)를 설명	선택한 이유, 의도, 대안 비교 / 코드의 기능이나 동작 설명
오래된 주석 제거	코드가 변경되면 주석도 함께 업데이트
Javadoc 사용 권장	public 클래스/메서드는 /** ... */ 문서 주석 사용
*TODO / FIXME 주석	작업 항목 추적 시 명확히 구분: // TODO: 로그인 로직 추가
API 문서 자동 생성 시 문서 주석 활용	javadoc 도구로 HTML 문서 생성 가능

❖ // TODO 와 // FIXME 는 개발자가 작업 중인 코드에 메모를 남길 때 사용하는 특별한 주석 패턴
IDE(예: IntelliJ, Eclipse)에서도 자동으로 인식되어 작업 추적(TODO 리스트) 용도로 널리 사용

- **// TODO** : 해야 할 작업(To-Do)을 기록해두는 주석
- **// FIXME** : 현재 코드에 문제가 있음을 나타내고 수정이 필요함을 알리는 주석

[참고] VS Code에서 주석 관리

주석 처리하고자 하는 라인을 선택 후 “ctrl+ /” – 토글 (맥북에서는 command + /)



자바의 제어문

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

Java의 제어문(Control Statements)

분류	제어문 종류	설명	사용 예시 또는 키워드
조건문	if, else if, else	조건에 따라 다른 블록을 실행	if (a > b) { ... }
	switch	여러 값 중 하나와 일치하는 분기 실행	switch (value) { case 1: ... }
반복문	for	조건에 따라 정해진 횟수만큼 반복	for (int i = 0; i < 10; i++)
	while	조건이 참인 동안 계속 반복	while (i < 10)
	do-while	최소 1회는 실행되고 조건이 참인 동안 반복	do { ... } while (i < 10)
분기문	break	반복문 또는 switch문을 즉시 종료	if (...) break;
	continue	현재 반복을 건너뛰고 다음 반복으로 진행	if (...) continue;
	return	현재 메서드의 실행을 종료 또는 값을 반환	return value; 또는 return;
예외 처리	try-catch-finally	예외 상황을 처리하기 위한 제어 흐름 구조	try { ... } catch (Exception e) { ... } finally { ... }
	throw, throws	예외를 발생시키거나 메서드에서 예외를 위임	throw new IOException();

If문

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

if, else if, else

- 조건식의 결과에 따라 코드 블록을 선택적으로 실행

```
int score = 85;
if (score >= 90) {
    System.out.println("A 학점");
} else if (score >= 80) {
    System.out.println("B 학점");
} else {
    System.out.println("C 학점 이하");
}
```

Java의 논리 연산자

연산자	의미	설명	예제 조건
&&	and	양쪽 조건이 모두 참이어야 전체 조건이 참	a > 0 && b > 0
||	or	한쪽 조건이 참이면 전체 조건이 참	
!	not	조건을 반대로 뒤집음 (참 → 거짓, 거짓 → 참)	!(a > 0)

- x && y (x와 y 모두 참이어야 함)
- x || y (x 또는 y 하나만 참이어도 됨)
- !x (x가 거짓이어야 함)

```
if (arriveAT <= “0900” && !leaderAtWork) {
    // 출근한 동료들과 커피를 마시러 간다
}
```

&&와 ||는 단락 평가(short-circuit evaluation)로 처리

a && b: a가 false이면 b는 평가하지 않음

a || b: a가 true이면 b는 평가하지 않음

[실습] 학점 계산기

01.training/02.if/GradeCalculator.java

GradeCalcuator.java 코드를 검토하고 실행해보자

- 사용자에게 점수 (0 ~ 100)를 입력받아 아래 기준에 따라 학점을 출력하는 프로그램
- If, else if, else 를 사용해서 구성

- 90점 이상: A
- 80점 이상: B
- 70점 이상: C
- 60점 이상: D
- 60점 미만: F

```
점수를 입력하세요: 85
학점: B
```

```
        // 학점 계산
        if (score >= 90) {
            grade = "A";
        } else if (score >= 80) {
            grade = "B";
        } else if (score >= 70) {
            grade = "C";
        } else if (score >= 60) {
            grade = "D";
        } else {
            grade = "F";
        }
```

Switch / Case 문

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

switch/case

- 여러 값 중 하나와 일치하는 분기를 선택하여 실행

```
int menu = 2;
switch (menu) {
    case 1:
        System.out.println("주식 조회");
        break;
    case 2:
        System.out.println("주식 매수");
        break;
    case 3:
        System.out.println("주식 매도");
        break;
    default:
        System.out.println("잘못된 선택");
}
```

switch 언제 사용해야 할까?

입력값이 정형화되어 있는 경우 (예: ENUM을 활용)에는 if문보다 switch/case문이 가독성이 좋음

```
public enum LoginChannel {
    FACEBOOK, KAKAO, APPLE, NAVER, DEFAULT;
}
```

```
 public static void executeLogin(LoginChannel channel) {
        switch(channel) {
            case FACEBOOK:
                facebookLogin();
                break;
            case KAKAO:
                kakaoLogin();
                break;
            case APPLE:
                appleLogin();
                break;
            case NAVER:
                naverLogin();
                break;
            case DEFAULT:
                defaultLogin();
                break;
        }
    }
```

[참고] enum 내부 method 사용 방법

enum 은 특수한 형태의 클래스

```
public enum LoginChannel {
   //public static final,  LoginChannel 클래스의 Singleton Instance
    FACEBOOK, KAKAO, APPLE, NAVER, DEFAULT;

    public boolean isSocial() {
        return this == FACEBOOK || this == KAKAO ||  this == NAVER;
    }
}
```

FACTBOOK은 name(), ordinal(), toString()을 가지고 있는 클래스 인스턴스

```
LoginChannel channel = LoginChannel.FACEBOOK;

// 직접 enum 상수에서 호출
boolean result1 = LoginChannel.FACEBOOK.isSocial();  // true
System.out.println(result1);

// 변수를 통해 호출
boolean result2 = channel.isSocial();  // true
System.out.println(result2);

// 조건문에서 활용
if (channel.isSocial()) {
    System.out.println(channel + "은 소셜 로그인입니다.");
}
```

switch 표현식

기존 Switch 문에 함수형 스타일로 값을 반환할 수 있게 확장된 문법 체계로 Java 14 이후 버전에서 지원

특징

- 표현식(Expression): 결과를 변수에 할당 가능
- break 불필요 — -> 사용 시 자동 종료
- 다중 문장 사용 시 {} 블록 + yield 필수
- N개 case에 같은 동작 가능: case 1, 2 -> "Value";

장점

- 코드 간결화
- 버그 방지 (break 누락 방지)
- 가독성 향상

```
int menu = 2;

String message = switch (menu) {
    case 1 -> "주식 조회";
    case 2 -> {
        System.out.println("거래 로그 기록");
        System.out.println("DB 저장");
        yield "주식 매수";
    }
    case 3 -> "주식 매도";
    default -> "잘못된 선택";
};

System.out.println(message);
```

switch 표현식

기존 방식

```
switch(channel) {
  case FACEBOOK:
    facebookLogin();
    break;
  case KAKAO:
    kakaoLogin();
    break;
  case APPLE:
    appleLogin();
    break;
  case NAVER:
    naverLogin();
    break;
  case DEFAULT:
    defaultLogin();
    break;
  }
}
```

switch expression

```
switch (channel) {
  case FACEBOOK -> facebookLogin();
  case KAKAO -> kakaoLogin();
  case APPLE -> appleLogin();
  case NAVER -> naverLogin();
  case DEFAULT -> defaultLogin();
}
```

[실습] Switch 코드 확인하기

01.training/03.switch/EnumSwitch.java

EnumSwitch.java 코드를 검토하고 실행해보자

[실습] 계산기

01.training/03.switch/Calculator.java

Calculator.java 코드를 검토하고 실행해보자

- 두 개의 숫자와 연산자(+, -, *, /)를 입력받아 계산 결과를 출력
- / 연산 시, 0으로 나누는 경우 "0으로 나눌 수 없습니다." 출력

```
첫 번째 숫자: 10
연산자(+ - * /): *
두 번째 숫자: 3
결과: 30
```

- **Calcuator.java의 Switch 문을 Switch Expression으로 변경해보세요**

```
// switch expression을 사용하여 계산
        double result = switch (operator) {
            case "+" -> (double)firstNumber + secondNumber;
            case "-" -> (double)firstNumber - secondNumber;
            case "*" -> (double)firstNumber * secondNumber;
            case "/" -> (double) firstNumber / secondNumber;
            default -> Double.NaN; // 잘못된 경우 NaN 반환
        };
```

While / Do While

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

while

조건이 참인 동안 반복

```
int i = 1;
while (i <= 3) {
    System.out.println("잔고 확인: " + i + "번째");
    i++;
}
```

무한 루프

```
public class skalatest5 {
    public static void main(String[] args) {
        Random random = new Random();
        int number = 100;
        while(true) {
            if(number <= 5){
                System.out.println("무한루프탈출" + number);
                break;
            }
            number = random.nextInt(100) + 1;
            System.out.println("현재 숫자 = " + number);
        }
    }
}
```

```
현재 숫자 = 95
현재 숫자 = 80
현재 숫자 = 28
현재 숫자 = 62
현재 숫자 = 84
현재 숫자 = 44
현재 숫자 = 65
현재 숫자 = 25
현재 숫자 = 23
현재 숫자 = 87
현재 숫자 = 62
현재 숫자 = 76
현재 숫자 = 10
현재 숫자 = 14
현재 숫자 = 29
현재 숫자 = 74
현재 숫자 = 16
현재 숫자 = 35
현재 숫자 = 24
현재 숫자 = 42
현재 숫자 = 24
현재 숫자 = 1
무한루프탈출1
```

do while

무조건 한 번은 실행되고, 이후 조건이 참이면 반복

```
int count = 0;
do {
    System.out.println("거래 내역 조회 시도: " + count);
    count++;
} while (count < 2);
```

[실습] 계산기

01.training/04.while/Calculator.java

Calculator.java에 while 문을 추가해서 적용해보자

- while문을 통해 첫번째 실행 시 계산 처리 실행
- 첫번째 실행 후 "계속할려면 c(continue) / 종료하려면 q(quit) 을 입력받도록
- c 이면 반복 실행, q이면 프로그램 종료

```
첫 번째 숫자: 10
연산자(+ - * /): /
두 번째 숫자: 2
결과: 5.0
계속하려면 c(continue) / 종료하려면 q(quit) 입력: c
첫 번째 숫자: 10
연산자(+ - * /): +2
두 번째 숫자: 4
잘못된 연산자입니다.
계속하려면 c(continue) / 종료하려면 q(quit) 입력: q
```

For / For Each

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

For

정해진 횟수만큼 반복

```
for (int i = 1; i <= 5; i++) {
    System.out.println("주식 가격 확인: " + i + "일차");
}
```

Array 개수만큼 처리

```
String[] students = {"Tom", "Jane", "Mike", "Chris", "Amy"};

for (int i = 0; i < students.length; i++) {
    System.out.println(students[i]);
}
```

중첩 For 반복문

For 반복문을 중첩하면 이중 배열을 쉽게 출력할 수 있음

```java
package skalajava;
public class skalatest11 {
    Run | Debug
    public static void main(String[] args) {
        printGuGuDan();
    }
    public static void printGuGuDan() {
        for(int i = 1; i <= 9; i++) {
            for (int j = 1; j <= 9; j++) {
                System.out.print(i * j + " " );
            }
            System.out.println();
        }
    }
}
```

```
1 2 3 4 5 6 7 8 9
2 4 6 8 10 12 14 16 18
3 6 9 12 15 18 21 24 27
4 8 12 16 20 24 28 32 36
5 10 15 20 25 30 35 40 45
6 12 18 24 30 36 42 48 54
7 14 21 28 35 42 49 56 63
8 16 24 32 40 48 56 64 72
9 18 27 36 45 54 63 72 81
```

for-each

- 배열(Array)이나 리스트의 모든 요소를 순서대로 꺼낼때 쓰는 가장 깔끔한 반복문

전통적 For 문

```
String[] students = {"Tom", "Jane", "Mike", "Chris", "Amy"};

for (int i = 0; i < students.length; i++) {
    System.out.println(students[i]);
}
```

for-each 문

```
String[] students = {"Tom", "Jane", "Mike", "Chris", "Amy"};

for (String student : students) {
    System.out.println(student);
}
```

항목	특징
가독성	코드가 간결해짐
안정성	인덱스 범위를 범위를 벗어나는 실수 (Index Out Of Bounds)를 문법적으로 차단
일관성	일반 배열 (String [] , Collection (ArrayList, HashSet, LnikedList etc) 상관없이 동일 문법 처리

[실습] 계산기 – For-Each 적용하기

01.training-code/05.for/Calculator.java

while 에서 개발했던 Calculator.java 코드에서 다음을 추가하자

1. String[] history = new String[100]; 을 선언
2. 반복적으로 계산한 결과를 String [] history 에 저장
   예) "10 + 10 = 20 " , "10 – 5= 5.0" …
3. while 종료 시 history에 저장되어 있던 계산 기록을 화면에 출력하고 종료
4. history에 있는 계산 기록을 화면에 출력 시 **for-each** 로 코딩

```
첫 번째 숫자 : 10
연산자(+ - * /): +
두 번째 숫자 : 10
결과 : 20.0
계속하려면 c(continue) / 종료하려면 q(quit) 입력 : c
첫 번째 숫자 : 10
연산자(+ - * /): -
두 번째 숫자 : 5
결과 : 5.0
계속하려면 c(continue) / 종료하려면 q(quit) 입력 : c
첫 번째 숫자 : 10
연산자(+ - * /): *
두 번째 숫자 : 10
결과 : 100.0
계속하려면 c(continue) / 종료하려면 q(quit) 입력 : c
첫 번째 숫자 : 10
연산자(+ - * /): /
두 번째 숫자 : 10
결과 : 1.0
계속하려면 c(continue) / 종료하려면 q(quit) 입력 : q
```

```
=== 계산 기록 ===
10 + 10 = 20.0
10 - 5 = 5.0
10 * 10 = 100.0
10 / 10 = 1.0
```

제어 흐름 변경 방법

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

제어문 – break

- 자신을 감싸고 있는 가장 가까운 반복문 또는 Switch 를 탈출

```
for (int i = 1; i <= 10; i++) {
    if (i == 4) {
        break;
    }
    System.out.println("거래 횟수: " + i);
}
```

- 중첩 반복문에서 한 번에 탈출하는 방법 (Labeled Break)

```
        OUTER_LOOP: // 바깥쪽 루프에 라벨을 붙임
        for (int i = 1; i <= 9; i++) {
            for (int j = 1; j <= 9; j++) {
                if (i == 5 && j == 5) {
                    break OUTER_LOOP; // 바깥쪽 루프까지 바로 탈출
                }
            }
        }
```

제어문 – continue

- 현재 회차의 루프만 건너뛰고 다음 회차로 점프

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
        continue;
    }
    System.out.println("처리 중인 거래 번호: " + i);
}
```

제어문 – return

- 루프는 물론, 해당 루프를 품고 있는 메서드 자체를 종료

```
public static void checkNumber(int number) {
        for (int i = 1; i <= 10; i++) {
            if (i == number && i > 5) {
                return;
            }
            System.out.println("반복문 진행 중... 현재 값: " + i);
        }
        System.out.println("함수 끝자리에 있는 남은 로직 (정상 종료 시에만 출력됨)");
    }
```

예외 처리 – throw / throws

- 예외를 직접 발생시키거나, 메서드에서 예외가 발생할 수 있음을 명시

```
public static void buyStock(int amount) throws Exception {
    if (amount <= 0) {
        throw new Exception("매수 금액은 0보다 커야 합니다.");
    }
}
```

- **throws** : 이 함수는 Exception을 발생 시킬 수 있으므로 try-catch 로 던져진 Exception을 받아야 함
- **throw**: Exception을 던지고 탈출

예외 처리 – try-catch 방식

- 예외 발생 시 프로그램이 중단되지 않도록 예외를 처리

```
try {
    System.out.println("주식 매수를 시도합니다...");

    // 금액을 0 이하(예: -500)로 넣으면 예외가 발생하여 catch 블록으로 점프
    buyStock(-500);

    // 예외가 발생하면 이 아래 줄은 실행되지 않고 바로 탈출
    System.out.println("주식 매수가 성공적으로 완료되었습니다.");

} catch (Exception e) {
    // throw new Exception("...")에 담았던 메시지가 e.getMessage()로 출력
    System.out.println("매수 실패 오류: " + e.getMessage());

} finally {
    // 예외 발생 여부와 상관없이 무조건 실행되는 블록
    System.out.println("주식 거래 프로세스 종료");
}
```

예외 처리: try-with-resources (1/2)

- 외부 자원(External Resource)를 사용하는 경우, 즉 JVM 메모리 외부에 의존하는 자원은 반드시 close() 필요

```
InputStream in = null;
OutputStream out = null;
Socket client = null;
try {
    client = serverSocket.accept();
    in = client.getInputStream();
    out = client.getOutputStream();
…
} catch (IOException e) {
    e.printStackTrace();
} finally {
    // 반드시 닫아야 함 (역순으로 닫음)
    try {
        if (out != null) out.close();
    } catch (IOException ignored) {}
    try {
        if (in != null) in.close();
    } catch (IOException ignored) {}
    try {
        if (client != null) client.close();
    } catch (IOException ignored) {}
}
```

참고

외부 자원 즉 Socket, File, DB Connection 등은
OS 에서 발급하는 FD (File Description)을 할당받음

close는 이러한 FD를 해지하는 역할

미 해지 시

- Socket port 고갈
- Too Many Open File
- Memory Leak

예외 처리: try-with-resources (2/2)

```
try (자원 선언1; 자원 선언2; ...) {
    // 자원을 사용하는 코드
} catch (예외타입 e) {
    // 예외 처리
}
```

```
try (
    // try 괄호() 안에 자원을 선언하면 따로 close()를 호출하지 않아도 자동으로 닫힙니다.
    Socket client = serverSocket.accept();
    InputStream in = client.getInputStream();
    OutputStream out = client.getOutputStream()
) {
…
} catch (IOException e) {
    e.printStackTrace();
}
```

참고

Stream은 **바이트(byte)** 단위 입/출력을 의미 (**socket, file, device** 등)

InputStream은 바이트를 읽음 (내부 문자/스트링으로 변환)

OutputStream은 바이트로 출력 (대부분 문자/스트링을 바이트로 변환)

[참고] Custom Exception

자신의 Exception을 만들어서 예외 상황을 명확하게 식별하고, 처리 로직을 구분하기 위해 사용

```
public class CustomException extends Exception {
    public CustomException(String message) {
        super(message);
    }

    public CustomException(String message, Throwable cause) {
        super(message, cause);
    }
}
```

```
public static void buyStock(int amount) throws CustomException {
    if (amount <= 0) {
        throw new CustomException("매수 금액은 0보다 커야 합니다.");
    }
}
```

[실습] 계산기

01.training-code/06.exception/Calculator.java

- 앞의 계산기에서 / 연산에서 0으로 나누고자 할때 ArithmeticException을 발생시키고 이것을 화면에 출력한 뒤 다시 계산을 실행하도록 코드를 개선

```
// 0으로 나누기 예외 처리
try {
    if (operator.equals("/") && secondNumber == 0) {
        throw new ArithmeticException("0으로 나눌 수 없습니다.");
    }
} catch (ArithmeticException e) {
    System.out.println(e.getMessage());
    e.printStackTrace();
    continue;
}
```

```
> java Calculator.java
첫 번째 숫자: 10
연산자(+ - * /): /
두 번째 숫자: 0
예외 발생: 0으로 나눌 수 없습니다.
java.lang.ArithmeticException: 0으로 나눌 수 없습니다.
        at Calculator.main(Calculator.java:29)
        at java.base/jdk.internal.reflect.DirectMethodHandleAccessor.invoke(DirectMethodHandleAccessor.java:103)
        at java.base/java.lang.reflect.Method.invoke(Method.java:580)
        at jdk.compiler/com.sun.tools.javac.launcher.Main.execute(Main.java:484)
        at jdk.compiler/com.sun.tools.javac.launcher.Main.run(Main.java:208)
        at jdk.compiler/com.sun.tools.javac.launcher.Main.main(Main.java:135)
첫 번째 숫자:
```

자바의 패키지

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

Java의 패키지(Package)

- Java의 패키지는 관련 있는 클래스들을 논리적으로 묶는 단위
- 일종의 폴더 구조이며, 클래스 간의 이름 충돌 방지, 접근 범위 관리, 재사용성 향상 등을 제공

```
package com.skala.stock;  // 소스 코드 상단에 패키지 선언
```

❖ 실제로 com/skala/stock/ 폴더에 해당 클래스 파일이 존재

```
com/
 └── skala/
     ├── stock/
     │    └── Stock.java
     └── player/
          └── Player.java
```

- Stock.java와 Player.java는 서로 다른 패키지에 존재
- 같은 이름의 클래스가 다른 패키지에 있어도 충돌하지 않음

pom.xml /gradle 사용 시 package의 시작은 /src/main/java

접근 제어자

- 접근 제어자는 클래스, 메서드, 필드, 생성자 등에 대해 외부에서의 접근 가능 범위(가시성) 를 제어하는 키워드
- 객체지향 프로그래밍(OOP)에서 정보 은닉(encapsulation) 을 구현하는 데 중요한 역할

접근 제어자	같은 클래스	같은 패키지	자식 클래스	외부 클래스
public	✅	✅	✅	✅
protected	✅	✅	✅	❌
(default)	✅	✅	❌	❌
private	✅	❌	❌	❌

❖ default는 접근 제어자를 명시하지 않았을 때의 상태 (package-private)

❖ protected는 같은 패키지 또는 상속받은 클래스에서 접근 가능

패키지와 접근 제어자의 연결

접근 제어자	설명	패키지 간 접근
public	모든 패키지에서 접근 가능	✅
protected	같은 패키지 + 다른 패키지의 하위 클래스에서 접근 가능	⭕ (상속관계만)
(default)	같은 패키지에서만 접근 가능 (접근 제어자 생략)	❌
private	같은 클래스 내부에서만 접근 가능	❌

```
package skala.domain;

class Stock {
    String name = "스칼라 AI"; // default 접근제어자
}
```

```
package skala;

import skala.domain.Stock;

public class Player {
    public static void main(String[] args) {
        Stock s = new Stock();        // 오류 발생
        System.out.println(s.name); // default 접근자는 다른 패키지에서 접근 불가
    }
}
```

[실습] Stock 에 패키지 적용해보기

01.training-code/01.snippet/package 참고

01.snippet/stock/ 내 Stock.java와 Main.java에 아래와 같이 package와 import를 넣고
컴파일 후 bin 디렉토리 아래에 생성된 클래스 디렉토리 구조를 확인해보자

Main.java

```
package skala;

import skala.domain.Stock;

public class Main {
    static void printStockInfo(Stock stock) {
        stock.printInfo();
    }
…
```

Stock.java

```
package skala.domain;

public class Stock {
    protected String name;
    protected double price;
…
```

실행하기

```
> javac -d bin *.java
> java -cp bin skala.Main
```

자바 필수 클래스

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

Java 필수 클래스

- 주로 java.lang 패키지에 포함된 핵심 클래스

클래스명	설명
Object	모든 클래스의 부모. toString(), equals() 등 객체 기본 기능 이해 필수
System	System.out.println()은 자바 입문의 시작. 콘솔 입출력, 시스템 종료 등
Thread	멀티스레드 기초 개념은 간단한 예제부터 시작 가능 (Thread.sleep() 등)
Exception	예외 처리(try-catch)는 자바의 안정성을 이해하는 데 중요
Runtime	JVM과 상호작용하는 대표 클래스. 메모리, gc, 프로세스 실행 가능
Class	클래스와 객체의 관계, 리플렉션 기초 개념을 위해 반드시 알아야 할 대상

Java 필수 클래스 – Object

- Java의 모든 클래스는 **Object를 암묵적으로 상속**
- 자주 사용되는 메서드:
  - **toString()**: 객체를 문자열로 표현
  - **equals()**: 객체 비교
  - **hashCode()**: 해시값 반환 (Map 등에서 사용됨)

```
public class Stock {
    String name;

    public Stock(String name) {
        this.name = name;
    }

    @Override
    public String toString() {
        return "Stock[name=" + name + "]";
    }

    @Override
    public boolean equals(Object obj) {
        if (obj instanceof Stock) {
            Stock other = (Stock) obj;
            return this.name.equals(other.name);
        }
        return false;
    }
}
```

참고) Object의 equals을 구현하지 않으면 참조값 비교

```
public class Main {
    public static void main(String[] args) {
        Stock s1 = new Stock("SKALA");
        Stock s2 = new Stock("SKALA");

        System.out.println(s1);   // toString() 호출
        System.out.println(s1.equals(s2));   // true
    }
}
```

Java 필수 클래스 – System

- 표준 입출력, 환경정보 접근, 시간 측정, 프로그램 종료 등 시스템 관련 기능 제공
- 자주 사용되는 메서드:
  - System.out.println()
  - System.currentTimeMillis()
  - System.exit(0)

```
public class SystemExample {
    public static void main(String[] args) {
        System.out.println("스칼라에 오신 것을 환영합니다.");

        long start = System.currentTimeMillis();
        for (int i = 0; i < 1000000; i++) {} // 더미 연산
        long end = System.currentTimeMillis();

        System.out.println("걸린 시간: " + (end - start) + "ms");

        // System.exit(0); // 프로그램 종료
    }
}
```

Java 필수 클래스 – Thread

- 동시성 처리를 위한 클래스
- 자주 사용되는 메서드:
- start()로 새 스레드를 실행
- sleep(ms)으로 일정 시간 일시정지 가능
- join()으로 start 된 thread가 종료될때 까지 대기

```
public class ThreadExample {
    public static void main(String[] args) {
        Thread t = new Thread(() -> {
            try {
                Thread.sleep(1000);
                System.out.println("1초 후에 실행되는 작업");
            } catch (InterruptedException e) {
                e.printStackTrace();
            }
        });

        t.start();
        System.out.println("메인 스레드 종료");
    }
}
```

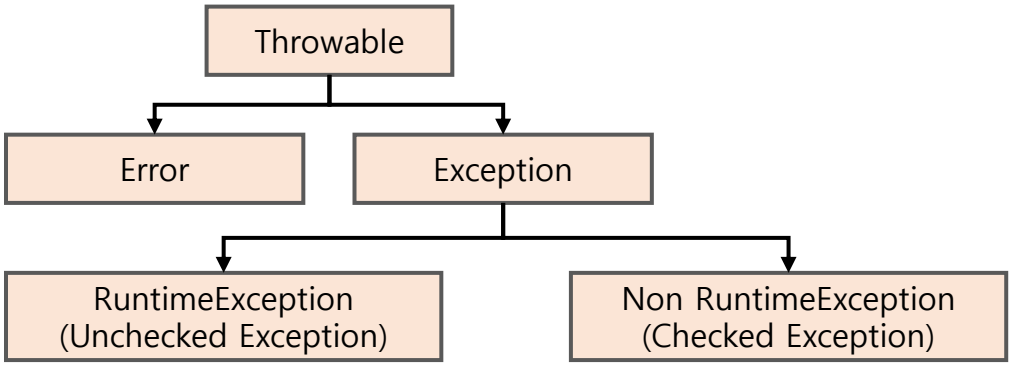
Thread를 start() 후에 별도 join()을 호출하지 않으면 main은 종료되지만, Thread가 종료될때 까지 JVM에서 대기

Java 필수 클래스 – Exception

- 예외 처리의 최상위 클래스
- 자주 사용되는 메서드:
- try-catch 문에서 사용
- getMessage(), printStackTrace() 메서드 자주 사용

Exception 처리를 하지 않으면 프로그램은 강제 종료됨

```
public class ExceptionExample {
    public static void main(String[] args) {
        try {
            int result = 10 / 0;
        } catch (Exception e) {
            System.out.println("예외 발생: " + e.getMessage());
            e.printStackTrace(); // 디버깅용
        }
    }
}
```



IOException, FileNotFoundException,
ClassNotFoundException,
SQLException (DB 접근 오류) 등

[참고] RuntimeException 과 Non-RuntimeException

Non-RuntimeException (Checked Exception)

- **외부 환경 문제에 의해 발생할 가능성이 높은 예외**
- 예측 가능하고 회복 가능(recoverable)한 경우가 많음
- Exception(단, RuntimeException 제외)을 상속한 예외
- **컴파일러가 예외 처리를 강제**
  → 반드시 try-catch or throws 선언 필요

IOException
SQLException
ClassNotFoundException
FileNotFoundException
InterruptedException

RuntimeException (Unchecked Exception)

- **개발자 실수(프로그래밍 오류)로 발생**
- 실행 중 잘못된 로직이나 데이터로 인해 발생
- RuntimeException 을 상속한 예외들
- 컴파일러가 try-catch 를 강제하지 않음

NullPointerException
ArrayIndexOutOfBoundsException
ArithmeticException
IllegalArgumentException
ClassCastException

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class CheckedExample {
    public static void main(String[] args) {
        try {
            BufferedReader reader
             = new BufferedReader(new FileReader("test.txt"));
            String line = reader.readLine();
            System.out.println(line);
            reader.close();
        } catch (IOException e) {
            System.out.println("파일을 읽는 중 오류 발생: " +
e.getMessage());
        }
    }
}
```

- readLine() 과 close() 는 IOException(Checked) 발생 가능
  → 따라서 반드시 try-catch 또는 throws IOException 필요

[참고] Non-RuntimeException의 Checked 이유

외부 상황에 영향을 받지 않도록 하기 위한 방법

1. 복구 가능성이 높은 예외이기 때문

파일, 네트워크, DB 같은 외부 리소스 문제는 언제든 발생할 수 있고,

프로그래머가 이를 적절히 처리해야 프로그램의 안정성이 유지됨.

2. 미 처리시 프로그램의 비정상 종료

체크 예외를 처리하지 않으면 앱이 중단될 수 있기 때문에 컴파일러가 강제

3. Java 언어 설계 철학

Java는 안정성을 중요시하며,

외부 자원 접근을 “안전하게” 처리하도록 강제함.

Java 필수 클래스 – Runtime

- JVM 자체를 다루는 클래스로 메모리 정보, 가비지 컬렉션, 외부 프로세스 실행 등에 사용
- 자주 사용되는 메서드:
  - Runtime.getRuntime() 으로 인스턴스를 얻음

```
public class RuntimeExample {
    public static void main(String[] args) {
        Runtime rt = Runtime.getRuntime();
        System.out.println("메모리 사용 전: " + rt.freeMemory());

        int[] bigArray = new int[1000000];
        System.out.println("메모리 사용 후: " + rt.freeMemory());

        bigArray = null;
        System.gc(); // 가비지 컬렉션 요청

        System.out.println("GC 후: " + rt.freeMemory());
    }
}
```

Java 핵심 클래스 – Class 클래스

- 자바 클래스 자체를 다루는 메타 클래스
- 자주 사용되는 메서드:
  - Class<?> 타입은 리플렉션(reflection)의 핵심
  - getClass(), Class.forName() 으로 접근

```
public class ClassExample {
    public static void main(String[] args) {
        String skala = "스칼라";
        Class<?> clazz = skala.getClass();

        System.out.println("클래스 이름: " + clazz.getName());       // java.lang.String
        System.out.println("슈퍼 클래스: " + clazz.getSuperclass()); // java.lang.Object
    }
}
```

객체지향 프로그래밍

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

객체 (Object)란?

어떤 행동을 하는쪽이 주체 라면, 그 행동의 영향을 받거나 향하는 대상이 객체

사람이 **상품**을 주문한다

- **사람**은 주문 행위를 하는 주체
- **상품**은 주문 행위가 향하는 대상, 즉 객체

주체가 객체에게 요청하고 객체는 자신의 책임을 수행한다.

- 객체는
- 메시지를 받고,
- 역할(책임)을 수행하며,
- 그 결과로 상태를 변경하거나 응답 반환

객체 (Object)란?

고객이 **은행직원**에게 **계좌 개설**을 요청한다

- **고객**은 요청 행위를 하는 **주체**
- **은행직원**은 요청 행위가 향하는 **객체**
- **계좌 개설**은 요청에 따른 **행동 (Behavior)**

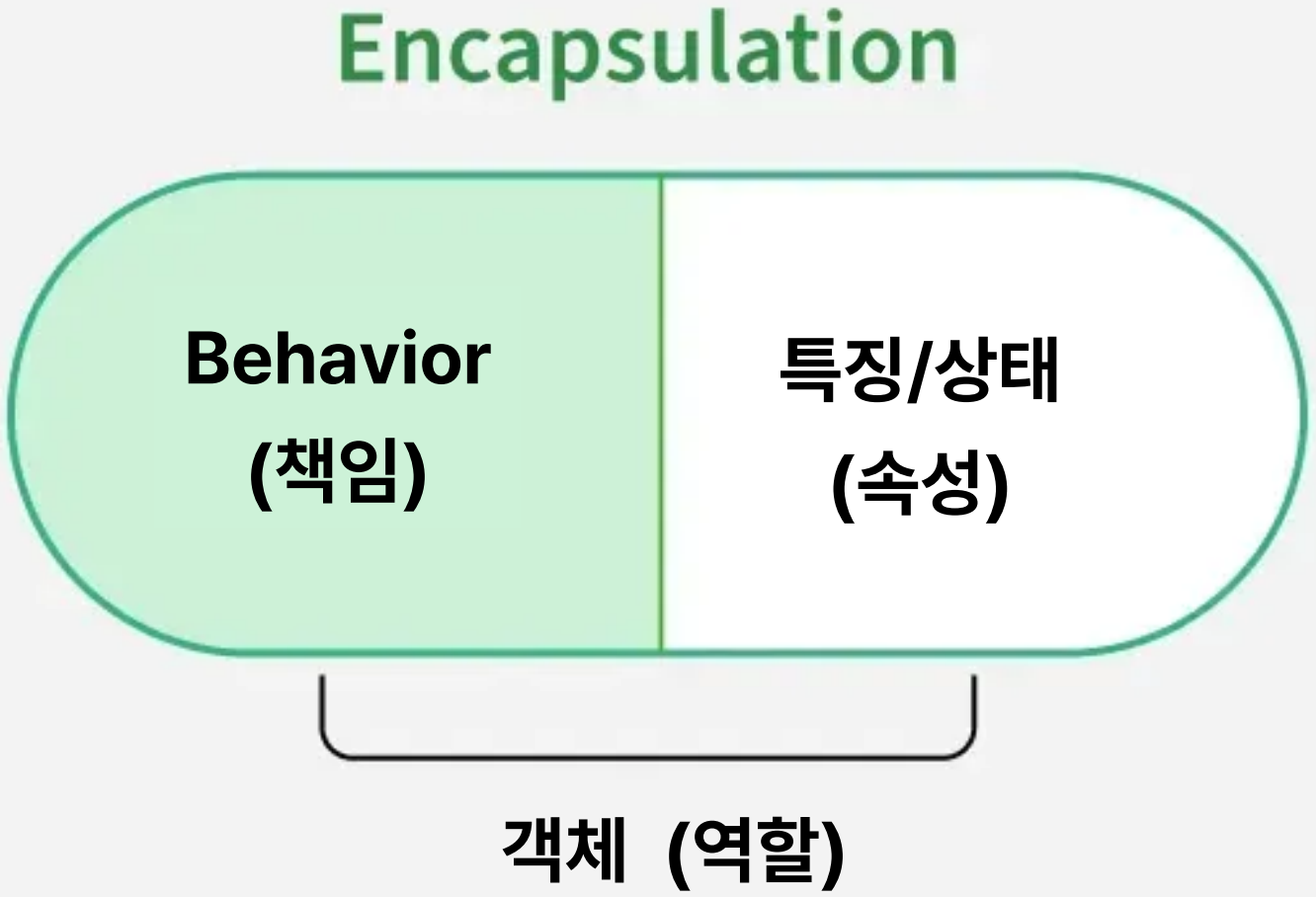
은행직원의 **특징/상태**는
- 직원 번호, 이름, 소속지점,
- 계좌 개설 권한 여부, 업무 처리 가능 시간

책임/역할은 객체가 해야 할 **일**이고, **행동**은 그 책임이 외부로 드러나는 **행위**

객체	요청 또는 자극	Behavior	속성
문	손잡이를 돌리고 민다	열린다	색상, 재질, 잠금 상태
자동차	가속 페달을 밟는다	속도를 낸다	색상, 속도, 연료량
사람	이름을 부른다	대답한다	이름, 나이, 성별, 직업
강아지	공을 던진다	뛰어가서 문다	이름, 품종, 나이
엘리베이터	버튼을 누른다	해당 층으로 이동한다	현재층, 운행상태, 수용인원

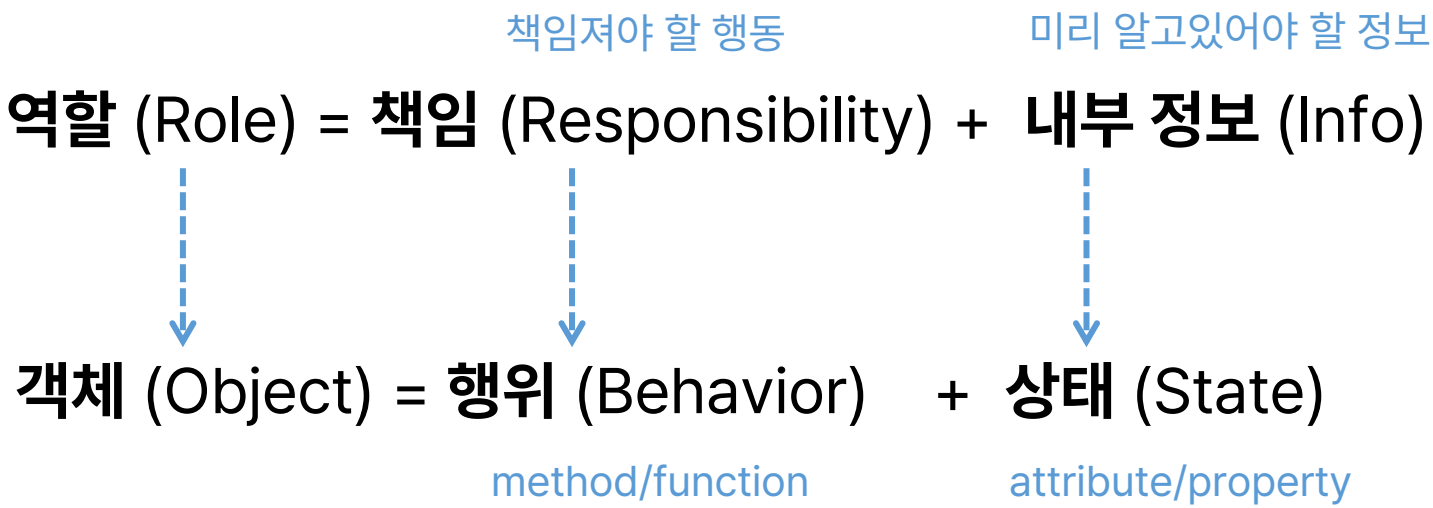
객체 (Object)란?

객체는 행동/책임 (Methods) 와 속성 (Data)를 Encapulation



객체 (Object)란?

시스템을 구성하기 개별적 역할 (Role)을 객체로 정의

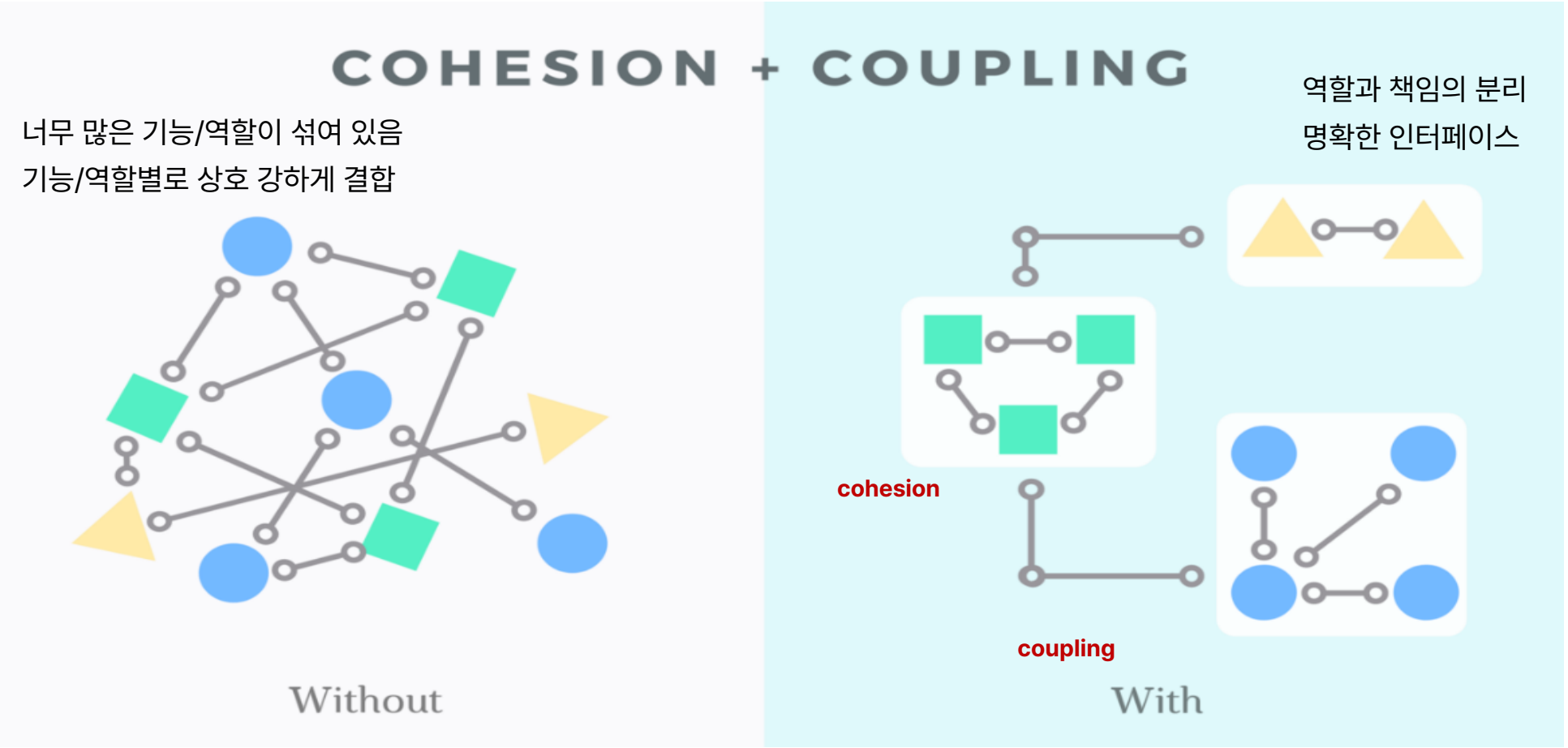


Encapsulation (캡슐화)

Cohesion (응집도)

좋은 설계란?

높은 응집도(cohesion) + 낮은 결합도 (coupling)

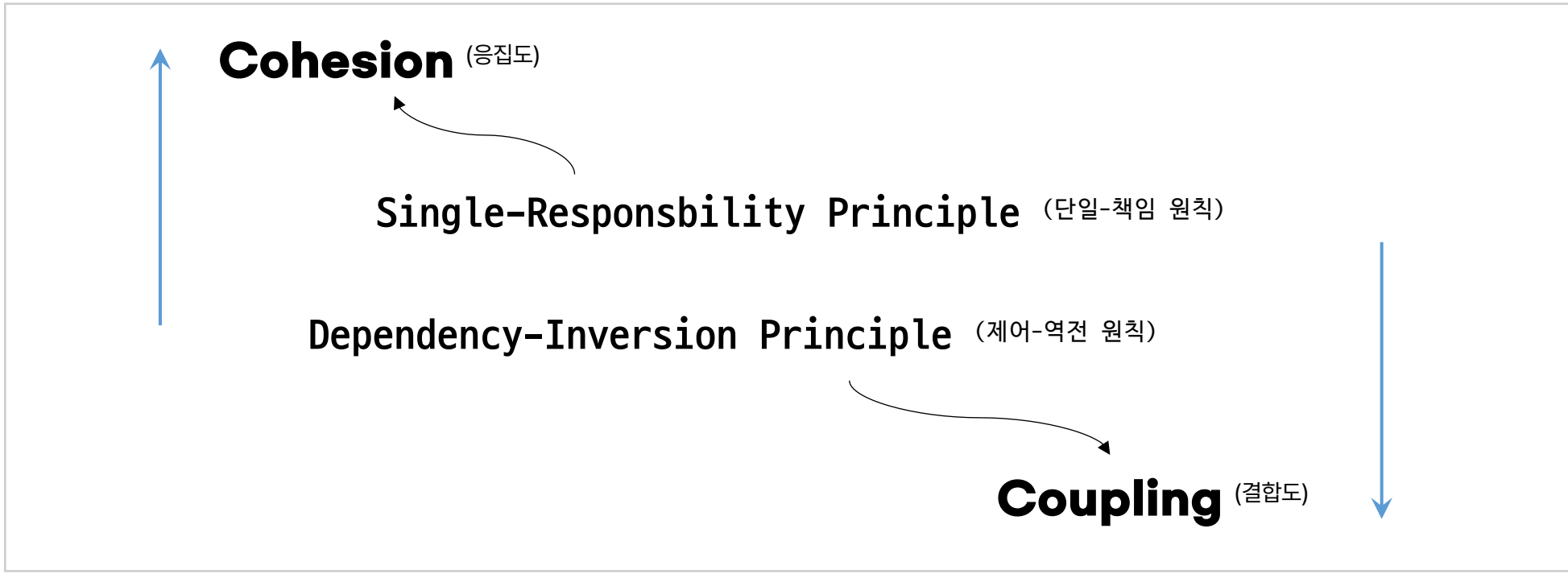


낮은 응집도 + 높은 결합도

높은 응집도 + 낮은 결합도

객체지향 프로그래밍 (Object-Oriented Programming)

시스템을 역할(Role)과 책임(Responsibility) 중심으로 객체(Object)로 모델링하여,
높은 응집도와 낮은 결합도를 통해 유지보수성과 확장성을 향상시키는 프로그래밍 패러다임



OOP의 핵심 특징

소프트웨어를 객체화하고 높은 응집도와 낮은 결합도를 제공하기 위한 핵심 설계 원칙

캡슐화 (Encapsulation) : 높은 응집도

What을 설명하는 Behavior (Method)만 노출하고 상세 구현과 속성은 숨김

- **유지보수성**: 내부 구현을 변경해도 외부에 영향을 주지 않으므로 코드를 수정하기 용이
- **보안성/안정성**: 외부에서 내부 데이터를 직접 조작하여 오염시키는 것을 방지

추상화 (Abstraction) : 낮은 결합도

인터페이스와 구현이 분리되어 향후 시스템이나 모듈로 부품을 쉽게 교체 가능

- **유지보수성**: 시스템의 복잡도를 낮추어 개발자가 코드를 더 쉽게 이해
- **상호 운영성**: 쉽게 교체 가능

상속 (Inheritance) : 중복 제거

이미 잘 만들어진 부모 클래스의 특성을 자시기 클래스가 물려받아 코드의 재사용성을 높임

- **수정 용이성**: 공통된 기능을 부모 클래스 한 곳에서만 수정하면 모든 자식 클래스에 자동 반영
- **코드 재 사용성**: 유사한 기능을 만들 때 처음부터 다시 개발할 필요가 없어 개발 속도가 빨라짐

다형성 (Polymorphism) : 낮은 결합도

동일 인터페이스 호출 시 상속 관계 다른 객체들이 각자의 방식으로 다르게 동작

- **확장성/변경용이성**: 기존의 클라이언트 코드를 전혀 수정하지 않고도, 새로운 클래스(기능)를 쉽게 추가

Java 기반 OOP

캡슐화 (Encapsulation)

- **private, protected** 등의 접근 제어자를 사용하여 데이터를 숨기고, getter 및 setter 메서드를 통해 접근을 제어함.

상속 (Inheritance)

- 기존 클래스(부모 클래스)의 속성과 기능을 새로운 클래스(자식 클래스)가 물려받아 확장하는 개념
- **extends** 키워드를 사용하여 상속 구현.

추상화 (Abstraction)

- 불필요한 세부 사항을 숨기고, 필요한 기능만을 노출하는 개념
- **abstract class** 와 **interface**로 구현

다형성 (Polymorphism)

- **메서드 오버로딩(Method Overloading):** 같은 이름의 메서드를 매개변수 타입이나 개수에 따라 다르게 정의.
- **메서드 오버라이딩(Method Overriding):** 부모 클래스의 메서드를 자식 클래스에서 재정의.

Java 기반 OOP - 캡슐화(Encapsulation)

- 객체의 속성(필드)을 외부에서 직접 접근하지 못하도록 숨기고, 공개된 메서드(getter/setter)를 통해서만 간접적으로 접근하도록 하는 것

❖ 현장에서는 대부분 Lombok을 활용하여 자동으로 getter와 setter를 생성

캡슐화 원칙	예시
필드는 private으로 선언	`private double price;`
접근은 getter/setter를 통해 제공	public double getPrice()
필요 시 setter에 유효성 검사 추가	if (price > 0) this.price = price;

```
public class Stock {
    // 1. 가격 필드는 private으로 숨김
    private double price;

    // 2. 접근은 getter를 통해 제공
    public int getPrice() {
        return this.price;
    }

    // 3. setter에 유효성 검사 추가
    public void setPrice(double price) {
        if (price > 0) { // 유효성 검사 규칙
            this. price += price;
        } else {
            System.out.println(
             ” 에러: 0원 이하 가격은 설정 불가”
            );
        }
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Stock stock = new Stock();

        // [에러] 외부에서 필드에 직접 접근 불가능
        // stock.price = -50000;

        // 정상 처리
        stock.setPrice(10000);
        // 유효성 검사로 에러 메시지 출력
        stock.setPrice(-5000);

        // getter를 통한 안전한 데이터 출력
        System.out.println(
          "현재 가격: " +
           stock.getPrice() + "원");
    }
}
```

[참고] 필드 공개와 캡슐화 비교

항목	public 필드 직접 접근	private + getter/setter (캡슐화)
접근 방식	stock.price = 10000;	stock.setPrice(10000);
간결성	간단하고 빠름	코드가 약간 길어짐
유효성 검사	불가능 (값 제한 못함)	setter에 검사 로직 가능
데이터 보호	불안전 (외부에서 자유롭게 변경 가능)	안전 (변경 제어 가능)
유지보수	변경 시 외부 코드 모두 영향 받음	내부 변경 시 외부 영향 최소화
기능 확장성	어려움 (로직 삽입 불가)	쉬움 (getter/setter에 로직 추가 가능)
OOP 원칙 준수	객체지향 위반	정보 은닉 원칙 충실
테스트 및 디버깅	예기치 않은 변경 가능성 존재	추적과 제어 쉬움

필드 공개 (public)

```
public class Stock {
    public String name;
    public double price; // 누구나 직접 수정 가능
}

Stock s = new Stock();
s.price = -1000;  // 유효하지 않은 값도 설정됨
```

캡슐화 (private)

```
public class Stock {
    private double price;

    public double getPrice() {
        return price;
    }

    public void setPrice(double price) {
        if (price > 0) this.price = price;
        else System.out.println("❌ 잘못된 가격");
    }
}

Stock s = new Stock();
s.setPrice(-1000); // → 경고 출력
```

Java 기반 OOP - 상속(Inheritance)

- 상속이란 기존(부모) 클래스의 속성과 메서드를 새로운(자식) 클래스가 물려받아 사용하는 객체지향 개념
- 상속은 extends 키워드를 사용하며, 코드 재사용성 향상과 유지보수 편의성을 제공

상속 원칙	예시
extend 키워드로 부모 클래스 상속	Public class PreferredStock extend Stock
부모클래스 생성자는 super()를 통해 호출	super(name, price);
필드는 protected 로 선언	protected String name;

부모 클래스

```
public class Stock {
    protected String name;
    protected double price;

    public Stock(String name, double price)
    {
        this.name = name;
        this.price = price;
    }

    public void printInfo()
   {
        System.out.println("[일반주] 종목: " +
        name + ", 가격: " + price + "원");
    }
}
```

자신 클래스

```
public class PreferredStock extends Stock {
    private double dividendRate;

    // 자식 생성자에서 super()로 부모 생성자 호출
    public PreferredStock(String name, double price,
double dividendRate) {
        super(name, price); // 부모 생성자 호출
        this.dividendRate = dividendRate;
    }

}


PreferredStock pstock = new PreferredStock( " 삼성”, 8000,
10);
pstock.printInfo();
```

Java 기반 OOP - 추상화(Abstraction)

- 추상화란 복잡한 시스템에서 핵심적인 개념만을 추출하고, 불필요한 세부사항은 숨기는 것
- Java에서는 abstract class 또는 interface를 사용하여 추상화를 구현

항목	추상 클래스 (abstract class)	인터페이스 (interface)
용도	상태와 일부 공통 구현을 자식들과 공유하기 위한 설계 도구	구현 없이 동작 규약만 선언하여, 다양한 객체들이 동일한 역할을 수행할 수 있게 함
다중 상속	불가능	가능
필드	인스턴스 변수, 생성자 포함 가능	상수만 가능 (public static final)
메서드	일부 구현 가능	default, static 메서드 구현 가능

Abstract Class

```
abstract class Asset {
    protected String name;
    protected double price;

    public Asset(String name, double price) {
        this.name = name;
        this.price = price;
    }

    public abstract void printInfo();
}
```

Interface

```
public interface Valuable {
    void printInfo();

}
```

Java 기반 OOP - 다형성(Polymorphism)

- 다형성이란 "하나의 타입으로 여러 형태의 동작을 표현할 수 있는 능력"을 의미
- 같은 메서드 호출이라도 객체의 실제 타입에 따라 다르게 동작하게 만드는 개념

다형성의 조건	설명
오버라이딩(Overriding)	자식 클래스가 부모의 메서드를 재정의해야 함
오버로딩 (Overloading)	동일 클래스에 동일이름으로 입력 파라메터만 다른 메소드 정의

오버라이딩

```
class Human {
   void introduce() {
      System.out.println(“저의 이름은” + name + “입니
다”);


class Student extends Human {

   @Override
   void introduce() {
      System.out.println (“저의 이름은 “ + name +
        ”이고, 학생 Id는 “ + studentId + “입니다”);
   }
}
```

오버 로딩

```
class Student extends Human {

   @Override
   void introduce() {
      System.out.println (“저의 이름은 “ + name +
        ”이고, 학생 Id는 “ + studentId + “입니다”);
   }

   void intrduce(Integer myId) {
      System.out.println (“저의 이름은 “ + name +
        ”이고, 나의 Id는 “ + myId + “입니다”);
   }
}
```

Java 기반 OOP – 업캐스팅 (Upcasting)

- 자식 객체를 부모 타입으로 참조

업캐스팅 / 다운캐스팅 예시

```
public class Main {
    public static void main(String[] args) {
        // 업캐스팅
        Stock s = new PreferredStock();  // PreferredStock(자식) → Stock (부모)

        s.printInfo();          // 자식의 오버라이딩된 메서드 실행
        s.showDividend();       // 오류: 부모 타입에서는 보이지 않음

        // 다운캐스팅 (다시 자식 타입으로 캐스팅)
        if (s instanceof PreferredStock) {
            PreferredStock ps = (PreferredStock) s;    // 다운 캐스팅
            ps.dividendRate = 5.0;
            ps.showDividend(); // 가능
        }
    }
}
```

[참고] 업케스팅 이유

구현과 사용 (절차/흐름)의 분리

- **사용 (Client/Interface)**: 코드를 사용하는 주체는 **무엇(What)을 할 수 있는가**에만 관심이 있고, 내부가 어떻게 돌아가는지는 알 필요가 없음
- **구현 (Implementation)**: **어떻게(How) 동작하는가**를 실제로 작성하는 영역

```
// 1. 사용 방법을 정의
class ElectronicProduct {
    void turnOn() {
        System.out.println("제품의 전원을 켬");
    }
}

// 2. 구체적인 구현들 (자식 클래스들)
class Television extends ElectronicProduct {
    @Override
    void turnOn() {
       System.out.println("TV 화면이 켜짐");
    }
}

class Audio extends ElectronicProduct {
    @Override
    void turnOn() {
       System.out.println("오디오: 음악이 나옴");
    }
}
```

```
// 3. 실제 실행 및 사용 영역
public class Main {
   // 이 메서드는 새로운 가전제품이 추가되어도
   // 절대 수정될 필요가 없음! (사용과 구현의 분리)
    static void pressPowerButton(ElectronicProduct product) {
        product.turnOn();
    }

    public static void main(String[] args) {
        // 자식 인스턴스를 부모 타입 변수에 담음 (업캐스팅)
        ElectronicProduct tv = new Television();
        ElectronicProduct audio = new Audio();

        // 사용처에서는 똑같은 버튼(메서드)만 클릭
        pressPowerButton(tv);     // 출력: TV 화면이 켜짐
        pressPowerButton(audio);  // 출력: 오디오에서 음악 출력
    }
}
```

[실습] 상속 실습

01.training-code/07.oop/stock

부모 클래스 Stock, 자식 클래스 PreferredStock 코드를 검토하고 실행해보자
printStockInfo의 UpperCast를 확인하고, 실행해보고 그 결과를 확인해보자

```
public class Main {
    static void printStockInfo(Stock stock) {
        stock.printInfo();
    }

    public static void main(String[] args) {
        Stock stock1 = new Stock("삼성전자", 80000);
        Stock stock2 = new PreferredStock("LG전자", 60000, 5.0);

        stock1.printInfo();
        stock2.printInfo();

        printStockInfo(new Stock("SAKALA", 80000))  ;
        printStockInfo(new PreferredStock("Netflix", 60000, 10.0));
    }
}
```

```
> cd 07.oop/sotck

> javac -d bin *.java
> Java -cp bin Main
```

[참고] 절차적 프로그래밍 vs 객체 지향 프로그래밍

- 절차적 프로그래밍(Procedural Programming)과 객체 지향 프로그래밍(Object-Oriented Programming, OOP)은 소프트웨어 개발의 두 가지 대표적인 패러다임

구분	절차적 프로그래밍	객체 지향 프로그래밍
기본 개념	순차적으로 명령어를 실행하여 문제 해결	객체라는 단위로 데이터를 추상화하고, 그 객체 간 상호작용으로 문제 해결
구조	함수 중심 (Function-based)	클래스와 객체 중심 (Class-based)
데이터 처리	데이터와 함수를 분리	데이터를 객체 내부에서 캡슐화하여 처리
재사용성	함수 재사용에 초점	클래스/객체 단위로 재사용 및 상속
유지보수성	복잡한 프로그램에서 유지보수 어려움	캡슐화, 상속, 다형성으로 유지보수 용이
주요 개념	순차, 조건, 반복	캡슐화, 상속, 다형성, 추상화
대표 언어	C, Pascal, Fortran 등	Java, C++, Python, C# 등
실행 흐름	위에서 아래로 순차적으로 흐름 제어	객체 간 메시지 전달을 통해 흐름 제어
성능	상대적으로 단순하고 빠름	객체 생성 및 메소드 호출 오버헤드 존재
예제	main() 함수에서 순차적으로 함수 호출	클래스 선언 → 객체 생성 → 메서드 호출

[참고] 프로그래밍 예시(C 스타일)

- **절차적 프로그래밍 (C 스타일)**

```c
#include <stdio.h>

void greet() {
    printf("Hello, World!\n");
}

int main() {
    greet();
    return 0;
}
```

- **객체지향 프로그래밍 (Java 스타일)**

```java
public class Greeter {
    public void greet() {
        System.out.println("Hello, World!");
    }

    public static void main(String[] args) {
        Greeter g = new Greeter();
        g.greet();
    }
}
```

[참고] static 클래스, 변수 , 메소드

static 키워드는 객체가 아니라 클래스 자체에 속사는 변수나 메소드를 정의할때 사용

```
public class ExampleTest {

    static class Example {
        public static final int INITIAL_COUNT = 10;     //Metaspace 의 클래스 메타데이터에 상수 풀에 존재
        private static int count = 0; // Metaspace 클래스 메타데이터에는 Heap 내 java.lang.Class의 Index 정보
        private int instanceId;

        Example() {
            count++;
            instanceId = count + INITIAL_COUNT;
        }

        public static int getLastInstanceId() {
            return count + INITIAL_COUNT;
        }

        public static int getCount() {
            return count;
        }

        public int getInstanceId() {
            return instanceId;
        }
    }
```

[참고] static 클래스, 변수 , 메소드

static 키워드는 객체가 아니라 클래스 자체에 속사는 변수나 메소드를 정의할때 사용

```
public class ExampleTest {

// { 앞의 코드 }


    public static void main(String[] args) {
        Example e1 = new Example();
        Example e2 = new Example();
        Example e3 = new Example();

        System.out.println("총 생성된 객체 수(count): " + Example.getCount());
        System.out.println("마지막 생성된 instanceId: " + Example.getLastInstanceId());

        System.out.println("e1의 instanceId: " + e1.getInstanceId());
        System.out.println("e2의 instanceId: " + e2.getInstanceId());
        System.out.println("e3의 instanceId: " + e3.getInstanceId());
    }
}
```

[참고] 설계 추상화 단위의 확장

절차적 (함수) : 코드 중복, 날코딩

객체 지향 (OOP) : 코드의 재사용, 패턴을 통한 변경 영향 최소화

컴포넌트기반 (CBD) : 컴토넌트 단위 재사용, 변경 영향 최소화

서비스 지향 아키텍처 (SOA): 컴포넌트를 실행단위로 확장, 서비스 통합 중심

마이크로서비스 아키텍처 (MSA): 빠른 개발, 빠른 변화 대응

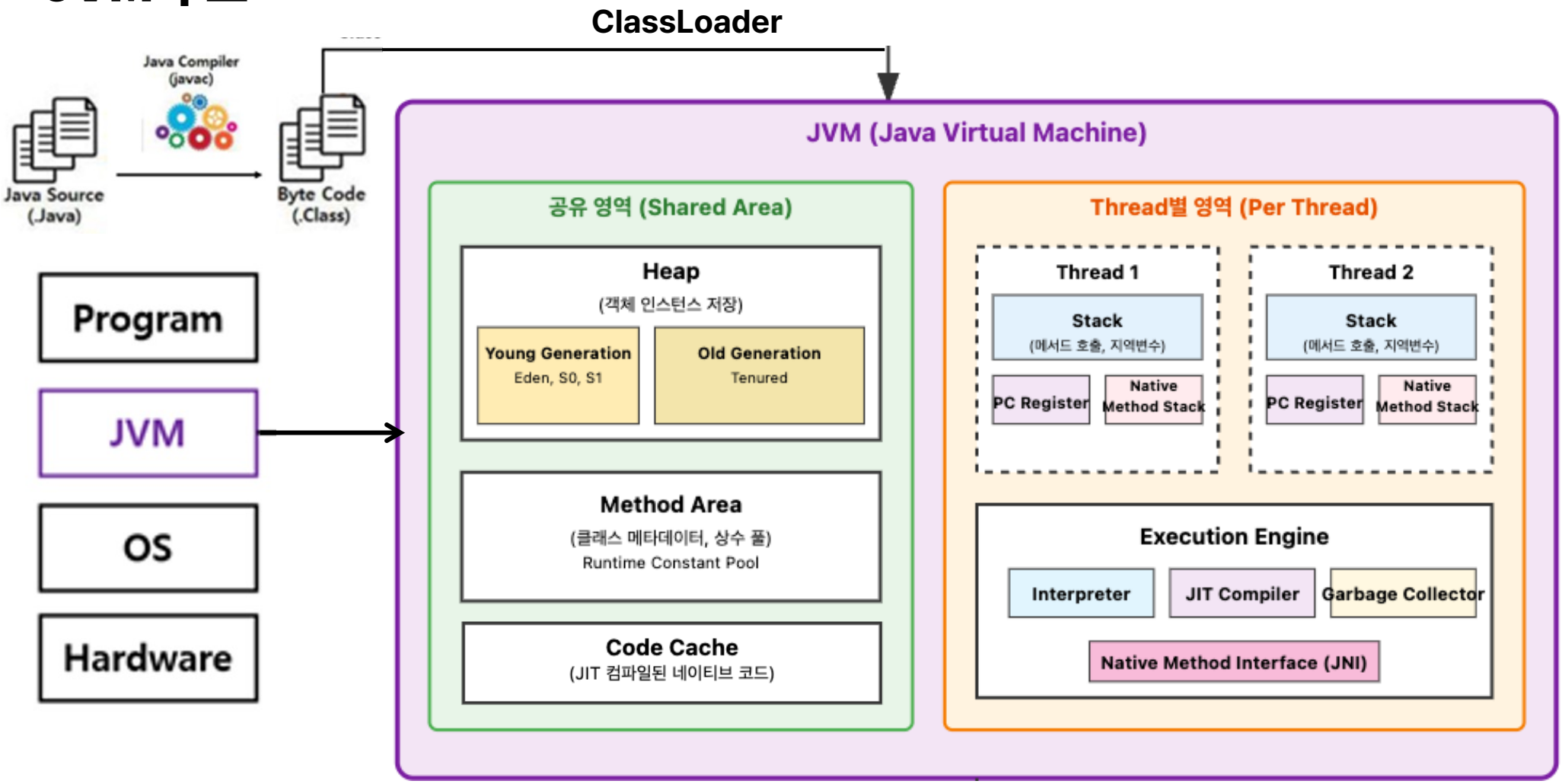
```
com.example
 ├ order
 │    ├ api           (OrderController, DTO)
 │    ├ application   (OrderService/UseCase)
 │    ├ domain        (Order, OrderItem, Policy, Event)
 │    └ infra  (JpaOrderRepository, FeignClient, Outbox 등)
 ├ payment
 │    ├ api
 │    ├ application
 │    ├ domain
 │    └ infra
 └ shipping
      ├ api
      ├ application
      ├ domain
      └ infra
```

Java 내부 구조 이해

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

JVM 구조



GC 과정에서는 모든 JVM 내 Thread 는 중단 (STW: Stop the World).

이로인한 성능 저하 및 처리 중단 발생

JDK 15부터 Z GC, 메모리 최적화, 동시 병렬처리 (Thread 미 중단) 등을 활용해서 성능 저하 최적화

자바와 JVM

클래스 로더

xx.class 파일을 로드하고, jar 파일 내 저장된 클래스를 JVM의 Metaspace (Heap 외부)에 탑재하는 역할

Metaspace (Method Area 구현체)

- **클래스 메타데이터**: 이름, 접근 제어자, 부모 클래스의 이름, 인터페이스 이름 메타 정보
- **메서드 메타데이터**: 이름, 반환 타입, 파라미터 타입, Bytecode
- **어노테이션 정보** 및 **필드/메서드의 구조적 레이아웃** 정보

참고: static variable, constant → Heap java.lang.Class

실행 엔진 (Execution Engine)

- 인터프리터와 JIT(Just In Time)
- 가비지 컬렉터 기능

RunTime Data Area

프로그램 수행을 위해 OS에서 할당 받은 메모리 공간

```
├ Method Area (공유)
├ Heap (공유)
├ Java Stack (스레드별)
├ PC Register (스레드별)
├ Native Method Stack (스레드별)
```

[참고] JVM – RunTime Data Area

Method Area

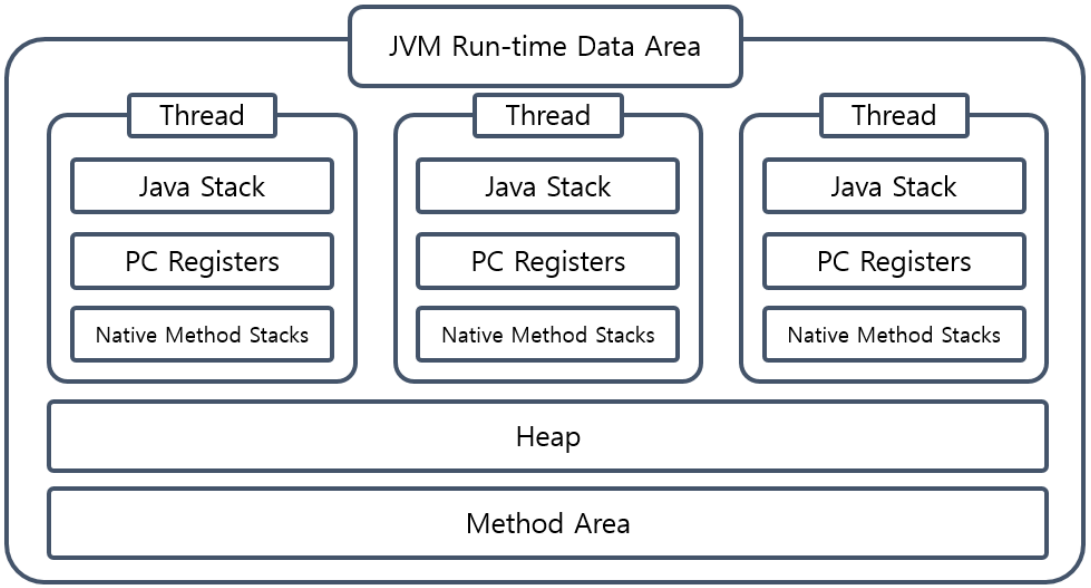
: 클래스 수준 정보 저장소

- 클래스 메타데이터
  (클래스 이름, 필드, 메서드 시그니처)
- 메서드 바이트코드

Heap

: 객체 (Object) 인스턴스 저장소

- new로 생성된 모든 객체
- 배열과 대부분의 메모리 사용량



Stack

: 메서드 실행 단위 저장송

- Thread 별로 하나씩 존재
- 메서드 호출 시 Stack 생성

PC Register (Program Counter)

: 현재 실행중인 바이트코드 위치
(여러 명령줄의 index)

- 각 스레드마다
- 다음에 실행할 명령 주소를 가르킴

```
a = b + c;   ⇒   0: iload_2      // b
                 1: iload_3      // c
                 2: iadd         // b + c
                 3: istore_1     // a = 결과
```

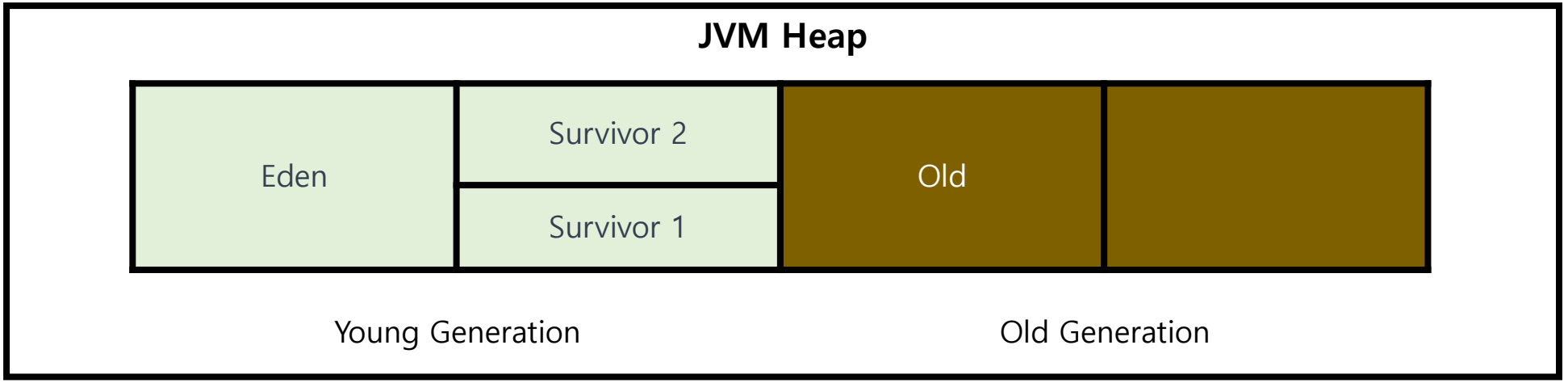
Native Method Stack

: JNI 기반 네이티브 코드 실행용

- C / C++ 로 작성된 메서드 호출 시 사용
- Java Stack과 유사한 구조

Heap 영역

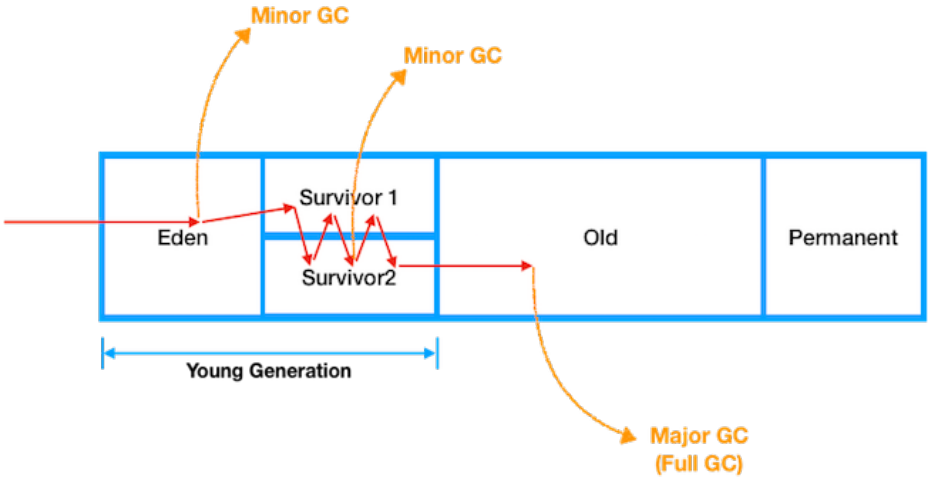
- Java 애플리케이션에서 생성된 객체와 배열이 저장되는 런타임 메모리 공간
- Garbage Collector에 의해 관리되며 JVM 성능에 영향을 줌



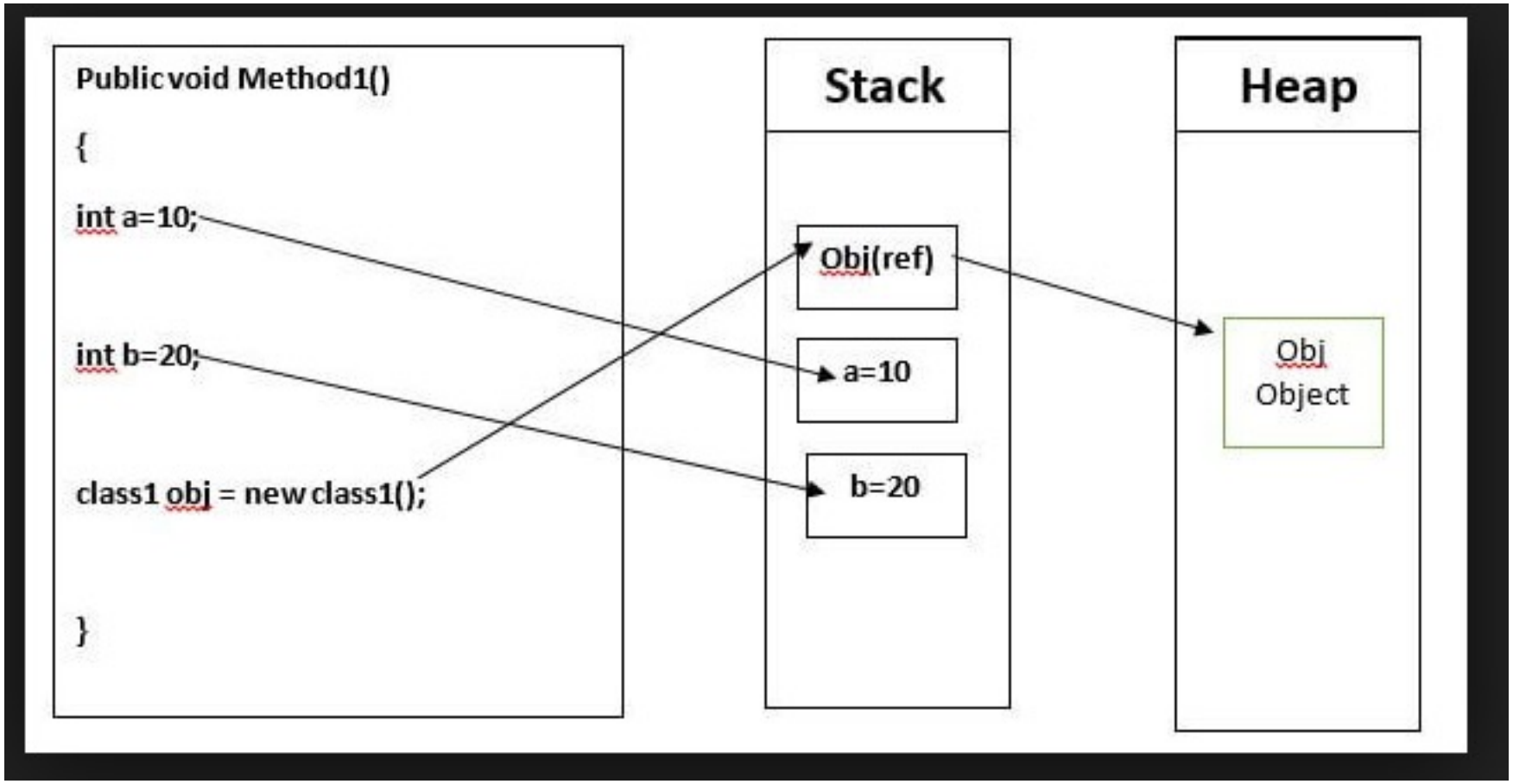
영역	설명	특징	GC 영향
Eden	새 객체가 생성되는 공간	대부분의 객체가 짧게 존재	Minor GC 대상
Survivor S0/S1	Eden에서 살아남은 객체가 이동	GC 후 번갈아 복사	Minor GC 대상
Old	생존 시간이 긴 객체 저장	대형 객체, 오랜 객체	Full GC 대상
Metaspace	Method Area의 구현체	Native 메모리 사용	미 대상

가비지 컬렉션 (Garbage Collection)

- 자바의 메모리 관리 방법 중의 하나로 JVM(자바 가상 머신)의 Heap 영역에서 동적으로 할당했던 메모리 중 필요 없게 된 메모리 객체(garbage)를 모아 주기적으로 제거하는 프로세스
- **Minor Garbage Collection**
  - Young Generation 영역에서 발생되는 GC를 Minor GC로 정의
  - Young Generation은 Old 대비 상대적으로 객체 정보가 작기 때문에 객체를 찾아 제거하는데 적은 시간 소요
- **Major Garbage Collection**
  - Old 영역에서 발생되는 GC를 Major GC로 정의 (Full GC로도 불린다)
  - Old 영역의 객체들은 Young Generation에서 살아남은 객체로 대상 객체가 크고 많아서 실행 속도가 느림
    (여러곳 참조, 참조구조 복잡, 여러 메모리에 분산된 메모리 접근, 객체 크기 증가)



[참고] 메모리 Stack vs Heap



상속

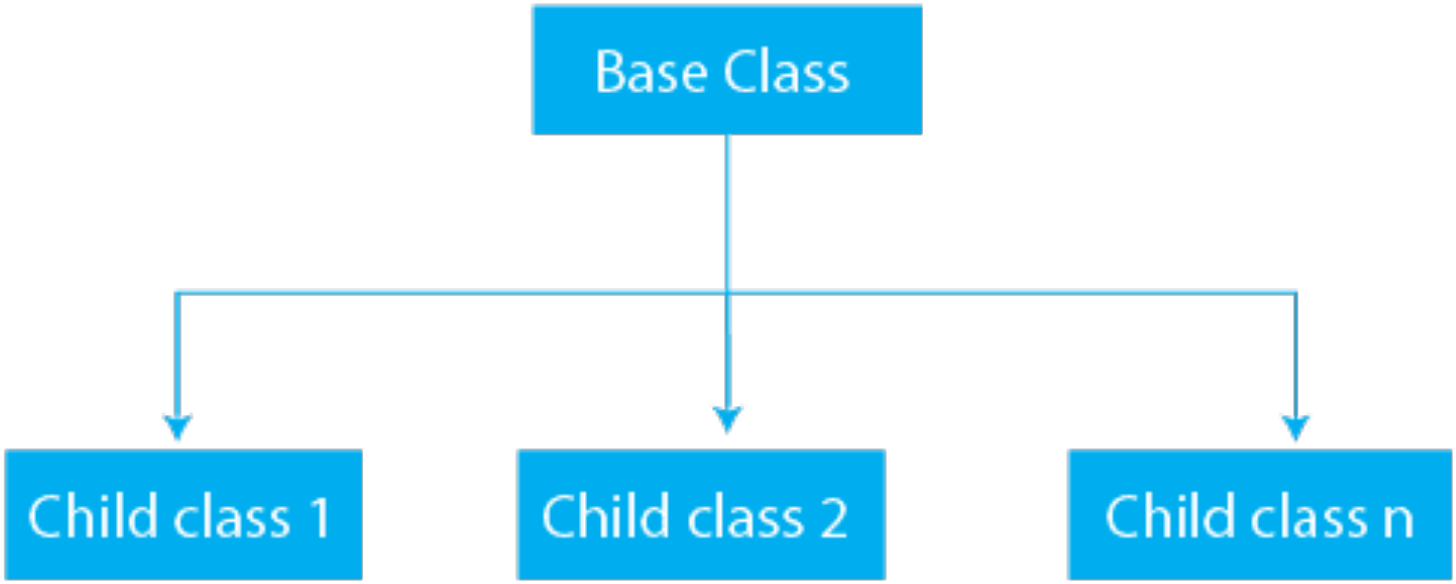
본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

상속 (Interitance)

기존 클래스(부모 클래스, Superclass)의 상태(State)와 행위(Behavior)를 자식 클래스(Subclass)가 물려받아 **재사용하고 확장할 수 있도록 하는 기법**

일반적으로 기능을 물려받는 쪽을 자식 클래스, 물려주는 쪽을 부모 클래스라고 함



접근 제한자 (Access Modifiers)

클래스 내 멤버 (Field, Method)는 접근 제안에 따라 상속 여부가 결정

접근 제한자	같은 패키지에서 접근	다른 패키지에서 접근	상속받은 자식 클래스에서 접근 가능 여부
`public`	O	O	O
`protected`	O	O (단, 상속받은 경우만)	O
(default)	O	X	X
`private`	X	X	X

- `public` → 어디서든 접근 가능. 당연히 자식 클래스에서도 사용 가능.
- `protected` → 같은 패키지에서는 자유롭게 접근 가능하며, 다른 패키지에서도 상속받은 경우 접근 가능.
- **(default, 패키지 접근제한자)** → 같은 패키지에서는 접근 가능하지만, 다른 패키지에서는 접근 불가능.
- `private` → 오직 해당 클래스 내부에서만 접근 가능. 자식 클래스에서도 직접 접근 불가능.

접근 제한자 (Access Modifiers)

부모 클래스

```
class Parent {
    public String publicField = "Public Field";
    protected String protectedField = "Protected Field";
    private String privateField = "Private Field";

    public void publicMethod() {
        System.out.println("Public Method");
    }

    protected void protectedMethod() {
        System.out.println("Protected Method");
    }

    private void privateMethod() {
        System.out.println("Private Method");
    }
}
```

자식 클래스

```
class Child extends Parent {
    void accessParentMembers() {
        // public 멤버 접근 가능
        System.out.println(publicField);
        publicMethod();

        // protected 멤버 접근 가능
        System.out.println(protectedField);
        protectedMethod();

        // private 멤버 접근 불가능 (컴파일 오류 발생)
        // System.out.println(privateField);  // 오류!
        // privateMethod();  // 오류!
    }
}
```

추상화: 인터페이스 이해하기

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

인터페이스의 구조

- 클래스가 구현해야 하는 메서드의 규약(계약, contract)를 정의하는 참조 타입
- 메서드 시그니처 (이름, 매개변수, 반환형) 만 선언하고 실제 구현은 인터페이스를 구현한 클래스가 담당

```
public interface Animal{

    void makeSound();

}
```

- class 키워드 대신 interface 키워드 활용
- 메소드의 정의만 해두고 구현체는 존재하지 않음

※ 메서드 시그니처 = 메서드 이름 + 파라미터 타입 목록
→ 메서드 이름, 파라미터의 개수, 파라미터의 순서, 파라미터의 타입

인터페이스 구현

인터페이스 선언

```
interface Animal{
    void makeSound();
}
```

상속 클래스 선언 (구현체)

```
Public class Dog implements Animal {
  @Override
  public void makeSound() {
  System.out.println("멍멍!");
  }
}

public class Cat implements Animal {
    @Override
    public void makeSound() {
        System.out.println("야옹!");
    }
}
```

Main 함수

```
public class AnimalSound {
    public static void main(String[] args) {
        Animal dog = new Dog();
        Animal cat = new Cat();

        dog.makeSound(); // 출력: 멍멍!
        cat.makeSound(); // 출력: 야옹!

        handleAnimalSound(cat); // 메서드로 전달
    }

    static void handleAnimalSound(Animal animal)
    {
        animal.makeSound();
    }
}
```

@Override Annotation 사용 이유

Override annotation은 필수는 아니지만 다음과 같은 이유로 사용 권고

1. 컴파일러 체크 기능

- @Override를 사용하면 부모 클래스(또는 인터페이스)의 메서드를 정확하게 재정의했는지 컴파일러가 검사
- 만약 오타가 있거나, 부모 클래스에서 메서드가 삭제되었을 경우, 컴파일 오류가 발생하여 실수를 방지

2. 가독성 향상

- @Override가 있으면 "이 메서드는 부모의 메서드를 재정의한 것이다"라는 정보를 명확하게 전달할 수 있어 코드 가독성이 좋음

3. 리팩토링 안정성

- 부모 클래스의 메서드 시그니처가 변경될 경우, @Override가 없는 경우에는 오류가 발생하지 않지만, 실행 시 문제가 발생 가능 존재
- @Override가 있으면 컴파일러가 문제를 감지하고 수정하도록 지원

인터페이스 메서드 접근 제어자

인터페이스의 메서드는 기본적으로 public이지만, 다른 접근 제어자 지원 가능

- 모든 메서드는 기본적으로 public 이며, 명시하지 않는 경우 자동 적용

```
interface Animal {
    void eat(); // public abstract 메서드

    // Java 9부터 가능한 private 메서드
    private void helperMethod() {
      System.out.println("This is a private method");
    }

    // default 메서드에서 private 메서드 호출 가능
    default void sleep() {
        helperMethod();
        System.out.println("Sleeping...");
    }
}
```

문제

라이브러리나 API 인터페이스에 새로운 메소드 추가 시

기존 모든 구현 클래스 컴파일 에러 발생 (하위호환성 깨짐)

해결

- default 메소드 구현
- 중복되는 내부 로직을 재사용하기 위한 private 추가

  (외부 구현 클래스에서 호출 불가)

```
interface MathUtil {
    static int square(int x) {
        return x * x;
    }
}

int twoSquare = MathUtil.square(2)
```

클래스 처럼 인터페이스도 자기 자신만의 유틸리티 메서드 지원

- 객체 생성 없이 인터페이스 명으로 직접 호출 가능

  예) Collections.sort(list)

다중 상속

자바는 다중 상속을 지원하지 않음 (대신 Interface 를 활용하여 다중 상속 지원)

다중 상속 지원 시 문제점

```
class A {
  void print() {
    System.out.println(“A”);
  }
}
class B {
  void print() {
    System.out.println(“B”);
  }
}
class C extends A, B { }

C printWhat = new C();
c.print() // A? B?
```

인터페이스 다중 상속

```
interface Flyable {
    void fly();
}
interface Swimmable {
    void swim();
}

abstract class Bird {
 public void moving () { System.out.println ("움직입니다!");}
 public void fly();
}

// 하나의 클래스가 두 인터페이스를 모두 구현
class Duck extends Bird implements Flyable, Swimmable {
    @Override
    public void fly() {
        System.out.println("오리가 하늘을 납니다!");
    }
    @Override
    public void swim() {
        System.out.println("오리가 물 위를 헤엄칩니다!");
    }
}

Duck duck = new Duck();
duck.fly();
duck.swim();
duck.moving();
```

[실습] Interface 실습

01.training-code/07.oop/stock-interface

07.oop/stock 코드에서기존 Stock 클래스를 Stock Interface로 변경해서 stock 코드가 동작하게 변경해보자

1. 01.training/07.oop/stock-interface 디렉토리를 생성하고, 01.training/07.oop/stock 의 PreferredStock.java, Stock.java, Main.java를 복사한다.
2. stock.java를 interface로 변경한다.
3. PreferredStock.java와 Main.java를 Interface Stock 에 맞게 수정한다.

```
public interface Stock {
    String getName();

    double getPrice();

    default void printInfo();
}
```

추상화: 추상 클래스

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

추상 클래스

- **추상 클래스 = Concrete Class + Interface**
- 최소 하나의 추상 메소드를 포함하고 있으면 추상 클래스이며, 직접 객체화 불가
- 공통 기능은 구현, 클래스별 개별 기능은 하위 클래스에서 구현하도록 강제

```
class Human {
    public String name;
    Human(String name) {
        this.name = name;
    }
}
```

```
abstract class Programmer extends Human {
    void introduce() {
        System.out.println(“my name is ” + this.name);
    }
    abstract void writeCode();
}
```

추상 클래스 예시

추상 클래스 정의

```
abstract class Animal {
    String name;

    public Animal(String name) {
        this.name = name;
    }

    // 하위 클래스에서 반드시 구현해야 함
    abstract void makeSound();

    public void sleep() {
        System.out.println(name + " is sleeping...");
    }
}
```

Main

```
public class Main {
    public static void main(String[] args) {
        Animal dog = new Dog("Buddy");
        Animal cat = new Cat("Kitty");

        dog.makeSound();  // Buddy says: Woof!
        cat.makeSound();  // Kitty says: Meow~

        dog.sleep();      // Buddy is sleeping...
        cat.sleep();      // Kitty is sleeping...
    }
}
```

구현 클래스

```
class Cat extends Animal {
    public Cat(String name) {
        super(name);
    }

@Override
    void makeSound() {
        System.out.println(name + " says: Meow~");
    }
}



class Dog extends Animal {
    public Dog(String name) {
        super(name);
    }

    @Override
    void makeSound() {
        System.out.println(name + " says: Woof!");
    }
}
```

[실습] abstract 실습

01.training-code/07.oop/stock-abstract

07.oop/stock 코드에서 Stock 클래스를 abstract 로 변경해서 stock 코드가 동작하게 변경해보자

1. 01.training/07.oop/stock-abstract 디렉토리를 생성하고,
   01.training/07.oop/stock 의 PreferredStock.java, Stock.java, Main.java를 복사한다.
2. stock.java를 abstract로 변경한다.
3. Stock.java의 printInfo() Method를 abstract method로 변경한다

[실습] abstract 실습

01.training-code/07.oop/Calculator

Calcuator 프로그램에서 Calculator 클래스를 abstract로 전환하는 코드를 검토해보자

1. 기존 Calcuator에서 입력은 그대로 두고 계산 (calculate) 과 이력 출력(printHistory) 부분의 구현을 추상화
2. MyCalculator.는 Calcuator를 상속해서 세부적인 동작인 계산과 이력 출력을 구현

이런 방식으로 구현을 하면 어떤 관점에서 효과가 있을까?

- 코드 재사용성 향상
- 전체 알고리즘 흐름의 표준화 및 관리 용이성
- 확장성 극대화
- 관심사의 분리

다형성

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

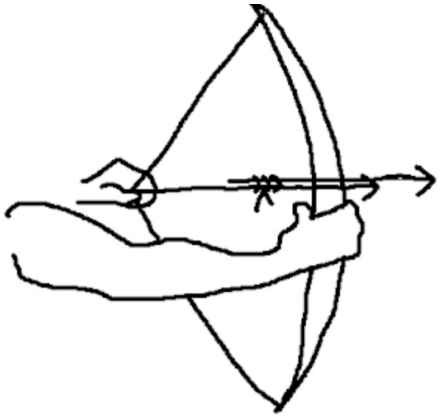
SK 주식회사 C&C

다형성 (Polymorphism)

하나의 타입 (클래스 또는 인터페이스)으로 여러 형태의 객체를 다룰 수 있는 능력
동일한 메서드 호출이 실행 시점에 실제 객체 타입에 따라 다르게 동작

오버라이딩 (Overriding) : 기존의 것을 덮어쓰다

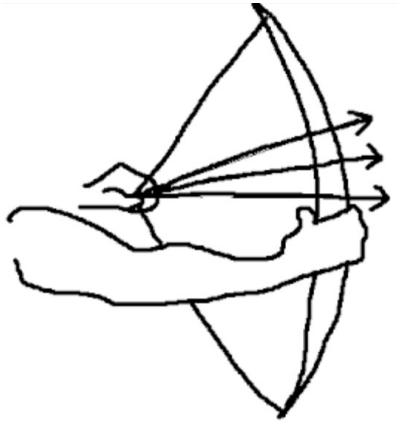
- 부모 클래스의 메소드를 자식 클래스가 동일한 형태로 또다시 구현하는 행위
- "덮어쓰기"



Overriding

오버로딩 (Overloading) : over (초과하여) + load (짐을 싣다)

- 입력 파라미터가 다른 경우에는 동일한 이름으로 메소드를 만들 수 있음
- 파라메터를 다르게 정의하고 리턴값도 다르게 적용 가능



Overloading

단, 리턴값만 다르게 설정은 안됨
Method Signature 구조는 리턴을 관리하지 않기때문에 리턴만 다른경우 Conflict 발생

메소드 오버라이딩(Overriding) 예시

부모 클래스

```
class Human {
    String name;

    // 생성자
    public Human(String name) {
        this.name = name;
    }

    // introduce 메서드
    void introduce() {
        System.out.println("저의 이름은 " + name + "입니
다.");
    }
}
```

Main

```
public class Main {
    public static void main(String[] args) {
        Human person = new Human("홍길동");
        Student student = new Student("김철수",
"S12345");

        person.introduce();
        student.introduce();
    }
}
```

자식 클래스

```
// Student 클래스 (Human 클래스를 상속)
class Student extends Human {
    String studentId;

    // 생성자
    public Student(String name, String studentId) {
        super(name); // 부모 클래스의 생성자 호출
        this.studentId = studentId;
    }

    @Override
    void introduce() {
        System.out.println("저의 이름은 " + name + "
이고, 학생 ID는 " + studentId + "입니다.");
    }
}
```

메소드 오버로딩(Overloading) 예시

Student 상속 클래스

```
class HighSchoolStudent extends Student {
    String highSchool;
    String subject;

    public HighSchoolStudent(    String name,
                                 String studentId,
                                 String highSchool,
                                 String subject) {
        super(name, studentId);
        this.highSchool = highSchool;
        this.subject = subject;
    }

    @Override
    void introduce() {
        System.out.println("저의 이름은 " + name + "이고,
학생 ID는 " + studentId + "입니다. 고등학교는 " +
highSchool + "입니다.");
    }

    // introduce 메서드 오버로딩
    void introduce(String message) {
        introduce(); // 위의 introduce() 호출
        System.out.println("제 전공은 " + subject + "입
니다. 나는 " + message + "을 추가하고 싶습니다");
    }
}
```

Main Class

```
public class Main {
    public static void main(String[] args) {
        HighSchoolStudent highStudent = new
HighSchoolStudent("John", "S9876", "DEF 고등학교", "
수학");
        highStudent.introduce("안녕하세요");
        highStudent.introduce();
  }
}
```

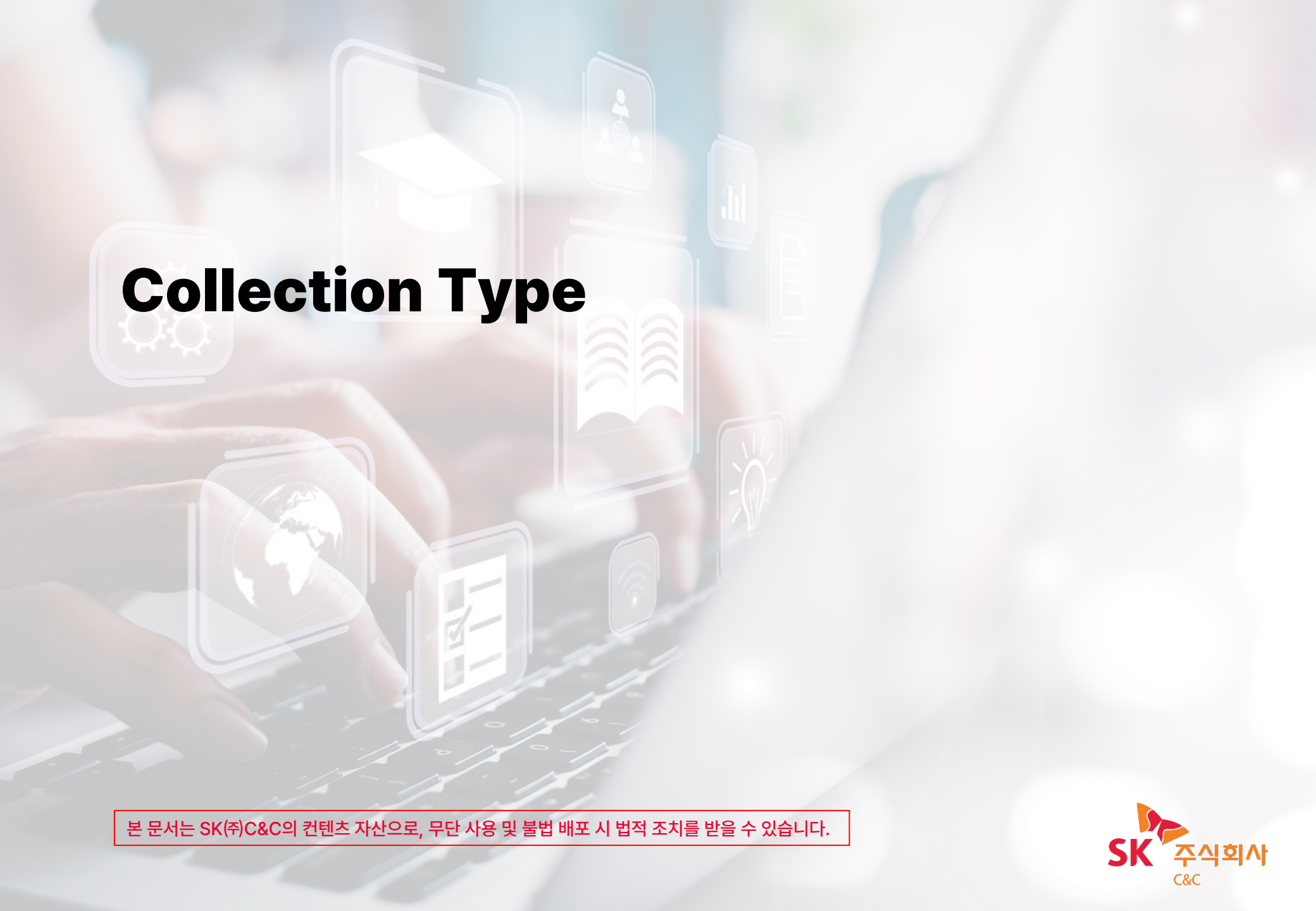
[실습] 다형성 실습

01.training-code/07.oop/stock-poly

앞에서 작업했던 stock 코드에 다음을 확인하고 추가해보자

1. PreferredStock.java의 printInfo()에 대해 Overriding과 Overloading을 적용해보자
2. Main.java에서 다양성 메소드 모두를 호출해보자

```
 // 메서드 오버라이딩
@Override
public void printInfo() {
    System.out.println(
        "[우선주] 종목: " + getName() +
        ", 가격: " + getPrice() +
        "원, 배당률: " + dividendRate + "%”
    );
}

// 메서드 오버로딩
public void printInfo(String prefix) {
    System.out.println(
        prefix + "[우선주] 종목: " + getName() +
                 ", 가격: " + getPrice() +
                 "원, 배당률: " + dividendRate + "%”
    );
}
```



Collection Type

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

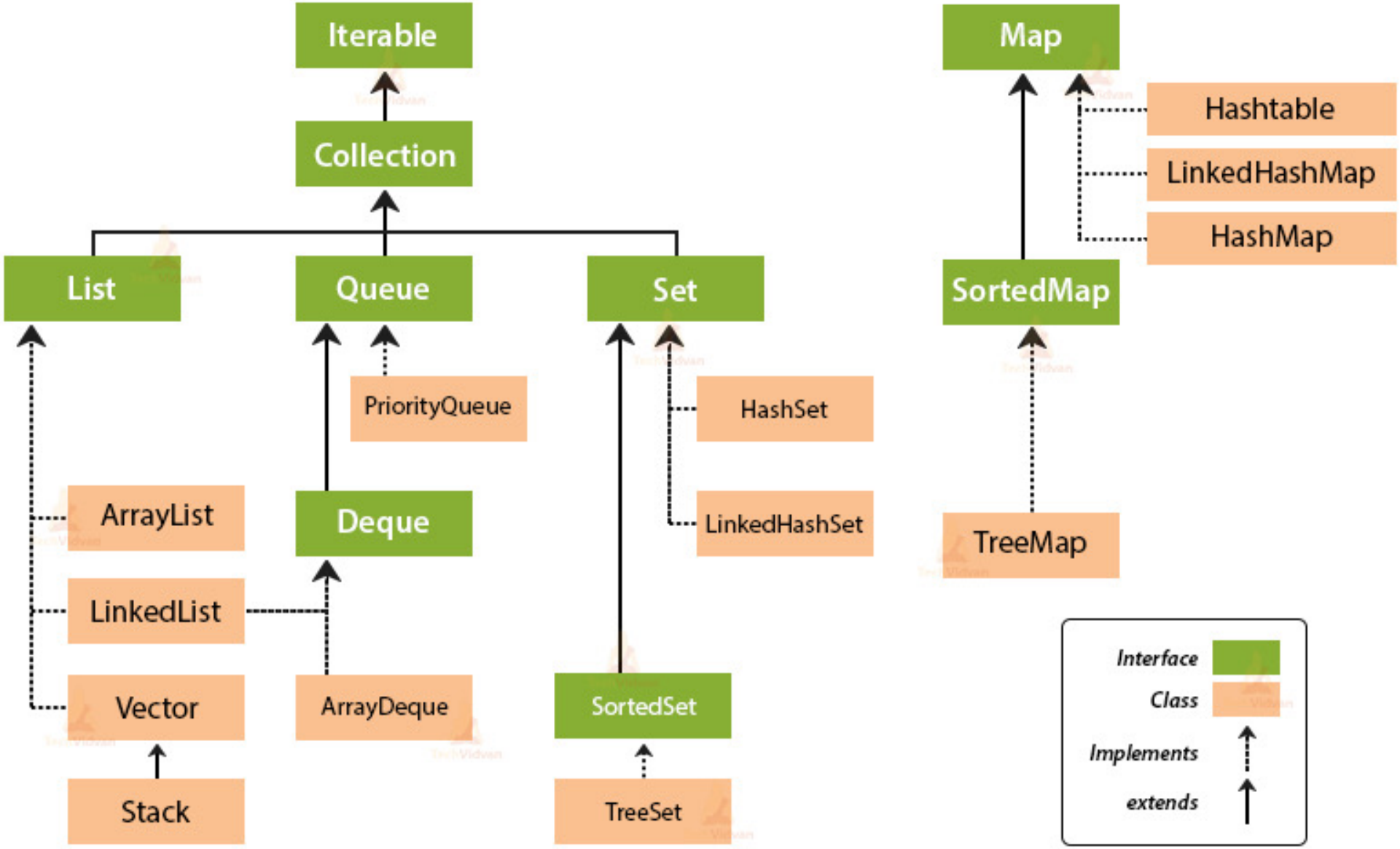
컬렉션의 종류

Java에서 **데이터의 집합**을 다루기 위한 표준화된 인터페이스와 클래스들의 집합으로, 데이터를 저장·검색·수정·삭제하는 공통 동작을 제공

유형	목적	주요 특징	대표 용도
List	순서가 있는 요소 집합 저장	- 중복 허용 - 인덱스로 요소 접근 가능 - 순서 유지	- 순서 있는 데이터 관리 - 데이터 조회·수정이 빈번한 경우 - 예: 이름 목록, 순차 처리 데이터
Set	중복 없는 요소 집합 저장	- 중복 불허	- 고유값 저장 - 중복 제거 - 예: 회원 ID, 태그 목록
Map	키-값 쌍 저장	- 키 중복 불허, 값은 중복 가능 - 키로 빠른 검색 가능	- 키 기반 데이터 매핑 - 빠른 조회 필요할 때 - 예: 사용자 정보(아이디 → 사용자 객체)
Queue	FIFO 방식의 요소 처리	- 먼저 들어온것인 먼저 나감	- 작업 대기열 - 예: 프린터 작업 큐, 이벤트 처리
Stack	LIFO 방식의 요소 처리	- 나중에 들어온 데이터가 먼저 나감	- 되돌리기(Undo) - 재귀 호출 처리 - 예: 브라우저 뒤로 가기, 수식 계산

컬렉션의 종류

Collection Type



리스트 (List)

순서가 있는 데이터를 저장하는 것을 목적으로 하며, 순차적으로 데이터 관리, 인텍스로 접근 기능을 제공

- **구현체**로는 **ArrayList, Vector(thread-safe), LinkedList** 등이 있음

```
public class VectorExample {
    public static void main(String[] args) {
        List<String> list = new ArrayList<>(); // new Vector<>() or new LinkedList<>()

        list.add("A");
        list.add("B");
        list.add("C");
        list.add("A");

        // forEach 문으로 get() 사용
        for (String item: list) {
         System.out.println(item);
        }
    }
}
```

for 문을 이용하는 방법

```
for (int i = 0; i < list.size(); i++) {
    System.out.println(list.get(i));
}
```

iterator

```
 // Iterator 사용
Iterator<String> iterator = list.iterator();
while (iterator.hasNext()) {
    System.out.println(iterator.next());
}
```

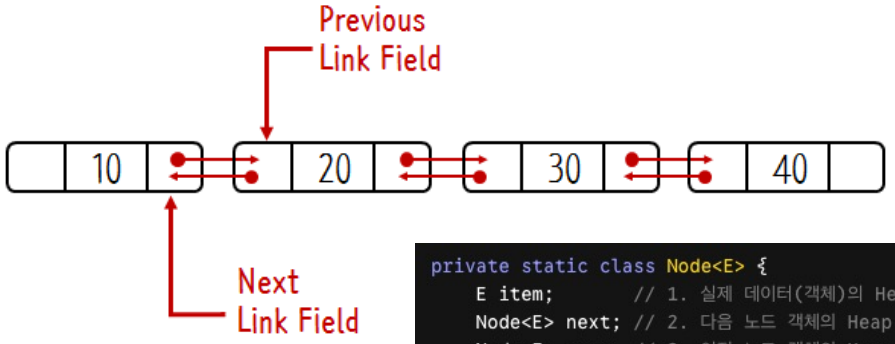
List 주요 메소드

메소드	설명	예시
add(E e)	요소 추가	list.add("A");
add(int index, E e)	특정 위치에 요소 추가	list.add(1, "B");
get(int index)	인덱스 위치의 요소 반환	list.get(0);
set(int index, E e)	인덱스 위치의 요소 변경	list.set(0, "Z");
remove(int index)	인덱스 위치 요소 제거	list.remove(1);
remove(Object o)	특정 요소 제거	list.remove("A");
size()	요소 개수 반환	list.size();
contains(Object o)	요소 존재 여부 확인	list.contains("B");
isEmpty()	비어있는지 확인	list.isEmpty();
clear()	모든 요소 삭제	list.clear();

[참고] Collection 구현체 타입

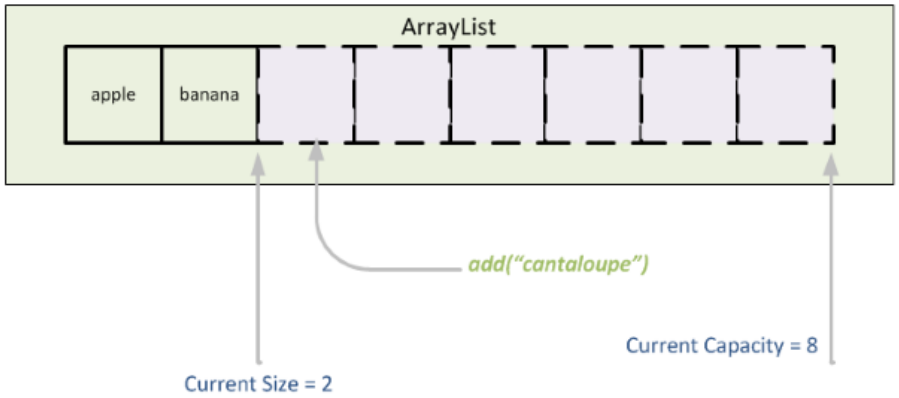
대부분의 Collection 구현체는 3가지 유형의 확장으로 만들어진다.

LinkedList 구조



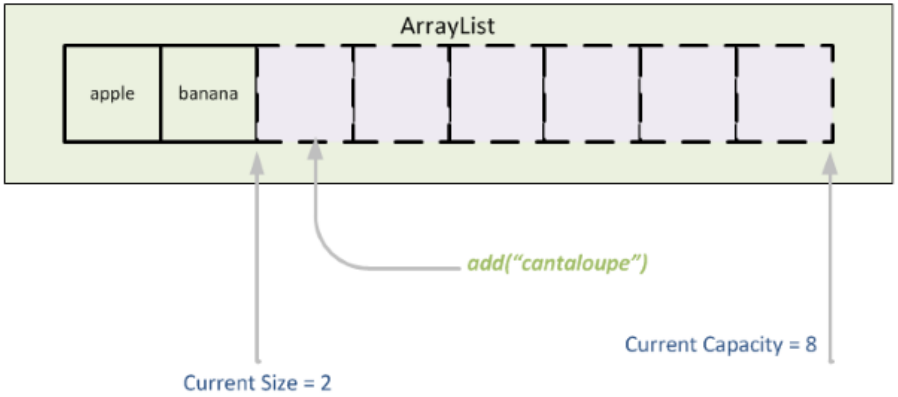

```
private static class Node<E> {
    E item;       // 1. 실제 데이터(객체)의 Heap 주소값
    Node<E> next; // 2. 다음 노드 객체의 Heap 주소값
    Node<E> prev; // 3. 이전 노드 객체의 Heap 주소값
}
```

Vector 구조 (ArrayList에 Thread-safe 지원)




```
public synchronized boolean add(E e) {
    modCount++;
    ensureCapacityHelper(elementCount + 1);
    elementData[elementCount++] = e;
    return true;
}
```

ArrayList 구조



[참고] 리스트 (List) 구현체

비교 항목	ArrayList	LinkedList	Vector
구조	동적 배열	이중 연결 리스트	동적 배열 (동기화 지원)
조회 속도	O(1) (빠름)	O(n) (느림)	O(1) (빠름)
삽입/삭제 속도	O(n) (느림)	O(1) (빠름)	O(n) (느림)
동기화	❌ (미지원)	❌ (미지원)	✅ (지원)
멀티스레드 환경	❌ (비추천)	❌ (비추천)	✅ (추천)
메모리 사용량	적음	많음 (노드 참조 필요)	적음
적합한 경우	조회가 빈번한 경우	삽입/삭제가 빈번한 경우	멀티스레드 환경에서 사용

List 활용하기

리스트의 다양한 메소드를 활용한 사례

```java
package skalajava;

import java.util.ArrayList;
import java.util.List;

public class ListExample {
    Run | Debug
    public static void main(String[] args) {
        // 1. 리스트 생성
        List<String> fruits = new ArrayList<>();

        // 2. 요소 추가: add()
        fruits.add(e:"사과");
        fruits.add(e:"바나나");
        fruits.add(e:"체리");

        // 3. 리스트의 크기 출력: size()
        System.out.println("과일의 개수: " + fruits.size()); // 출력: 과일의 개수: 3

        // 4. 특정 요소에 접근: get()
        System.out.println("첫 번째 과일: " + fruits.get(index:0)); // 출력: 첫 번째 과일: 사과

        // 5. 요소 수정: set()
        fruits.set(index:1, element:"오렌지");
        System.out.println("수정된 리스트: " + fruits); // 출력: 수정된 리스트: [사과, 오렌지, 체리]

        // 6. 요소 삭제: remove()
        fruits.remove(o:"체리");
        System.out.println("삭제된 리스트: " + fruits); // 출력: 삭제된 리스트: [사과, 오렌지]

        // 7. 리스트에 있는 모든 요소 반복: forEach()
        System.out.println(x:"리스트의 모든 과일:");
        fruits.forEach(System.out::println);

        // 8. 리스트가 비어있는지 확인: isEmpty()
        System.out.println("리스트가 비었나요? " + fruits.isEmpty()); // 출력: false

        // 9. 모든 요소 삭제: clear()
        fruits.clear();
        System.out.println("모든 과일 삭제 후 리스트: " + fruits); // 출력: 모든 과일 삭제 후 리스트: []

        // 10. 리스트에 요소가 다시 추가됨
        fruits.add(e:"망고");
        fruits.add(e:"키위");
        System.out.println("다시 추가된 리스트: " + fruits); // 출력: 다시 추가된 리스트: [망고, 키위]
    }
}
```

```
과일의 개수: 3
첫 번째 과일: 사과
수정된 리스트: [사과, 오렌지, 체리]
삭제된 리스트: [사과, 오렌지]
리스트의 모든 과일:
사과
오렌지
리스트가 비었나요? false
모든 과일 삭제 후 리스트: []
다시 추가된 리스트: [망고, 키위]
```

[실습] List

01.training-code/08.collection/Calculator.java

06.exception/Calculator.java에서

1. String[] history 로 선언되어 있던 Array 구조를 ArrayList 로 변경하고,

2. 기록을 추출해서 화면에 프린트하는 for 문을 Iterator<String>로 변경해보자

```
//*** String[] history = new String[100];
List<String> history = new ArrayList<>();


…
// 기록 저장
// if (historyCount < history.length) {
// history[historyCount] = record;
// historyCount++;
// }

history.add(record);


…

Iterator<String> iterator = history.iterator();
System.out.println("\n=== 계산 기록 ===");
while (iterator.hasNext()) {
    System.out.println(iterator.next());
}
```

[참고] Collection 사용 처

최근 Cloud Native, MSA, Stateless 구조가 대세가 되면서 Collection에 대량의 데이터를 쌓아두지 않음

ArrayList, LinkedList 등의 차이는 의미 없음

셋 (Set)

Set. 중복이 허용되지 않는 데이터를 관리

- HashSet, SortedSet, TreeSet 등이 있음

```
public class SetExample {
    public static void main(String[] args) {
        Set<String> set = new HashSet<>(); // LinkedHashSet, TreeSet도 가능

        set.add("A");
        set.add("B");
        set.add("C");
        set.add("A"); // 중복 무시

        // forEach 사용
        for (String item : set) {
            System.out.println(item);
        }

        // Iterator 사용
        Iterator<String> iterator = set.iterator();
        while (iterator.hasNext()) {
            System.out.println(iterator.next());
        }
    }
}
```

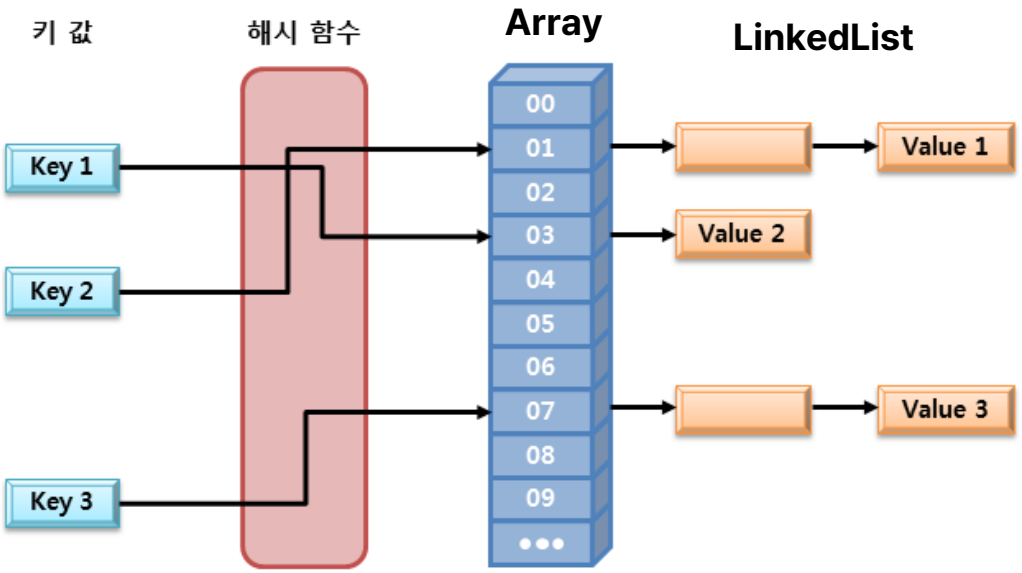
Set 주요 메소드

메소드	설명	예시
add(E e)	요소 추가 (중복 허용 안됨)	set.add("A");
remove(Object o)	요소 제거	set.remove("A");
contains(Object o)	요소 존재 여부 확인	set.contains("B");
size()	요소 개수 반환	set.size();
isEmpty()	비어있는지 확인	set.isEmpty();
clear()	모든 요소 삭제	set.clear();
iterator()	Iterator 반환	set.iterator();

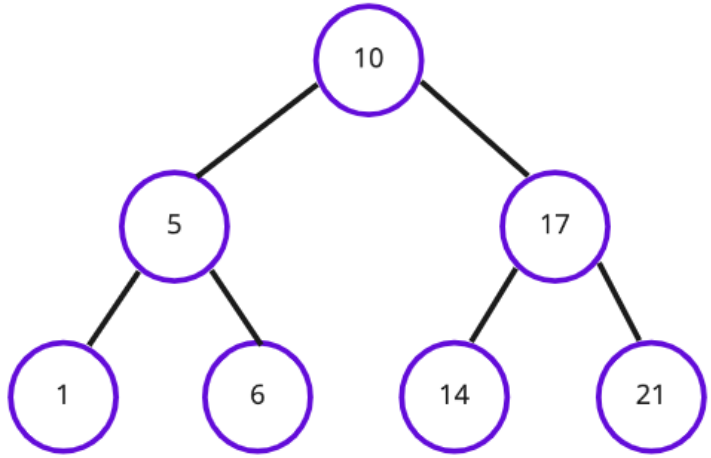
Set 구현체

구현체	목적	중복 허용	순서 유지	정렬 지원	비고
HashSet	빠른 검색/삽입/삭제가 필요하다면	❌ (미지원)	❌ (미지원)	❌ (미지원)	가장 빠른 일반 Set
LinkedHashSet	순서를 유지하면서도 빠른 성능을 원한다면?	❌ (미지원)	✅ (삽입 순서 유지)	❌ (미지원)	순서 유지 필요 시
TreeSet	정렬이 필요하다면	❌ (미지원)	✅ (정렬된 순서 유지)	✅ (오름차순/사용자 정의)	정렬된 집합 필요 시

HashSet 구조



TreeSet 구조



Map

키(Key)와 값(Value) 쌍을 저장하는 데이터 구조

- 구현체로는 HashMap, LinkedHashMap, TreeMap 등이 있음
- 주요 메소드로는 put ( ), get ( ), entrySet(), keySet(), values(), containsKey ( ) 등

```
public class MapExample {
    public static void main(String[] args) {
        // HashMap 생성 (키: String, 값: String)
        Map<String, String> map = new HashMap<>();

        // 데이터 추가
        map.put("1", "A");
        map.put("2", "B");
        map.put("3", "C");
        map.put("4", "A"); // 값 중복 가능 (Key만 유일하면 됨)

        // 1. forEach 문 (Key, Value 동시 사용)
        System.out.println("=== forEach ===");
        for (Map.Entry<String, String> entry : map.entrySet()) {
            System.out.println(entry.getKey() + " : " + entry.getValue());
        }
    }
}
```

Map

KeySet 이용해서 값 조회하기

```
        // 2. KeySet 이용 (Key로 값 조회)
        System.out.println("\n=== KeySet ===");
        for (String key : map.keySet()) {
            System.out.println(key + " : " + map.get(key));
        }
```

Iterator 를 사용

```
        // 3. Iterator 사용
        System.out.println("\n=== Iterator ===");
        Iterator<Map.Entry<String, String>> iterator = map.entrySet().iterator();
        while (iterator.hasNext()) {
            Map.Entry<String, String> entry = iterator.next();
            System.out.println(entry.getKey() + " : " + entry.getValue());
        }
```

Map 주요 메소드

메소드	설명	예시
put(K key, V value)	키-값 쌍 추가/변경	map.put("1", "A");
get(Object key)	키에 해당하는 값 반환	map.get("1");
remove(Object key)	키로 엔트리 제거	map.remove("1");
containsKey(Object key)	특정 키 존재 여부	map.containsKey("1");
containsValue(Object value)	특정 값 존재 여부	map.containsValue("A");
size()	엔트리 수 반환	map.size();
isEmpty()	비어있는지 확인	map.isEmpty();
clear()	모든 엔트리 삭제	map.clear();
keySet()	키 집합 반환	map.keySet();
values()	값 컬렉션 반환	map.values();
entrySet()	키-값 엔트리 집합 반환	map.entrySet();

Map 구현체

구현체	내부 구조	순서 유지	목적	비고
HashMap	해시 테이블 (체이닝 + JDK8부터 버킷 트리화)	×	빠른 검색/삽입/삭제	메모리 효율 좋고 전반적으로 가장 빠름
LinkedHashMap	HashMap + 이중 연결 리스트	○ (기본: 삽입 순서, 옵션: 접근 순서)	순서를 보장하면서 빠른 성능 지원	순서 유지 비용 약간 증가
TreeMap	Red-Black Tree(균형 이진 탐색 트리)	○ (정렬된 순서)	정려리 필요하면	정렬/범위 질의에 유리

- 빠른 검색/삽입/삭제가 필요하다면? → HashMap
- 순서를 유지하면서도 빠른 성능을 원한다면? → LinkedHashMap
- 정렬이 필요하다면? → TreeMap

[실습] Map

01.training-code/08.collection/CalculatorMap.java

앞서 List를 적용했던 Calculator.java에서

1. List --> Map, ArrayList → HashMap으로 선언하고 코드를 변경하자

2. Key는 계산 이력 스트링에 대한 hashcode (32bit 정수)를 사용

2. 기록을 추출해서 화면에 프린트는 Iterator로 적용해보자

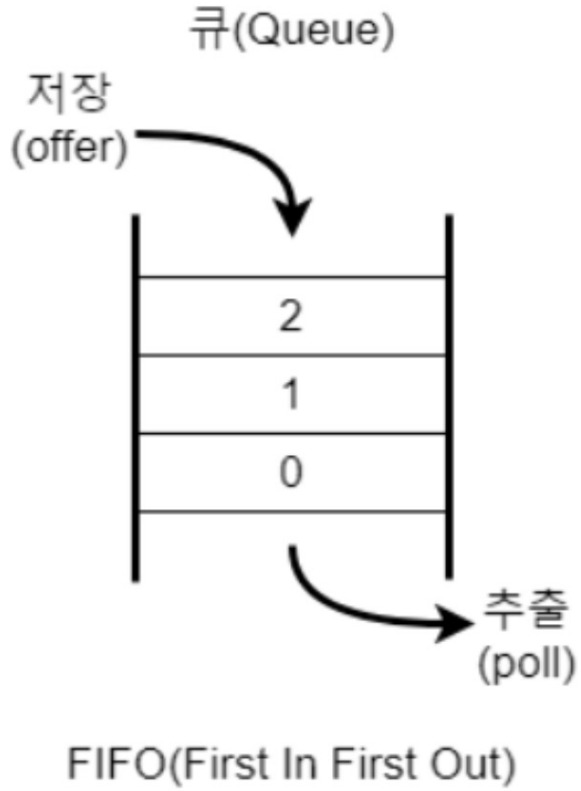
```
Map<Integer, String> historyMap = new HashMap<>();
…

historyMap.put(record.hashCode(), record);

…

// Map의 entrySet()을 이용하여 기록 출력
for (Map.Entry<Integer, String> entry : historyMap.entrySet()) {
    System.out.println(entry.getKey() + ": " + entry.getValue());
}
```

큐 (Queue)

데이터의 in-out 을 FIFO 방식으로 관리 (First In First Out)

- 구현체는 ArrayDeque, LinkedList (검색 기능), PriorityQueue

```
public class QueueExample {
    public static void main(String[] args) {
        Queue<String> queue = new LinkedList<>();
        queue.offer("A");
        queue.offer("B");
        queue.offer("C");
        queue.offer("A");

        // forEach 사용
        for (String item : queue) {
            System.out.println(item);
        }

        // poll() 사용 (FIFO 꺼내기)
        while (!queue.isEmpty()) {
            System.out.println(queue.poll());
        }

       // Iterator 사용
       Iterator<String> it = queue.iterator();
       while (it.hasNext()) {
          System.out.println(it.next());
       }
    }
}
```



Queue 주요 메소드

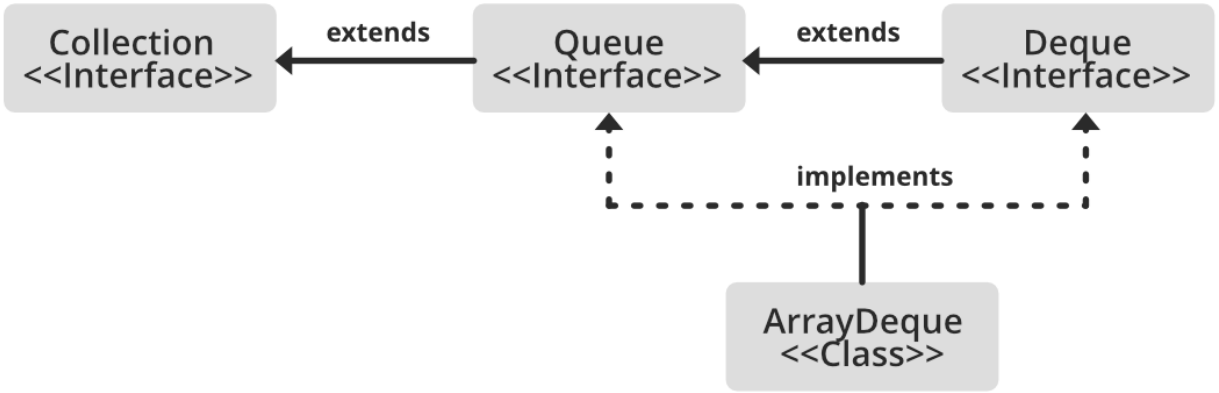
메소드	설명	예시
offer(E e)	요소 추가 (실패 시 false 반환)	queue.offer("A");
add(E e)	요소 추가 (실패 시 IllegalStateException)	queue.add("B");
poll()	첫 요소 제거 후 반환 (없으면 null)	queue.poll();
remove()	첫 요소 제거 후 반환 (없으면 예외)	queue.remove();
peek()	첫 요소 조회 (없으면 null)	queue.peek();
element()	첫 요소 조회 (없으면 예외)	queue.element();
size()	요소 개수 반환	queue.size();
isEmpty()	비어있는지 확인	queue.isEmpty();
clear()	모든 요소 삭제	queue.clear();

큐 (Queue) 구현체

데이터의 in-out 을 FIFO 방식으로 관리 (First In First Out)

- ArrayDeque(속도가 가장 빠름), LinkedList (검색 기능), PriorityQueue (오름차순또는 내림차순 순서 변경) 등이 있음

구현체	인터페이스	내부 구조	스레드 안전	주요 특징 / 사용처
LinkedList	Queue, Deque	이중 연결 리스트	❌	가장 단순한 큐, 학습/간단한 큐
ArrayDeque	Queue, Deque	원형 배열	❌	**가장 빠른 일반 큐/스택**, LinkedList 대체
PriorityQueue	Queue	힙(Heap)	❌	우선순위 큐 (정렬 기준 처리)
ConcurrentLinkedQueue	Queue	Lock-free 링크 구조	⭕	고성능 비차단 큐 (멀티스레드)
ConcurrentLinkedDeque	Deque	Lock-free	⭕	멀티스레드 Deque



LinkedList는 중간 삽입/삭제 기능이 빠르지만, ArrayDeque는 접근 속도 및 탐색이 빠르다.

Deque

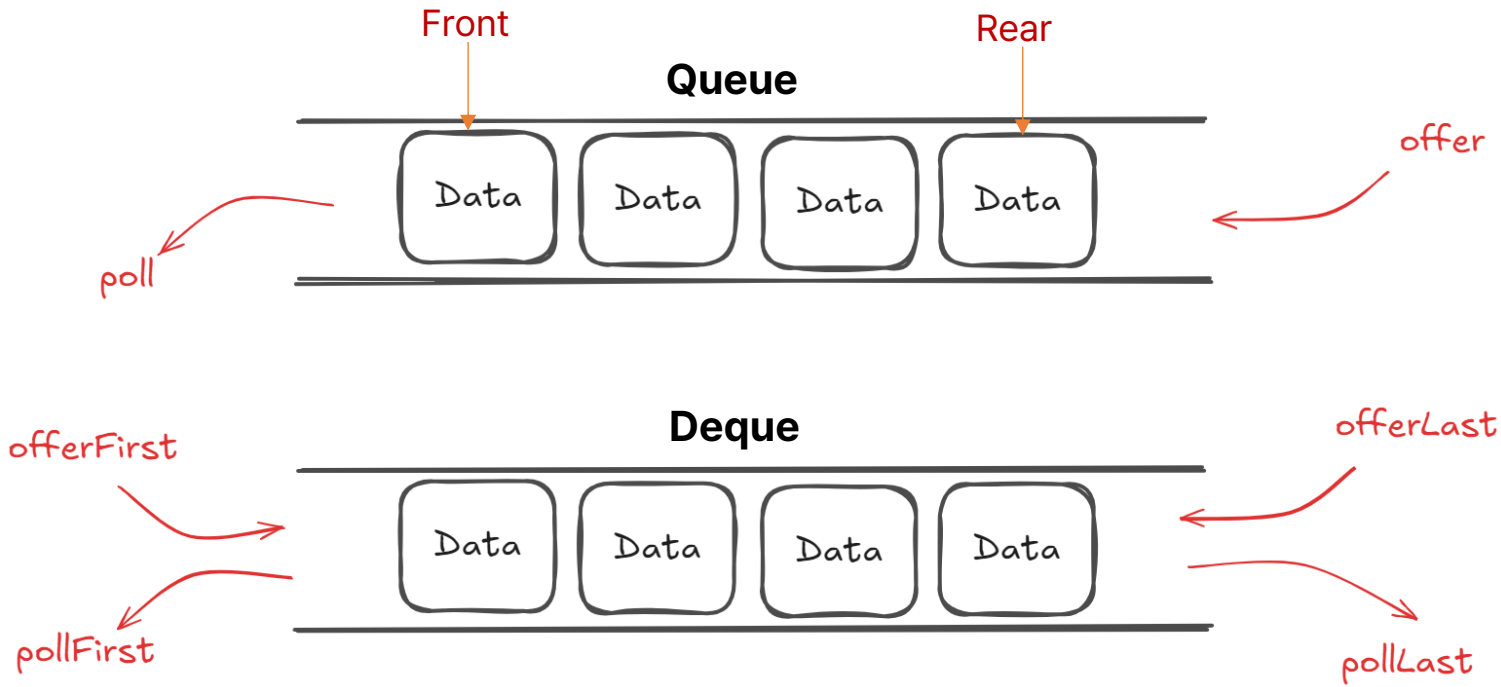
양쪽에서 넣고, 양쪽에서 빼는 자료 구조 (Double-Ended Queue)

Queue 사용 예 (FIFO)

```
Queue<Integer> q = new ArrayDeque<>();
q.offer(1);
q.offer(2);
q.offer(3);

System.out.println(q.poll()); // 1
System.out.println(q.poll()); // 2
```

Deque 사용 예 (양방향)

```
Deque<Integer> d = new ArrayDeque<>();

d.offerLast(1);   // 뒤에 삽입
d.offerLast(2);
d.offerFirst(0);  // 앞에 삽입

System.out.println(d.pollFirst()); // 0
System.out.println(d.pollLast());  // 2
```

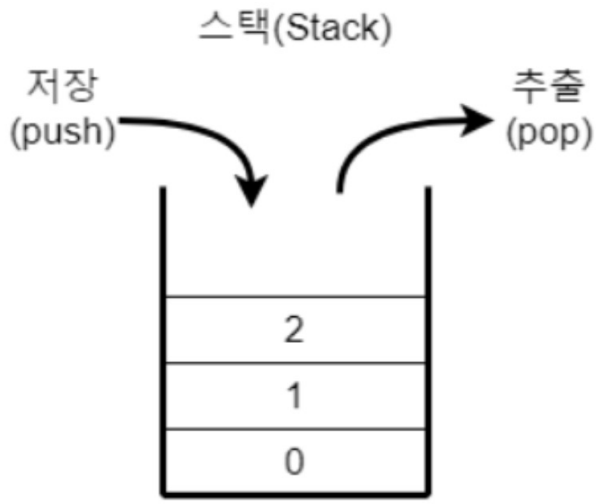


스택 (Stack)

데이터의 in-out 을 LIFO 방식으로 관리 (Last In First Out)

ArrayDeque(속도가 가장 빠름), Stack(Thread-safe: 성능 느림) 등이 있음

```
public class StackExample {
    public static void main(String[] args) {
        Stack<String> stack = new Stack<>();

        stack.push("A");
        stack.push("B");
        stack.push("C");
        stack.push("A");

        // forEach 사용
        for (String item : stack) {
            System.out.println(item);
        }

        // pop() 사용 (LIFO 꺼내기)
        while (!stack.isEmpty()) {
            System.out.println(stack.pop());
        }

       // Iterator 사용
       Iterator<String> it = stack.iterator();
       while (it.hasNext()) {
          System.out.println(it.next());
       }
    }
}
```



LIFO(Last In First Out)

Stack 주요 메소드

메소드	설명	예시
push(E e)	스택에 요소 삽입	stack.push("A");
pop()	맨 위 요소 제거 후 반환	stack.pop();
peek()	맨 위 요소 조회	stack.peek();
search(Object o)	요소 위치 반환 (1부터 시작)	stack.search("A");
empty()	비어있는지 확인	stack.empty();
size()	요소 개수 반환	stack.size();
clear()	모든 요소 삭제	stack.clear();

Collection<E>

Java Collection 에서 제공하는 최상위 인터페이스이며, List, Set, Queue 등 자료 구조들의 공통 메소스 집합체

```
public interface Collection<E> extends Iterable<ED> { ...}
public interface List<E> extends Collection<E> { ... }
```

공통 메소드 표준화

List, Set, Queue는 내부 동작 방식은 달라도,

요소 추가(add), 삭제(remove), 포함 여부 확인(contains) 같은 공통 기능은 모두 Collection에 정의

다형성 제공

코드 작성 시 특정 구현체(ArrayList, HashSet, ...)가 아닌,

Collection<E> 타입으로 변수를 선언하면 자료구조를 교체하기 쉬워집니다.

Collection 주요 메소드

메소드	설명	메소드
add(E e)	요소 추가	add(E e)
addAll(Collection<? extends E> c)	다른 컬렉션의 모든 요소 추가	addAll(Collection<? extends E> c)
remove(Object o)	요소 제거	remove(Object o)
removeAll(Collection<?> c)	특정 컬렉션의 모든 요소 제거	removeAll(Collection<?> c)
clear()	모든 요소 삭제	clear()
contains(Object o)	포함 여부 확인	contains(Object o)
isEmpty()	비어있는지 여부	isEmpty()
size()	요소 개수	size()
iterator()	순회용 Iterator 반환	iterator()

Collection<E> 사용 방법

1. 자료 구조의 종류에 상관없이 공통된 처리만 할때

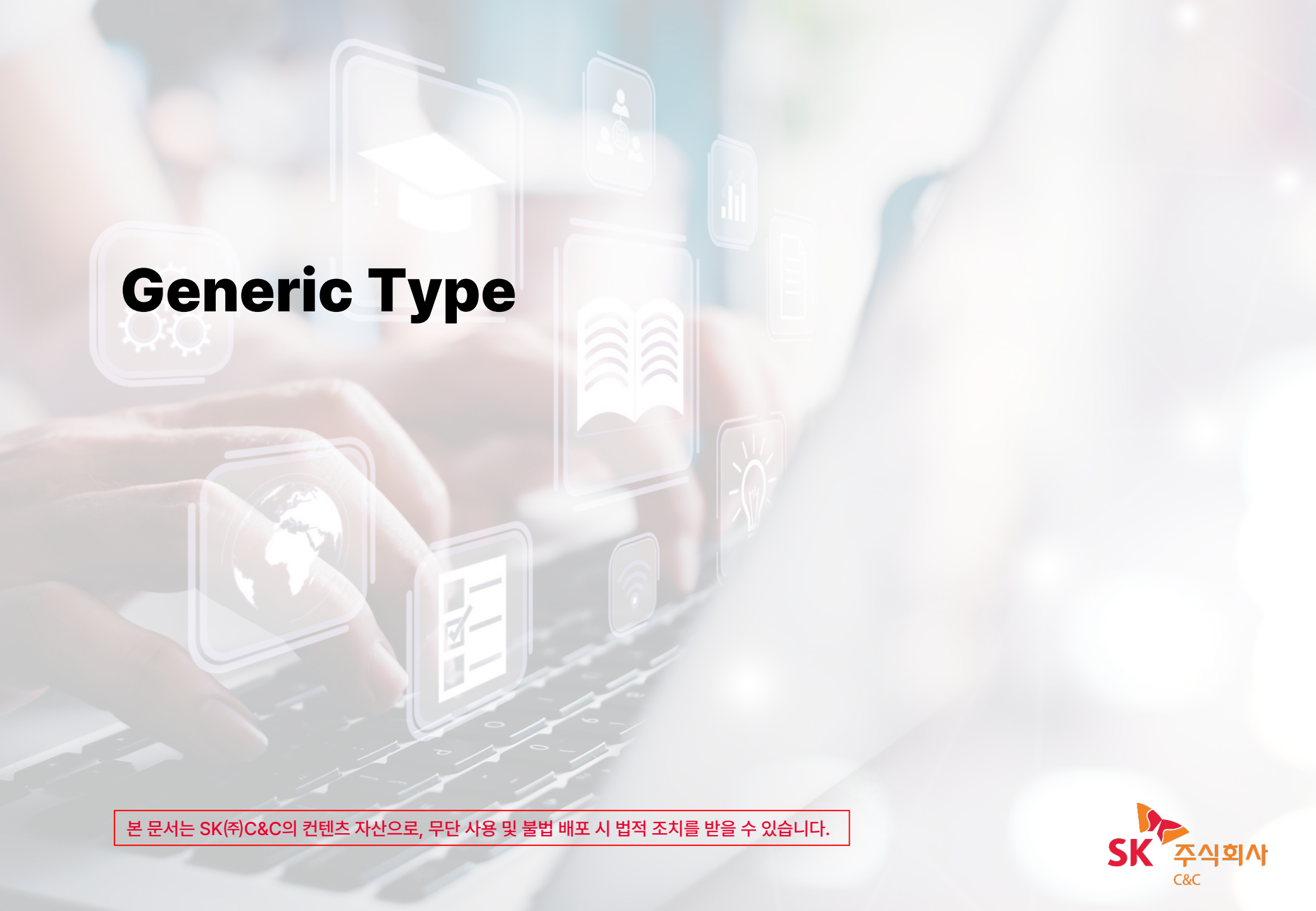
```
public void printAll(Collection<String> collection) {
    for (String item : collection) {
        System.out.println(item);
    }
}
```

2. 구현체를 유연하게 교체할 필요가 있을때

```
Collection<String> names = new ArrayList<>(); // 처음에는 ArrayList
names.add("Alice");
names.add("Bob");

names = new HashSet<>(names); // 나중에 HashSet으로 변경
```

3. 공통 메소드만 필요할때

```
Collection<Integer> numbers = new LinkedList<>();
numbers.add(1);
numbers.add(2);
System.out.println(numbers.contains(2)); // true
numbers.remove(1);
```



Generic Type

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

제네릭 (Generic) 이란 ?

타입별로 중복 코드를 작성하는 것을 막아서 **코드 재사용성**을 높이기 위한 방식

클래스나 메서드에서 사용할 데이터 타입을 컴파일 시점에 결정

```
class Box<T> {
    private T item;

    public void set(T item) {
        this. item = item;
    }

    public T getItem() {
        return item;
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Box<String> stringBox = new Box<>();
        stringBox.set("Hello");
        System.out.println(stringBox.get()); // Hello

        Box<Integer> integerBox = new Box<>();
        integerBox.set(100);
        System.out.println(integerBox.get()); //100
    }
}
```

- **T**는 타입 매개변수(Type Parameter)
- 사용할 때는 구체적인 타입으로 지정함

컴파일 후

```
Box stringBox = new Box();
stringBox.set("Hello");
System.out.println((String) stringBox.get());
```

관용적인 제네릭 타입 매개 변수

- 제네릭 타입 매개변수의 이름은 꼭 T일 필요는 없으나 관례적으로 T를 많이 사용
- 가독성과 의미 전달을 위해 관용적인 알파벳 약어를 사용하는 것을 권고

타입 매개변수	의미	사용 예시
T	Type (일반적인 타입)	class Box<T>
E	Element (컬렉션 요소용)	List<E>
K	Key (Map의 키)	Map<K, V>
V	Value (Map의 값)	Map<K, V>
R	Return Type (반환 타입)	Converter <T, R>
?	와일드 카드 (불특정 타입)	List<?>

```
class Pair<K, V> {
    private K key;
    private V value;

    public Pair(K key, V value) {
        this.key = key;
        this.value = value;
    }

    public K getKey() { return key; }
    public V getValue() { return value; }
}
```

```
Pair<String, Integer> stockPair = new Pair<>("SKALA-AI", 10000);
System.out.println("종목: " + stockPair.getKey() + ", 가격: " +
stockPair.getValue());
```

제네릭 타입의 제한

- 상속 관계가 있는 클래스들 사이에서 타입 안정성을 유지하면서 코드의 범용성을 높이는 데 필수적인 도구

❖ 상한 제한 (Upper Bound – extends)

```
class NumberBox<T extends Number> {
    private T value;

    public void printDouble() {
        System.out.println(value.doubleValue());
    }

    public void setValue(T value) {
        this.value = value;
    }
}
```

- Number Type 또는 상속받은 하위 타입만 허용
- **T extends Number**는 T가 Number 또는 그 하위 클래스 (Integer, Double, Float 등)만 허용

```
NumberBox<Integer> intBox = new NumberBox<>();
intBox.setValue(100);
intBox.printDouble();  // 출력: 100.0
```

❖ 하한 제한 (Lower Bound – super)

```
public static void addIntegers(List<? super Integer> list) {
    list.add(1);
    list.add(2);
}
```

- Integer의 상위 타입만 허용
- 제네릭 클래스 정의에는 사용하지 않고, **메서드 매개변수나 와일드카드에서만** 사용
- List<Number>나 List<Object>에 안전하게 Integer를 추가할 수 있음
- 반대로 list.get()은 Object로만 얻을 수 있음 (하한이므로 타입 정보가 모호)

```
Number
 ├── Integer
 └── Double

<? extends Number> → [Number, Integer, Double]
<? super Integer>  → [Integer, Number, Object]
```

제네릭 와일드카드 : <? extends T> (1/2)

T 타입 또는 T의 하위 (자식) 타입이라면 무엇이든 사용 가능하다는 의미

- T를 최상위 부모 (상하선)으로 제한, 자식 클래스만 매개변수로 받는 방식
- 상한 와일드 카드 (Upper Bounded Wildcard)

```
    [ Fruit ]  <-- 상한선 (T)
     /     \
[ Apple ] [ Banana ]
```

List<T>를 사용하는 경우 : 정상 처리

```
public void printFruit(List<Fruit> fruits)
{ ... }

// 정상 동작
List<Fruit> apples = new ArrayList<>();
printFruit(apples);
```

List<T>를 사용하는 경우 : 에러 발생

```
// 실행 시 에러 발생!
List<Apple> apples = new ArrayList<>();
//에러! List<Fruit> 자리에 List<Apple> 불가
printFruit(apples);
```

List<? extends T> 사용

```
public void printFruit(List<? extends Fruit> fruits)
{
    for (Fruit f : fruits) {
        System.out.println(f.getName());
    }
}

List<Apple> apples = new ArrayList<>();
apples.add(new Apple());
apples.add(new Apple());

// 에러 없이 컴파일 및 실행 성공!
printFruit(apples);

// 바나나 리스트도 똑같은 메서드로 처리
List<Banana> bananas = new ArrayList<>();
bananas.add(new Banana());
printFruit(bananas);
```

제네릭 와일드카드 : <? extends T> (2/2)

이미 요소가 채워진 리스트를 받아서 읽기 전용으로 사용하기 위한 타입

- ? extends T: T의 하위 타입 중 하나

```
List<? extends Number> nums = List.of(1, 2, 3);
```

nums 안에 들어있는 실제 타입은 Integer, Double, Long ... 뭐가 되었든 반드시 Number의 자식

❖ get 할 때

```
Number n = nums.get(0);  // OK
```

어떤 구체 타입인지는 몰라도, 최소한 Number 라는 건 확실 → 그래서 리턴 타입을 Number로 받는 것은 100% 안전

❖ add 할 때

참고, List는 실제로 보관하는 정확한 타입과 일치하는 타입만 add 할 수 있음

```
nums.add (Integer.valueOf(10));  // 컴파일 오류
```

컴파일러 입장에선 nums 가 정확히 무슨 리스트인지 알 수 없음

실제 타입이 List<Integer> , List<Double> , List<BigDecimal>일 수 있으므로 컴파일 오류 발생

❖ 예시

```
List<Integer> productPrices
    = Arrays.asList(100, 200, 300);
List<Double> discountRates
    = Arrays.asList(0.1, 0.2, 0.3);
```

```
void printNumbers(List<? extends Number> nums) {
    for (Number n : nums) {
        System.out.println(n);  // 읽기는 OK
    }

    // nums.add(10); // ❌ 불가
}
```

제네릭 와일드카드 : <? super T>

이미 T 타입 이상의 상위 타입을 받도록 하여 리스트에 값을 넣기(write) 위한 타입

- ? super T: T의 상위 타입 중 하나

```
List<? super Integer> list;
```

nums 안에 들어있는 실제 타입은 Integer, Double, Long ... 뭐가 되었든 반드시 Number의 자식

❖ add 할 때

```
list.add(Integer.valueOf(10));    // OK
```

list가 실제로 어떤 리스트인지 몰라도, 최소 **Integer 를 담을 수 있는 리스트**라는 건 확실

❖ get 할 때

```
Integer obj = list.get(0);  // 컴파일 오류
```

```
Object obj = list.get(0);  // OK
```

실제 타입이 List<Integer> 일 수도 있지만, List<Number>, List<Object> 일 수도 있기 때문에
정확한 요소 타입을 컴파일러가 알 수 없음

❖ 예시

```
List<Object> objects = new ArrayList<>();
List<? super Integer> list = objects;

list.add(100);  // OK
list.add(200);  // OK

Object o = list.get(0);  // 읽기는 Object
```

[참고] static method에 제네릭 타입

static method에 대한 제네릭 타입 지정 방식

```
import java.util.List;

class Box<T> {
    private T value;

    public Box(T value) {
        this.value = value;
    }

    // 인스턴스 메서드: 클래스 제네릭 T를 활용
    public void showValue() {
        System.out.println("Box contains: " + value);
    }

    // static 메서드: 메서드 자체 제네릭 U를 활용
    public static <U extends Number> void process(List<U> list) {
        for (U item : list) {
            System.out.println("Processing: " + item);
        }
    }
}
```

제네릭 활용 사례

```java
public class GenericMethodExample {
    // 제네릭 메서드 정의
    public static <T> void printArray(T[] array) {
        for (T element : array) {
            System.out.println(element);
        }
    }

    Run | Debug
    public static void main(String[] args) {
        Integer[] intArray = {1, 2, 3, 4, 5};
        String[] stringArray = {"Hello", "World"};

        System.out.println(x:"Integer Array:");
        printArray(intArray); // Integer 배열 출력

        System.out.println(x:"\nString Array:");
        printArray(stringArray); // String 배열 출력
    }
}
```

[실습] Generic Example

09.generic/GenericsExample.java

아래 동작을 수행하는 Box<T> Generic Type Class를 만들어서 코드를 완성하세요

```
public class GenericsExample {
    public static void main(String[] args) {
        // Integer 타입의 Box
        Box<Integer> integerBox = new Box<>();
        integerBox.setItem(10);
        System.out.println("Integer Value: " +
            integerBox.getItem());

        // String 타입의 Box
        Box<String> stringBox = new Box<>("Hello, Generics!");
        System.out.println("String Value: " +
            stringBox.getItem());
    }
}
```

```
public class Box<T> {
    private T item;

    public Box() {
    }

    public Box(T item) {
        this.item = item;
    }

    public void setItem(T item) {
        this.item = item;
    }

    public T getItem() {
        return item;
    }
}
```

[실습] Generic WildCard (1/2)

09.generic/BoundedGenericsExample.java

BoundedGenericsExample.java에서 사용하는 BoundedBox.java를 검토 및 실행

- sumBox 의 input parameter 는 읽기 전용 , list.add(10)를 추가해서 쓰기 시 에러가 나는지 확인하기

읽기 전용 : <? extend T>

```
// ------------------------------------------------------
// Upper Bound Wildcard 메서드:
// Number 하위 타입 리스트의 합계를 반환
// ------------------------------------------------------
public static double sumBox(List<? extends Number> list) {
    double sum = 0;
    for (Number n : list) {
        sum += n.doubleValue();
    }

    // 컴파일 에러: upper bound wildcard 리스트에는 추가 불가
    // list.add(10);
    return sum;
}
```

```
// ---- Upper Bound Wildcard: sumBox(List<? extends Number>) ----
List<Integer> intList = new ArrayList<>(List.of(1, 2, 3, 4, 5));
List<Double>  dblList = new ArrayList<>(List.of(1.1, 2.2, 3.3));

System.out.println("Integer 리스트 합계: " + sumBox(intList));
System.out.println("Double  리스트 합계: " + sumBox(dblList));
```

[실습] Generic WildCard (2/2)

09.generic/BoundedGenericsExample.java

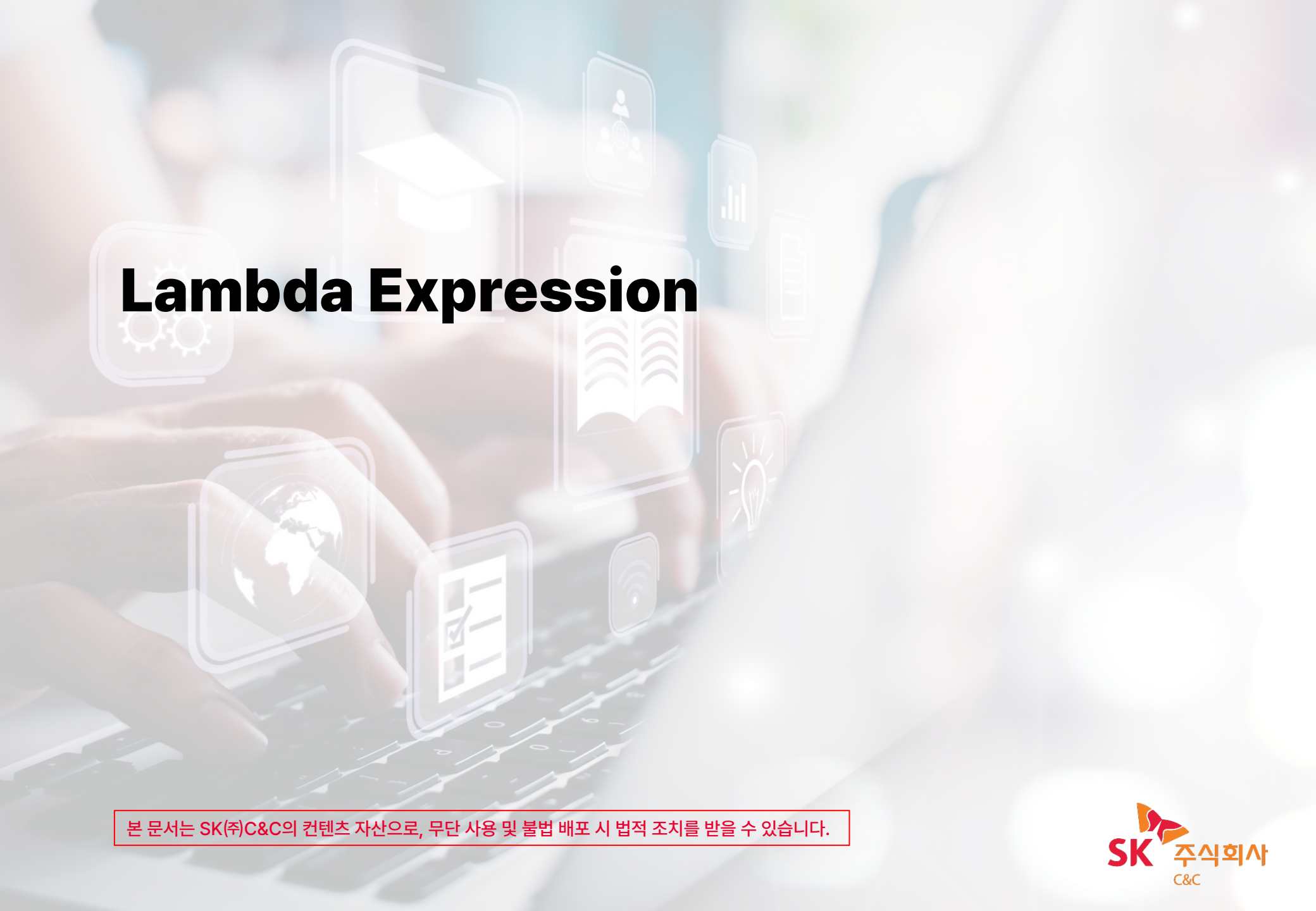
BoundedGenericsExample.java에서 사용하는 BoundedBox.java를 검토 및 실행

- addBox 의 input parameter 는 쓰기 전용 , list.get()을 추가해서 읽기 에러가 발생하는지 확인하기

쓰기 전용 : <? super T>

```
// ----------------------------------------------------
// Lower Bound Wildcard 메서드: <? super Integer>
//   Integer 및 그 상위 타입(Number, Object)만 허용
//   → 값을 하나씩 받아서 리스트에 추가
// ----------------------------------------------------
public static void addBox(List<? super Integer> list, int value) {
        list.add(value);
        // 컴파일 에러: lower bound wildcard는 Object로만 꺼낼 수 있음
        // Integer first = list.get(0);
}
```

```
// ---- Lower Bound Wildcard: addBox(List<? super Integer>, int) ----
List<Number> numberList = new ArrayList<>();  // Number는 Integer의 상위 타입 → 허용

// addBox를 값 하나씩 여러 번 호출
addBox(numberList, 10);
addBox(numberList, 20);
addBox(numberList, 30);
```



Lambda Expression

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

함수형 프로그램(Functional Programming)

Java의 함수형 프로그래밍(Functional Programming)은 함수를 일급 객체(First-Class Citizen)로 취급하여, 불변성(immutability)과 순수 함수(pure function)를 기반으로 프로그램을 작성하는 방식

함수를 일급 객체(First-Class Citizen)로 사용

- 함수를 변수에 저장, 전달, 반환할 수 있음
- 함수 자체를 데이터처럼 다루는 방식

불변성(Immutability)

- 함수 내부에서 외부 변수를 변경하지 않는 구조
- 병렬 처리에서 안정성과 예측 가능성이 높아짐

순수 함수(Pure Function)

- 같은 입력 → 같은 출력
- 외부 상태를 변경하지 않음(Side Effect 없음)

선언형 스타일(Declarative Style)

- **어떻게**가 아니라 **무엇을** 할지에 집중
- 시스템이 실행 흐름을 알아서 처리
- Stream, Optional 등이 대표적인 예

일급 함수

함수를 값처럼 취급할 수 있는 언어 특성

1. 함수를 변수에 할당

```
def say_hello():
    print("Hello")

# 함수를 변수에 할당
say_hello_var = say_hello

# 변수를 통해 함수 실행
say_hello_var()
```

2. 함수를 인자로 전달

```
def execute(fn):
    fn()  # 전달된 함수를 실행

# 람다(익명 함수)를 인자로 전달하며 execute 실행
execute(lambda: print("Executed"))
```

3. 함수를 반환 (return)

```
def make_adder(x):
    # 내부 함수 정의 후 반환
    def adder(y):
        return x + y
    return adder

add5 = make_adder(5)
print(add5(3))  # 출력: 8
```

4. 객체의 속성(properties) 로 사용

```
# 딕셔너리의 key-value 쌍으로 함수를 정의
calculator = {
    "add": lambda a, b: a + b
}

# 딕셔너리의 Key로 함수에 접근하여 실행
print(calculator["add"](2, 3))  # 출력: 5
```

순수 함수 (Pure Function)

동일한 입력에는 항상 동일한 출력을 반환하며, 함수 외부 변수의 상태를 변경하지 않는 함수

비순수 함수

```
# 외부 상태 (Global 변수)
count = 0

def increase_and_get():
    global count
    count += 1  # 외부 상태를 변경 (Side Effect)
    return count

# 검증 및 특징 확인
print(increase_and_get())  # 출력: 1
print(increase_and_get())  # 출력: 2 (다른 출력)
```

순수 함수

```
def square(x):
    return x * x  # 같은 입력, 같은 출력

# 검증 및 특징 확인
print(square(4))  # 출력: 16
print(square(4))  # 출력: 16
```

순수 함수 특징

순수 함수를 유지해야 하는 이유

장점	설명
예측 가능성	같은 입력 → 같은 출력, 디버깅이 쉬움
테스트 용이	외부 의존 없이 독립적으로 테스트 가능
병렬 처리 안전	외부 상태를 건드리지 않으니 race condition 없음
캐싱 가능	결과를 재활용(Memoization) 가능
리팩토링 안정성	의존성 적어서 코드 변경 영향 최소화

익명 클래스 (Anonymous Class)

클래스 기반 언어인 Java 환경에서 함수형 프로그램을 지원하기 위한 방식

익명 클래스(Anonymous Class)란?

- 이름이 없는 클래스
- 한 번만 사용할 목적으로 정의
- 클래스를 선언하면서 동시에 인스턴스를 생성
- 주로 인터페이스나 추상 클래스를 구현할 때 사용

문제점

- 너무 장황함
- 가독성 감소
- 함수 전달을 위한 최소 표현이 아님

Single Abstract Method Interface

```
@FunctionalInterface
public interface Runnable {
   public void run();
}
```

```
public class BeforeLambdaExample {
   public static void main(String[] args) {

      // 메서드 호출 시 익명 클래스 전달
      executeTask(new Runnable() {
         @Override
         public void run() {
            System.out.println("작업 실행 중...");
         }
      });
   }

   static void executeTask(Runnable task) {
      task.run();
   }
}
```

람다 표현식(Lamda Expression)

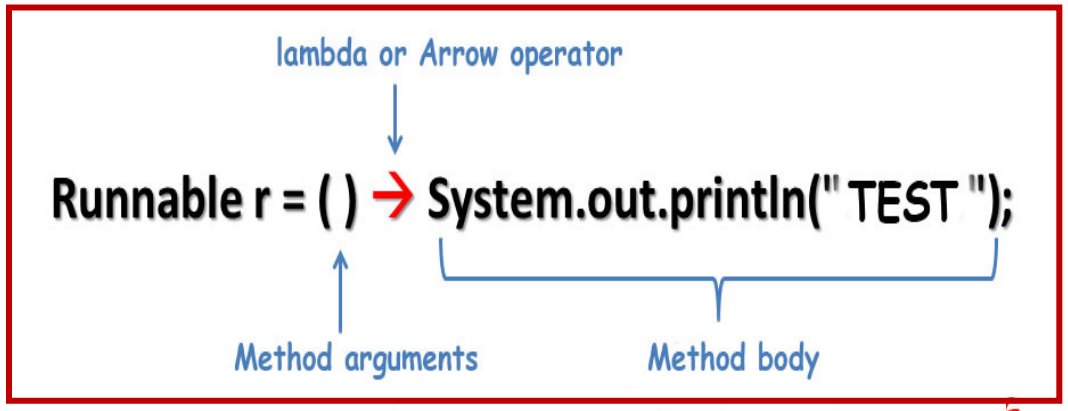
익명 클래스를 간결하게 표현하기 위한 문법, 함수형 인터페이스를 인스턴스로 생성하는 가장 짧은 방식

```
@FunctionalInterface
public interface Runnable {
    void run();
}
```

익명 클래스

```
Runnable r = new Runnable() {
    @Override
    public void run() {
        System.out.println(" TEST ");
    }
};
```

Lambda Expression




```
r.run()
```

람다 표현식(Lamda Expression)

익명 클래스를 간결하게 표현하기 위한 문법, 함수형 인터페이스를 인스턴스로 생성하는 가장 짧은 방식

배경

- Java 8(2014) 이전: Java는 순수 객체지향 언어로, 메서드도 반드시 클래스의 일부여야 함.
- Java 8: 람다, Stream API, java.util.function 패키지 도입
  → 코드를 값처럼 전달 가능
  → 선언적·함수형 스타일 코딩 가능
- 간결성, 병렬 처리 용이성, 컬렉션 처리의 함수형 스타일 지원

목적	설명	예시
코드 간결화	익명 클래스 작성의 반복 코드 제거	list.forEach(item -> System.out.println(item));
함수형 프로그래밍 지원	함수를 값처럼 전달	stream.filter(x -> x > 10)
병렬/선언형 처리	Stream API와 결합하여 병렬 연산 지원	list.parallelStream().map(...).reduce(...)

람다 표현식(Lamda Expression) 방식

익명 클래스

```
// Runnable 인터페이스를 구현하는 클래스
class MyRunnable implements Runnable {
    @Override
    public void run() {
        System.out.println("별도의 스레드에서 실행");
    }
}

public class Main {
    public static void main(String[] args) {
        //1. 일반적인 클래스 생성
        Runnable myRunnable = new MyRunnable();
        Thread thread1 = new Thread(myRunnable);
        thread1.start();

    }
}
```

람다 표현식

```
public class Main {
  public static void main(String[] args) {
    //2. 람다 표현식 사용 (요즘 더 일반적임)
    Runnable lambdaRunnable = () ->
System.out.println(”별도 스레드에서 실행");

    Thread thread2 = new Thread(lambdaRunnable);
    thread2.start();
  }
}
```

[참고] 익명 클래스 vs Lambda 비교

익명 함수를 간결하게 표현하기 위한 문법, 함수형 인터페이스를 인스턴스로 생성하는 가장 짧은 방식

익명 클래스

```
Runnable r = new Runnable() {
    @Override
    public void run() {
        System.out.println("Hello");
    }
};
```

Lambda

```
Runnable r = () -> System.out.println("Hello");
```

항목	익명 클래스	람다식
목적	객체(클래스) 생성	함수 전달(행위 전달)
코드 길이	길고 반복적	매우 간결
의미	객체지향적 해결	함수형 프로그래밍 해결
실행 구조	내부적으로 클래스 생성	함수형 인터페이스 구현체 생성
사용성	가독성 낮음	가독성·유지보수 높음

함수형 인터페이스

- 단 하나의 **추상 메서드(abstract method)**만을 가지는 인터페이스
- 람다 표현식은 바로 이 **함수형 인터페이스의 익명 구현 객체**로 생성
- **@FunctionalInterface** : 함수형 인터페이스임을 명시적으로 나타내는 어노테이션

```
@FunctionalInterface
interface Calculator {
    int operate(int a, int b);
}
```

```
public class LambdaExample {
    public static void main(String[] args) {
        Calculator add = (a, b) -> a + b;
        Calculator multiply = (a, b) -> a * b;

        System.out.println("덧셈: " + add.operate(10, 5));
        System.out.println("곱셈: " + multiply.operate(10, 5));
    }
}
```

표준 함수형 인터페이스

자주 쓰이는 입력과 출력의 형태를 미리 정의한 함수형 인터페이스 (java.util.function)

인터페이스	추상 메서드	설명	예제
Runnable	void run()	인자와 반환값이 없는 실행용 작업	() -> System.out.println("Hello")
Supplier<T>	T get()	값을 반환하지만 매개변수는 없음	() -> "SKALA"
Consumer<T>	void accept(T t)	값을 소비하지만 반환값은 없음	s -> System.out.println(s)
Function<T,R>	R apply(T t)	하나의 인자를 받아서 값을 반환	x -> x.length()
Predicate<T>	boolean test(T t)	조건식(참/거짓)을 평가	x -> x.startsWith("S")
Comparator<T>	int compare(T o1, T o2)	두 값 비교 작업에 사용	(a, b) -> a.length() - b.length()

```java
@FunctionalInterface
public interface Function<T, R> {
    R apply(T t);
}
```

함수형 인터페이스 – Runnable

- Runnable 인터페이스는 run()이라는 단 하나의 메서드를 가진 매우 간단한 인터페이스
- 매개변수를 받지도 않고, 값을 반환하지도 않으며, 동시에 실행될 수 있는 작업을 나타내기 위해 설계
- 보통 별도의 스레드(thread)에서 실행될 코드 블록으로 구성

```
public class Main {
    public static void main(String[] args) {
        Runnable lambdaRunnable = () -> System.out.println("별도의 스레드에서 실행");
        Thread thread2 = new Thread(lambdaRunnable);
        thread2.start();
    }
}
```

함수형 인터페이스 – Predicate

- Predicate 인터페이스는 단일 값을 매개변수로 받아 true 또는 false를 반환하는 간단한 함수
- 주로 데이터를 필터링하는 데 사용 - 특정 조건을 기반으로 객체 리스트를 필터링할 때 사용

```
public class Main {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("Alice", "Bob", "Charlie", "David");

        // 문자열이 "A"로 시작하는지 테스트하는 Predicate 생성
        Predicate<String> startsWithA = (s) -> s.startsWith("A");

        // Predicate를 사용하여 리스트 필터링
        List<String> namesStartingWithA = names.stream()
                                               .filter(startsWithA)
                                               .collect(Collectors.toList());

        System.out.println(namesStartingWithA); // 출력: [Alice]

        // filter 메서드에 직접 람다를 사용할 수도 있음
        List<String> namesWithLength5 = names.stream()
                                             .filter(s -> s.length() == 5)
                                             .collect(Collectors.toList());

        System.out.println(namesWithLength5); // 출력: [Alice, David]
    }
}
```

함수형 인터페이스 – Function

- Function 인터페이스는 단일 값을 매개변수로 받아 처리 결과를 반환하는 간단한 함수
- 데이터에 대한 값 변환을 위해 사용

```
public class Main {
    public static void main(String[] args) {
        List<String> names = Arrays.asList("Alice", "Bob", "Charlie", "David");

        // 문자열을 대문자로 바꾸는 Function 생성
        Function<String, String> toUpper = s -> s.toUpperCase();

        // Function을 사용하여 리스트 변환
        List<String> upperNames = names.stream()
                                       .map(toUpper)
                                       .collect(Collectors.toList());
    }
}
```

함수형 인터페이스 – Comparator (1/2)

- Comparator 인터페이스는 객체 컬렉션의 사용자 정의 정렬 순서를 정하는 데 사용
- compare()라는 단 하나의 추상 메서드를 가지며, 이 메서드는 동일한 타입의 두 객체를 받아 정수를 반환
  - 음수: 첫 번째 객체가 두 번째 객체보다 앞에 온다.
  - 0: 두 객체의 순서가 같다.
  - 양수: 첫 번째 객체가 두 번째 객체보다 뒤에 온다.

```
class Person {
    private String name;
    private int age;

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public int getAge() {
        return age;
    }

    @Override
    public String toString() {
        return name + " (" + age + ")";
    }
}
```

함수형 인터페이스 – Comparator (2/2)

```
public class Main {
    public static void main(String[] args) {
        List<Person> people = Arrays.asList(
            new Person("Alice", 30),
            new Person("Bob", 25),
            new Person("Charlie", 35)
        );

        // 나이순으로 정렬하는 Comparator 생성
        Comparator<Person> byAge = (p1, p2) -> Integer.compare(p1.getAge(), p2.getAge());

        // Comparator를 사용하여 리스트 정렬
        Collections.sort(people, byAge);

        System.out.println(people); // 출력: [Bob (25), Alice (30), Charlie (35)]
    }
}
```

람다 표현식(Lamda Expression) 개발 방식

1. 함수형 인터페이스 정의

```
@FunctionalInterface
public interface IAddable<T> {
    // 두 개의 객체를 더하는 메서드
    T add(T t1, T t2);
}
```

2. 함수형 인터페이스 기반 처리 함수

```
public static <T> void addGeneric(IAddable<T> adder, T a1, T a2) {
        System.out.println(adder.add(a1, a2));
}
```

3. Lambda 표현식을 이용한 Functional Interface 구현

```
// 문자열 결합
addGeneric((s1, s2) -> s1 + s2, "Hello, ", "World!");  // Hello, World!

// 정수 덧셈
addGeneric((i1, i2) -> i1 + i2, 10, 20);  // 30

// Double 타입도 가능
addGeneric((d1, d2) -> d1 * d2, 3.5, 2.0);  //7.0
```

[실습] Lambda Expression

10.lambda/AddableDemo.java

IAddable Functional Interface를 Lambda Expression으로 구현해서
1. 문자열 결합 a1: “Hello”, a2: “World”
2. 정수 덧셈 a1: 10 , a2: 20
3. Double 타입 곱셈 a1: 3.5, a2: 2.0 으로 동작하는 코드를 작성해라

```
@FunctionalInterface
interface IAddable<T> {
    T add(T a1, T a2);
}

public class AddableDemo {
    // 제네릭 메서드: 어떤 타입이든 처리 가능
    public static <T> void addGeneric(IAddable<T> adder, T a1, T a2) {
        System.out.println(adder.add(a1, a2));
    }
    public static void main(String[] args) {
        // 문자열 결합  a1: “Hello”, a2: “World”
        addGeneric (…)

        // 정수 덧셈  a1: 10 , a2: 20
        addGeneric();

        // Double 타입도 가능 (예시: 곱셈)  a1: 3.5, a2: 2.0
        addGeneric(..);
```

메서드 참조(Method Reference)

- 메서드 참조(Method Reference)는 람다 표현식을 더 간결하게 표현하는 문법
- 기존 메서드를 그대로 전달하고 싶을때 사용하는 문법

Method Reference 기본 구조

```
클래스명(or 객체명) :: 메서드명
```

주어진 x를 가지고 println 메소드를 호출하라 : **x, ->, (x) 제거**

```
list.forEach(x -> System.out.println(x));
```

```
list.forEach(System.out::println);
```

유형	Lambda	Method Reference
정적 메서드 참조	x -> String.valueOf(x)	String::valueOf
특정 객체의 인스턴스 메서드 참조	x -> System.out.println(x)	System.out::println
특정 타입의 임의 객체 메서드 참조	s -> s.toUpperCase()	String::toUpperCase
생성자 참조	() -> new ArrayList<String>()	ArrayList::new

```
List<String> stocks = List.of("SKALA AI", "SKALA 에듀", "K-테크");

// 람다 표현식
stocks.forEach(s -> System.out.println(s));

// 메서드 참조
stocks.forEach(System.out::println);
```

람다 표현식에서 외부 변수 사용하기

람다 표현식에서 사용하는 외부 변수는 함수내 Capture 사용
사용되는 외부 변수는 final 또는 effectively final 로 설정

```
// OK

int base = 10;  // effectively final (변경되지 않음)

Function<Integer, Integer> adder = x -> x + base;

System.out.println(adder.apply(5));  // 15
```

```
int base = 10;

base = base + 5; // ❌ 컴파일 오류

Function<Integer, Integer> adder = x -> x + base;

base = 20;  // ❌ 컴파일 오류
```

```
int base = 10;

Runnable r = () -> {
    base = base + 1;  // ❌ 컴파일 오류
};
```

클로저로 동작 이유

- 외부 변수 시점과 람다 함수 호출 시점의 차이로 인한 데이터 불일치 문제 해소
- 스레드 안정성 (thread safety)
- 가비지 컬렉션 최적화

[참고] effective final 이란

effectively final은 **'변수 타입의 특성'**이 아니라 **'코드에서 실제로 값이 변경되었는지'**로 판단

코드에서 실제로 값이 한 번도 변경되지 않았다면 → effectively final

```
int base = 10;  // effectively final (변경되지 않음)

Function<Integer, Integer> adder = x -> x + base;

System.out.println(adder.apply(5));  // 15
```

변경할 수 있는 타입이냐를 보지않고, 코드에서 실제로 변경했나만 확인

```
int base = 10;  // effectively final (변경되지 않음)

Function<Integer, Integer> adder = x -> x + base;

System.out.println(adder.apply(5));  // 15

base = 20;  // ❌ 컴파일 오류
```

[참고] 람다함수가 순수 함수를 유지해야 하는 이유

장점	설명
예측 가능성	같은 입력 → 같은 출력, 디버깅이 쉬움
테스트 용이	외부 의존 없이 독립적으로 테스트 가능
병렬 처리 안전	외부 상태를 건드리지 않으니 race condition 없음
캐싱 가능	결과를 재활용(Memoization) 가능
리팩토링 안정성	의존성 적어서 코드 변경 영향 최소화

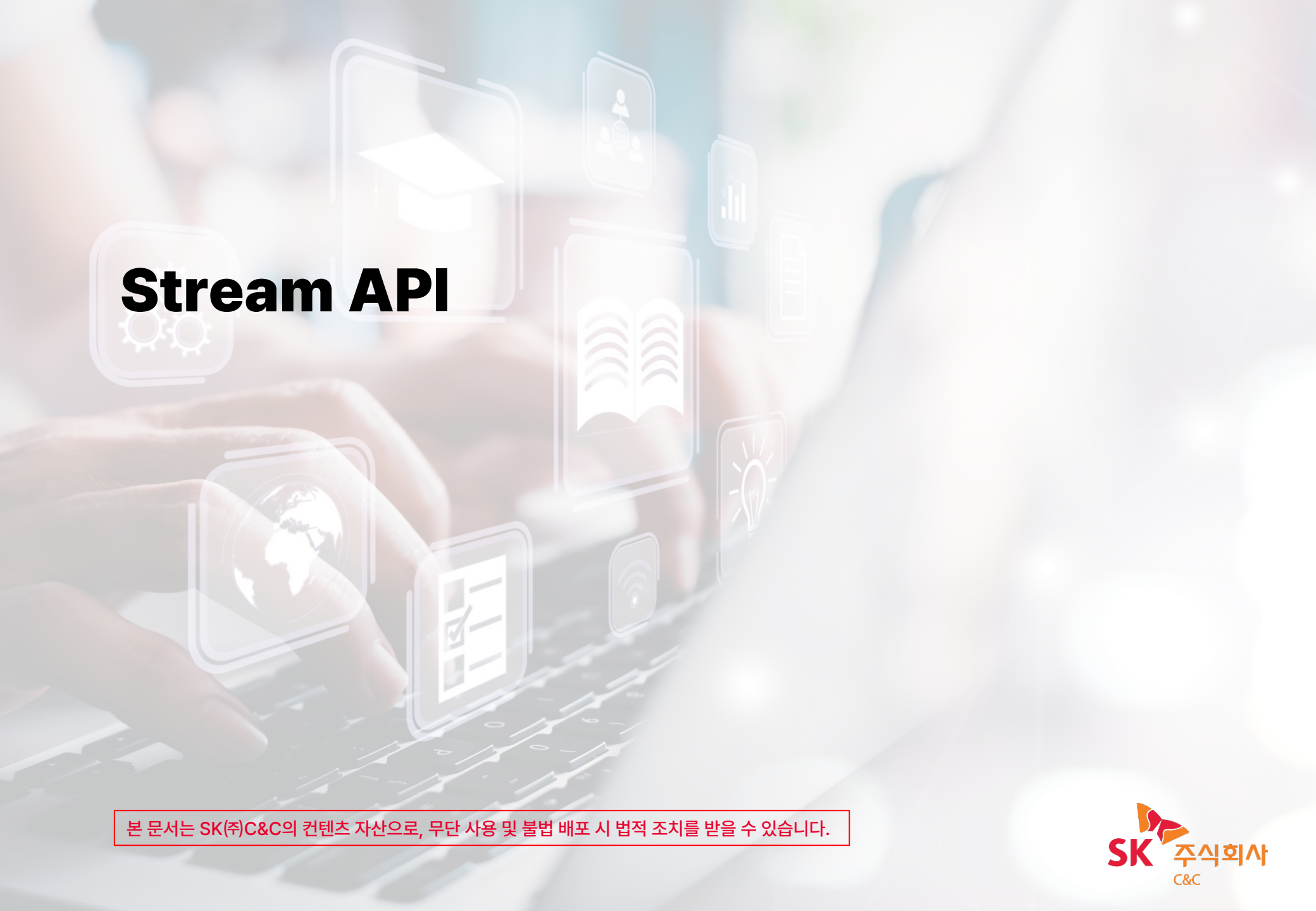
[실습] 외부 변수 확인

10.lambda/LambdaExternalVariable.java

Lambda Expression에서 사용하는 외부 변수 사용 특성을 간단하게 테스트해보자
base의 effectively final 특성을 확인해보자

```
import java.util.function.*;

public class LambdaExternalVariable {
    public static void main(String[] args) {
        int base = 10;
        base = base + 5; // 외부 변수 base를 변경
        Function<Integer, Integer> addBase = (x) -> x + base;
        System.out.println(addBase.apply(5)); // 15

        // 외부 변수는 람다식 내부에서 final 또는 effectively final이어야 함
        // base = 20;

    }
}
```

이부분에 대한 것은 comment를 해지하고 동작시켜보자
정상 동작하는지 확인해보자



Stream API

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

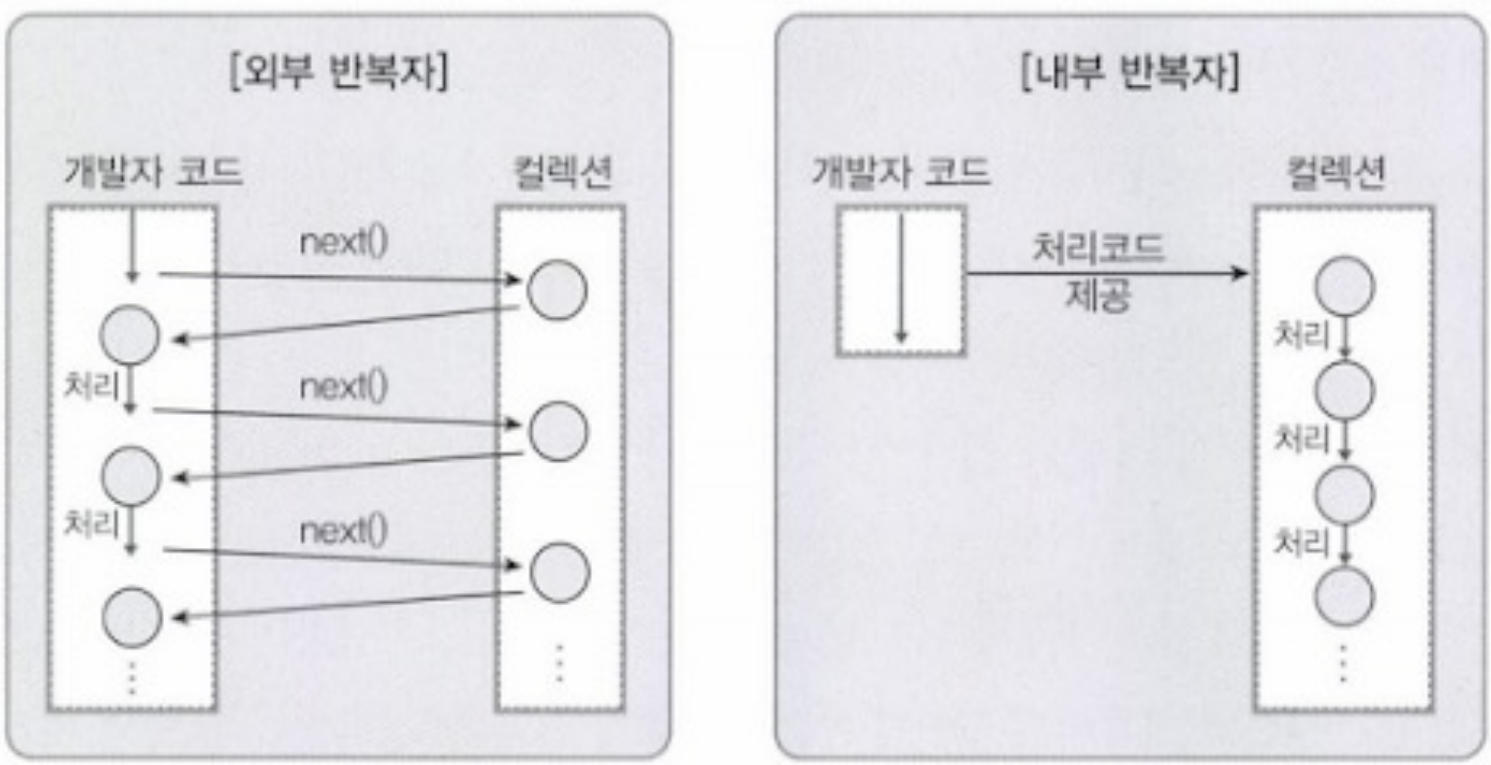
SK 주식회사 C&C

Stream API 란?

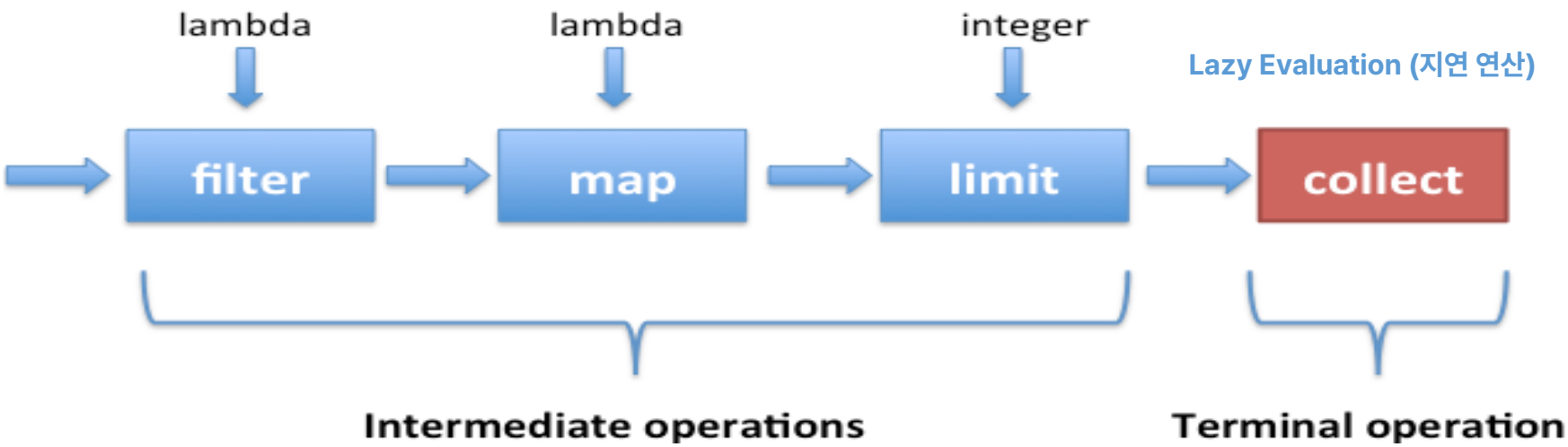
Java 8에서 도입된 기능으로 컬렉션(List, Set, Map 등)의 요소들을 선언적(Declarative)이고 함수형(Functional) 스타일로 처리할 수 있도록 지원하는 API

기존의 for-loop 또는 Iterator 기반의 반복적인 데이터 처리 코드를 간결하고 가독성 높게 작성하도록 지원

현행 구조 **Stream 구조**



Stream API 란?




```
List<String> result = names.stream()
    .filter(name -> name.startsWith("A"))  // filter (Predicate<T> 사용)
    .map(String::toUpperCase)              // map (Function<T, R> 사용)
    .limit(2)                               // limit (최대 2개 요소로 제한)
    .collect(Collectors.toList());         // collect (Terminal operation)
```

Stream API 란?

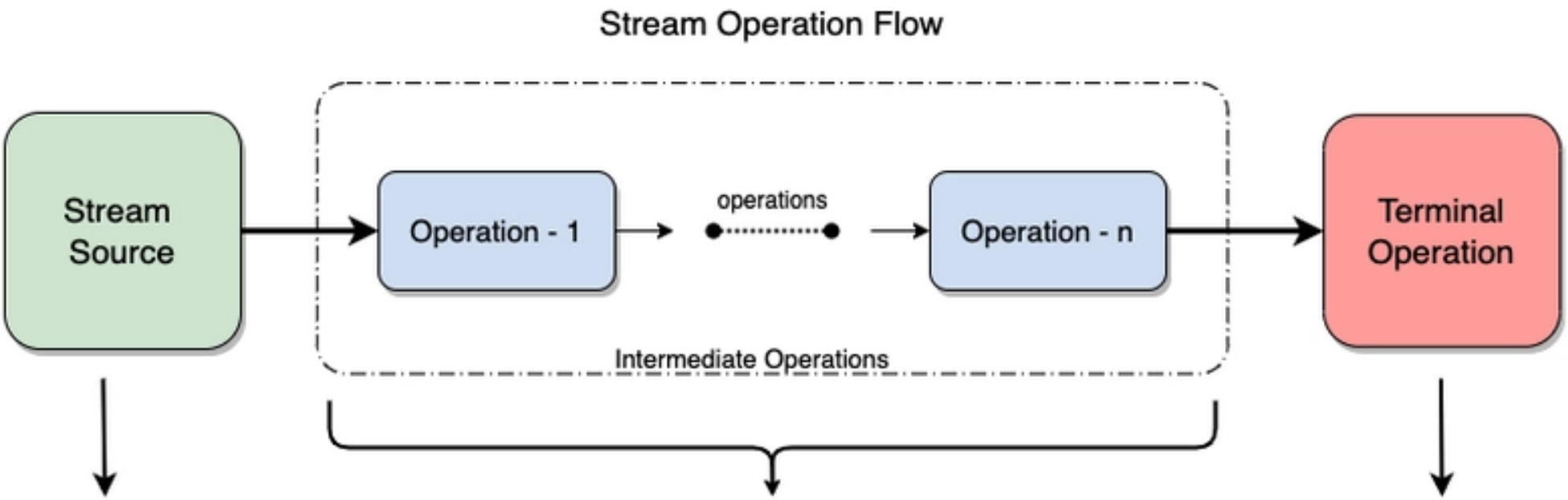
일반적인 코드 구조

```
List<String> result = new ArrayList<>();
for (String name : names) {
    // 1. "A"로 시작하는지 검사 (filter)
    if (name != null && name.startsWith("A")) {
        // 2. 대문자로 변환하여 결과 리스트에 추가 (map & collect)
        result.add(name.toUpperCase());

        // 3. 2개를 채웠으면 반복 종료 (limit)
        if (result.size() == 2) {
            break;
        }
    }
}
```

Stream 코드 구조

```
List<String> result = names.stream()
        .filter(name -> name.startsWith("A")) // filter (Predicate<T> 사용)
        .map(String::toUpperCase)             // map (Function<T, R> 사용)
        .limit(2)                             // limit (최대 2개 요소로 제한)
        .collect(Collectors.toList());        // collect (Terminal operation)
```

Stream API 란?



List, Map, Set, Arrays etc

- map(), filter(), unordered(), peek(), mapToInt() etc
- distinct(), sorted(), limit() etc

min(), max(), sum(), count(), average(), reduce(), collect()

Stream API 예시 (1/2)

12.lambda/sample01

문제: 현재 판매 중이고 십만원 이하인 상품의 개수는 ?

```
public class Product {

    private  int id;
    private boolean usable;
    private String category;
    private int price;

    public int getId()
    public boolean isUsable()…
    public String getCategory()
    public int getPrice() …

}
```

```
public void setProduct() {
    Product p1 = new Product(1, true, "fashion", 50000);
    Product p2 = new Product(2, true, "fashion", 38000);
    Product p3 = new Product(3, true, "it", 250000);
    Product p4 = new Product(4, true, "it", 85000);
    Product p5 = new Product(5, true, "it", 99000);
    Product p6 = new Product(6, true, "it", 75000);
    Product p7 = new Product(7, true, "furniture", 350000);
    Product p8 = new Product(8, false, "furniture", 210000);
    Product p9 = new Product(9, false, "furniture", 58000);
    Product p10 = new Product(10, false, "it", 120000);

    products.add(p1);
    products.add(p2);
    :
}
```

Stream API 예시 (2/2)

for each 사용

```
public void filterBeforeJava8() {
    int count = 0;

    for(Product pd : products) {

        if(pd.isUsable() &&

            pd.getPrice() <= 100000) {

            count++;
        }
    }


    System.out.println("현재 판매 중이고 가격이 십만원 이하인 상품의 개수 : " +
            count + "개 ");
}
```

상품목록에서 개별 상품을 구해서

현재 판매 중이고

십만원 이하라면

개수를 1증가

Stream API

```
public void filterJava8() {
    long count = products.stream()
                         .filter(p -> p.isUsable())
                         .filter(p -> p.getPrice() <= 100000)
                         .count();


    System.out.println("현재 판매 중이고 가격이 십만원 이하인 상품의 개수 : " +
            count + "개 ");
}
```

상품 중에서

판매 중이고

십만원 이상인 상품의

개수를 구함

Stream API

stream()은 선언적 방식으로 동작한다

```
public void filterJava8() {
    long count = products.stream()
                         .filter(p -> p.isUsable())
                         .filter(p -> p.getPrice() <= 100000)
                         .count();
}
```

Stream<Product> 반환

`Predicate<Product>` 등록 → `Stream<Product>` 반환

최종 연산 → `long` 반환

어떻게 ? 동작하는지 설명해보자

[실습] Stream API

11.lambda/sample01

StreamMain.java 의 products List에서
Product의 usable = true 인 모든 product의 price 합을 계산하고 화면에 출력하는 코드를 작성해보자
현재 for 문의 코드를 Stream API 로 전환해라

일반적인 List 처리

```
int count = 0;
for(Product p : products) {
    if(p.isUsable()) {
        if(p.getPrice() <= 100000) {
            count++;
        }
    }
}
```

Stream API 처리

```
// Stream API 사용
long count2 = products.stream()
    .filter(Product::isUsable)
    .filter(p -> p.getPrice() <= 100000)
    .count();
```

Java Stream API

다음과 같은 연산을 호출하면 스트림이 실행됩니다:

최종 연산	설명	반환값
`collect()`	스트림을 리스트, 셋, 맵 등으로 수집	`List<T>` , `Set<T>` , `Map<K,V>`
`forEach()`	각 요소에 대해 특정 동작 수행	`void`
`reduce()`	모든 요소를 하나의 값으로 합침 (예: 합계, 곱셈)	`Optional<T>` 또는 `T`
`count()`	요소의 개수를 반환	`long`
`findFirst()`	첫 번째 요소를 반환	`Optional<T>`
`findAny()`	아무 요소나 반환 (병렬 스트림에서 유용)	`Optional<T>`
`allMatch()`	모든 요소가 조건을 만족하는지 검사	`boolean`
`anyMatch()`	하나라도 조건을 만족하는지 검사	`boolean`
`noneMatch()`	모든 요소가 조건을 만족하지 않는지 검사	`boolean`
`toArray()`	스트림을 배열로 변환	`T[]`

Stream API 사용 객체 예시

- Stream API를 사용할 수 있는 객체는 주로 java.util.stream.Stream을 생성할 수 있는 데이터 소스

❖ List

```
List<String> names = List.of("SKALA", "JAVA", "STREAM");
names.stream().map(String::toLowerCase).forEach(System.out::println);
```

❖ 배열

```
int[] numbers = {1, 2, 3, 4, 5};
IntStream.of(numbers).forEach(System.out::println);
```

❖ Map

```
Map<String, Integer> stock = Map.of("SKALA AI", 15000, "SKALA EDU", 12000);
stock.entrySet().stream()
    .filter(e -> e.getValue() > 13000)
    .forEach(e -> System.out.println(e.getKey()));
```

❖ Files.lines() – 파일 한 줄씩 읽기

```
Files.lines(Paths.get("data.txt"))
     .filter(line -> line.contains("SKALA"))
     .forEach(System.out::println);
```

Method Chaining

메서드 체이닝이란, 한 메서드의 반환값이 다음 메서드를 호출할 수 있도록 설계되어 연속적으로 호출하는 방식
즉, 여러 개의 메서드를 한 줄로 연결하여 호출하는 프로그래밍 기법

예시) Java Stream API

java.util.stream.Stream<T>

```
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
names.stream()
        .filter(name -> name.startsWith("A"))
        .map(String::toUpperCase)
        .forEach(System.out::println);  // "ALICE"
```

예시) Method chaining & Usage

```
class Person {
    private String name;
    private int age;

    public Person setName(String name) {
        this.name = name;
        return this;  // 자기 자신을 반환하여 체이닝 가능하게 만듦
    }

    public Person setAge(int age) {
        this.age = age;
        return this;
    }

    public void display() {
        System.out.println("이름: " + name + ", 나이: " + age);
    }
}

// 메서드 체이닝을 이용한 객체 생성 및 설정
    Person person = new Person()
                        .setName("홍길동")
                        .setAge(30);
```

[실습] 아래의 요청에 따라 직접 작성해보자

11.lambda/sample02

다음 요청에 따라 코드를 작성해봅시다.

List<Integer> numbers = List.of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)

를 Stream의 입력으로 사용해서

1. n*2 로 리턴하는 method map()을 등록해라
2. map 처리 결과값이 n > 5 이 값만 Stream 처리되도록 filter()를 설정해라
3. forEach를 이용해서 Stream을 통해 전달된 숫자를 프린트해라

```
public class StreamMain {
    public static void main(String[] args) {
        List<Integer> numbers = List.of(1, 2, 3, 4, 5, 6, 7, 8, 9, 10);

     // 여기에 코드를 작성하세요

    }

}
```

자바 Util 클래스

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

주요 java.util 클래스

- java.util 패키지는 자바에서 가장 자주 사용하는 유틸리티 클래스들이 모여 있는 핵심 패키지

클래스명	주요 용도 및 설명
Collections	컬렉션(List, Set 등)을 정렬하거나 섞는 등의 다양한 정적 유틸리티 메서드를 제공. ex. sort(), shuffle()
Arrays	배열 관련 유틸리티 제공. ex. Arrays.sort(), Arrays.toString() 등
Optional<T>	값이 있을 수도 있고 없을 수도 있는 경우를 처리하는 컨테이너. null 체크를 대체할 수 있는 안전한 방식.

java.util – Collections

- 컬렉션(List, Set, Map 등)을 조작하기 위한 정적(static) 유틸리티 메서드 모음 클래스입니다.
- java.util.Collection 인터페이스와는 다른 개념

메서드	설명
sort(List)	리스트를 오름차순 정렬
reverse(List)	리스트의 요소 순서 반전
shuffle(List)	리스트 요소를 무작위로 섞기
max(Collection) / min(Collection)	최대값 / 최소값 찾기
frequency(Collection, Object)	특정 요소의 등장 횟수 계산
replaceAll(List, oldVal, newVal)	특정 값 일괄 치환

```
List<Integer> prices = new ArrayList<>();
prices.add(18000);
prices.add(9500);
prices.add(12000);

// 오름차순 정렬
Collections.sort(prices);
System.out.println("오름차순: " + prices);  // [9500, 12000, 18000]

// 내림차순 정렬
Collections.reverse(prices);
System.out.println("내림차순: " + prices);  // [18000, 12000, 9500]
```

java.util – Arrays

- 배열 관련 기능을 지원하는 정적(static) 메서드 유틸 클래스
- 배열은 Java의 기본 데이터 구조이지만, 기능이 제한적이므로 Arrays 클래스를 통해 다양한 조작이 가능

메서드	설명
sort(array)	배열 정렬 (기본형, 참조형 모두 가능)
toString(array)	배열을 문자열로 변환하여 출력 가능
copyOf(array, length)	배열 복사 (길이 지정 가능)
equals(array1, array2)	배열 간 값 비교
fill(array, value)	배열을 동일한 값으로 초기화
binarySearch(array, key)	정렬된 배열에서 이진 탐색 수행

```
import java.util.Arrays;

public class StockArray {
    public static void main(String[] args) {
        int[] prices = {18000, 9500, 12000};

        Arrays.sort(prices);  // 오름차순 정렬

        // 정렬된 주식 가격: [9500, 12000, 18000]
        System.out.println("정렬된 주식 가격: " + Arrays.toString(prices));
    }
}
```

java.util – Optional

- Optional은 null로 인해 발생할 수 있는 예외(NullPointerException)를 방지하기 위한 컨테이너 클래스
- 값이 존재할 수도 있고, 없을 수도 있는 상황에서 명시적으로 null 여부를 표현하기 위해 사용

Optional 생성 방법	설명
Optional.of(value)	절대 null이 아닌 값 감싸기 (null이면 예외 발생)
Optional.ofNullable(value)	Value가 null 이면 Optional.empty() 를 리턴
Optional.empty()	비어 있는 Optional 생성

주요 메서드	설명
isPresent()	값이 존재하는지 여부 (deprecated 경향)
ifPresent(Consumer)	값이 존재하면 처리 수행
get()	값 반환 (존재하지 않으면 예외 발생 NoSuchElementException – 권장 X) get() 보다는 orElse, orElseGet을 권고
orElse(value)	값이 없으면 기본값 반환
orElseGet(Supplier)	값이 없으면 지연된 로직으로 값 생성
orElseThrow()	값이 없으면 예외 발생

[실습] java.util – Optional

01.snippet/OptionalExample.java

Optional 을 테스트해보자

```
public class OptionalParamExample {

    public static void printName(Optional<String> optName) {
        // 값이 있으면 대문자로 출력, 없으면 "이름 없음"
        String name = optName
                .map(String::toUpperCase)                    // 값이 있으면 대문자로 변환
                .orElse("이름 없음");                          // 값이 없으면 기본값
        System.out.println("결과: " + name);
    }

    public static void emailCheck(Optional<String> email) {
        // 값이 있고 '@' 가 포함된 경우만 소문자로 변환 후 발송
        email.filter(s -> s.contains("@"))                   // 유효한 이메일만
             .map(String::toLowerCase)                       // 소문자로 변환
             .ifPresent(s -> System.out.println("메일 발송: " + s));
    }

    public static void main(String[] args) {
        Optional<String> email1 = Optional.of("hong@gmail.com"); // 값이 있는 경우 (정상 이메일)
        emailCheck(email1);                                  // "메일 발송: hong@gmail.com"

        Optional<String> email2 = Optional.empty();          // 값이 없는 경우
        printName(email2);                                   // "결과: 이름 없음"

        // null 가능성을 가진 변수
        String nullableName = null;
        Optional<String> name3 = Optional.ofNullable(nullableName);
        emailCheck(name3);                                   // '@' 가 없으므로 발송 안 됨
        printName(name3);                                    // "결과: HONGGILDONG"
    }
}
```

Reflection

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

Reflection 이란?

모르는 대상을 사용해야 할때 대상에 대해 정보를 알려주고 사용할 수 있도록 도와 주는 기술

Rumtime 시점에 클래스에 대한 정보를 검색해 클래스 또는 객체를 조작할 수 있도록 지원

특징

- 컴파일 시점이 아니라 런타임에 타입을 조작
- Private 포함 모든 속성/멤버 접근
- 동적 객체 생성, 메소서 호출 지원

필요성

- 프레임워크 내부 동작 시

  - 스프링 DI는 @Autowired 필드에 리플렉션으로 값 주입

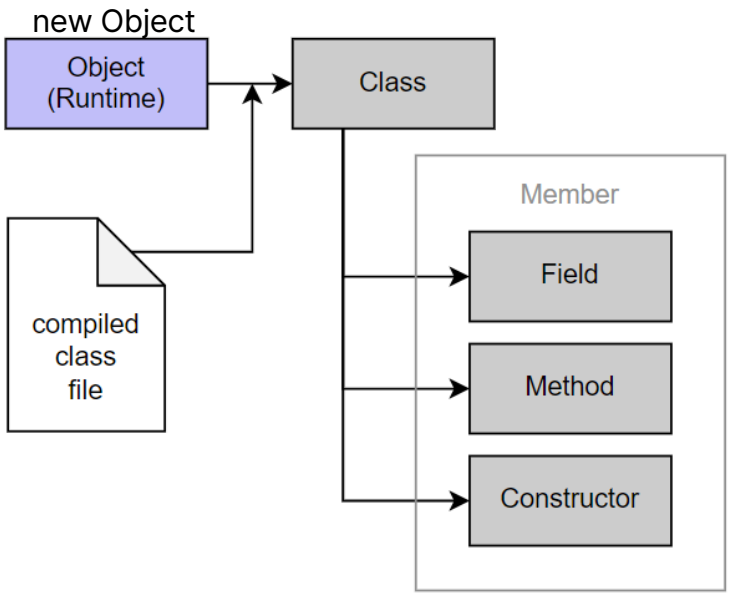
  - ORM (JPA)은 엔티티 필드를 리플렉션으로 읽고 쓰기
- 유연한 코드 구조

  - 클래스 이름을 문자열로 전달 받아 객체 생성

  - 플러그인 구조, 확장 가능한 아키텍처 제공

Reflection 의미

- 메타데이터 저장소(메타스페이스, Metaspace)에 올려놓은 클래스의 구조를 읽어오고 조작할 수 있는 기능
- reflection = Metaspace 정보 조회/조작 도구
- java.lang.reflect 패키지에 포함



Class 와 Member

- **Class**: 클래스를 나타내며, Member를 가져오기 클래스를 조회
  Class<?> clazz = Class.forName(" com.example.User");
- **Constructor**: 생성자를 나타내며, 이를 통해 Instance를 생성 가능
  Object instance = clazz.getDeclaredConstructor().newInstance();
- **Method** : 메소드를 나타내며, 호출하여 사용 가능
  clazz.getDeclaredMethod("get", String.class).invoke(instance, "Hi Reflection");
- **Field** : 속성을 나타내며, 필드 값을 조회/변경 가능
  clazz.getDeclaredField(" message").set(instance, "my message")

메서드	대상 생성자	상속 포함
getConstructor()	public 생성자만	포함
get**Declared**Constructor()	선언된 생성자	제외

Why Reflection (1/2)

실행 시점에 객체가 결정되어야 하는 경우 이를 위한 방법 제공 예) 클래스 이름, 파라메터 유형 으로 대상 객체 자동 생성 및 실행 등

1) 런타임 동적 로딩 및 조작

- 컴파일 시점이 아닌 실행 중에 객체 정보를 확인하거나 수정 가능.
- 클래스 이름이 동적으로 결정되거나, 조건에 따라 특정 메서드를 실행해야 할 때 사용.

```
Class<?> clazz = Class.forName( " com.skala.myapp.domain.User");  //User.class로 도 사용 가능
var ctor = clazz.getDeclaredConstructor(String.class, Integer.class);

ctor.setAccessible(true);

Object obj = ctor.newInstance(“홍길동”, 20);
```

2) 어노테이션 기반 동작 제어

- 런타임에 애노테이션을 읽어서 특정 메서드나 필드에 대해 자동 동작을 수행.
- 예: Spring에서 @Autowired로 의존성 주입, @RequestMapping으로 URL-메서드 매핑

```
Method[] methods = obj.getClass().getDeclaredMethods();
for (Method method : methods) {
    // method에 @MyAnnotation이 붙어 있는지 확인
    if (method.isAnnotationPresent(MyAnnotation.class)) {
        method.invoke(obj); // 애노테이션이 붙은 메서드 실행, static method는 method.invoke(null)
    }
}
```

Why Reflection (2/2)

3) 플러그인 시스템 및 동적 확장

- 실행 중에 플러그인 추가, 또는 인터페이스를 구현하지 않은 클래스도 동적으로 실행 가능.
- 외부 패키지의 클래스를 런타임에 로드하여 확장 기능 구현 가능.
- 다른 클래스를 로드하려면, 해당 클래스가 존재하는 경로를 알 수 있는 **클래스 로더**가 필요

```
// this.class.getClassLoader() 가능
ClassLoader classLoader = MyClass.class.getClassLoader();
Class<?> dynamicClass = classLoader.loadClass("com.plugin.MyPlugin");
Object pluginInstance = dynamicClass.getDeclaredConstructor().newInstance();
```

[참고] 새로운 Annotation 만들기

Annotation을 생성 및 활용 방식

```
@Retention(RetentionPolicy.RUNTIME)    // SORUCE (Override 등), CLASS, RUNTIME(@Autowired)
@Target(ElementType.METHOD)            // TYPE, FIELD, METHOD, CONSTRUCTOR etc
public @interface MyAnnotation {}

public class MyService {
    @MyAnnotation
    public void runTask() {
        System.out.println("태스크 실행");
    }

    public void normalMethod() {
        System.out.println("일반 메서드");
    }
}
```

Reflection을 통해 Annotation Method 호출

```
MyService obj = new MyService();
Method[] methods = obj.getClass().getDeclaredMethods();
for (Method method : methods) {
    if (method.isAnnotationPresent(MyAnnotation.class)) {
        method.invoke(obj); // ”runTask() 실행"
    }
}
```

Reflection 클래스

Reflection을 사용하기 위한 클래스 목록

클래스 이름	설명
Class<?>	클래스 자체를 나타내는 메타 정보 객체
Constructor<?>	생성자를 나타내는 객체
Method	클래스의 메서드를 나타내는 객체
Field	클래스의 필드를 나타내는 객체
Parameter	메서드나 생성자의 파라미터 정보를 나타내는 객체
Modifier	클래스, 메서드, 필드 등의 접근 제한자(public, private 등)를 나타내는 객체

Reflection 핵심 클래스: Class<?>

클래스 획득

```
// 1) 클래스 리터럴
Class<Person> clazz1 = Person.class;

// 2) 객체에서 가져오기
Person p = new Person();
Class<?> clazz2 = p.getClass();

// 3) 문자열로부터 로딩 (동적 로딩)
Class<?> clazz3 = Class.forName("com.example.Person");
```

동적 객체 생성하기

```
Class<?> clazz = Class.forName("com.example.Person");

// 기본 생성자 호출
Object obj = clazz.getDeclaredConstructor().newInstance();

// 파라메터 있는 생성자
Object obj2 = clazz.getDeclaredConstructor(String.class)
                   .newInstance(”himang10”)
```

필드 (Field) 접근

```
Class<?> clazz = Class.forName("com.example.Person");
Object obj = clazz.getDeclaredConstructor().newInstance();

Field field = clazz.getDeclaredField("name");
field.setAccessible(true);  // private 필드 접근 허용

field.set(obj, "홍길동");     // 값 설정
System.out.println(field.get(obj)); // 값 읽기
```

메서드 (Method) 호출

```
Class<?> clazz = Class.forName("com.example.Person");
Object obj = clazz.getDeclaredConstructor().newInstance();

// sayHello(String name) 메서드 호출
Method method = clazz.getMethod("sayHello", String.class);
method.invoke(obj, "홍길동");
```

Reflection 핵심 클래스: Class<?>

생성자 사용

```
Class<?> clazz = Class.forName("com.example.Person");

// (String, int) 생성자 호출
Constructor<?> constructor = clazz.getConstructor(String.class, int.class);

Object obj = constructor.newInstance("홍길동", 20);
```

모든 맴버 조회하기

```
Class<?> clazz = Person.class;

Field[] fields = clazz.getDeclaredFields();
Method[] methods = clazz.getDeclaredMethods();
Constructor<?>[] constructors = clazz.getDeclaredConstructors();
```

Reflection 의 한계와 주의 사항

사용 시 다음 사항을 고려 필요

항목	설명
성능 저하	직접 접근보다 느림 (런타임 탐색 오버헤드)
캡슐화 파괴	private 접근 가능 → 설계 규칙 위반 위험
런타임 오류 증가	컴파일 시점 타입 체크가 어려움

Reflection

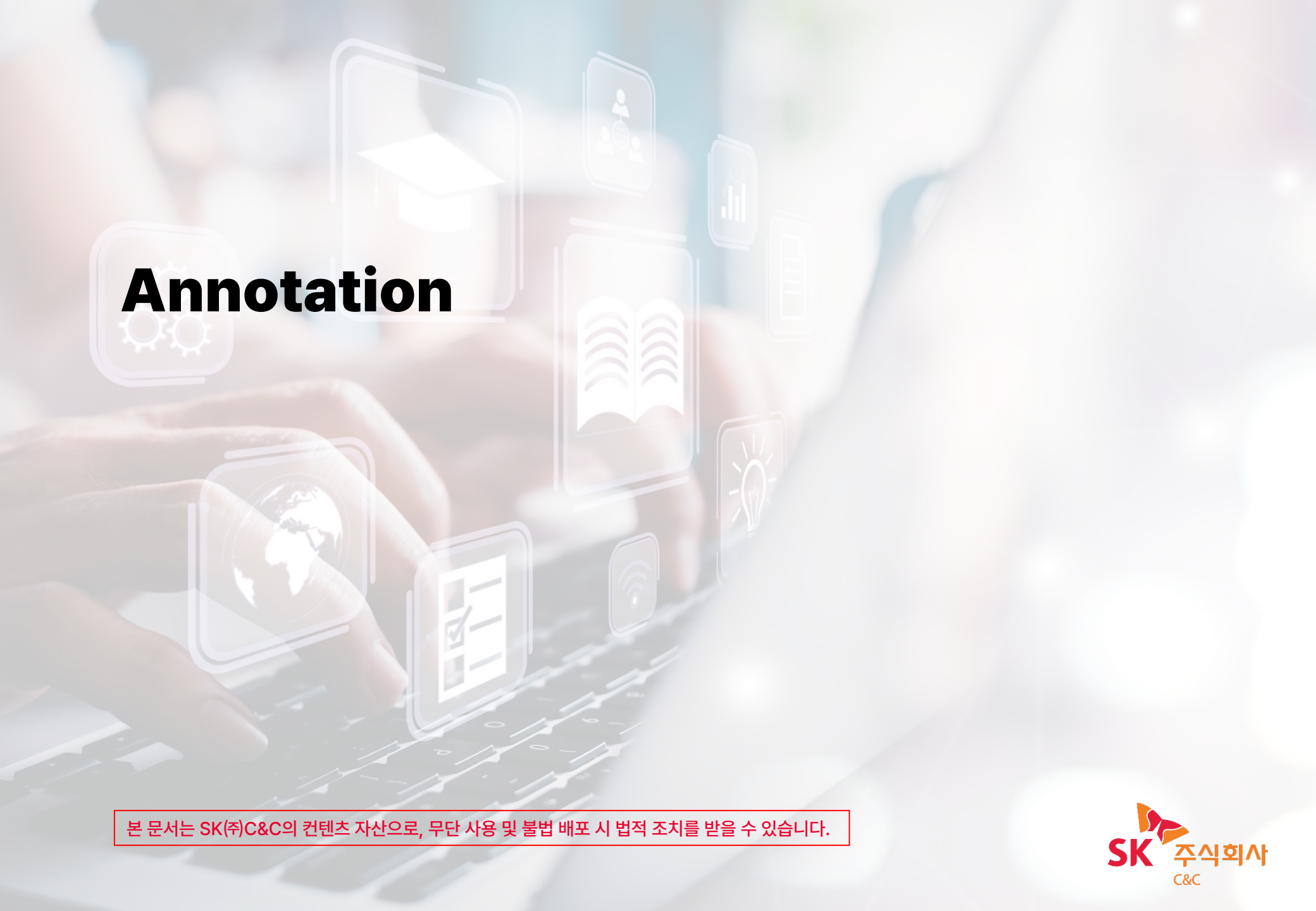
01.traing-code/12.reflection/

Reflection 관련 코드를 통해 Reflection 개념을 이해해보자

- **Class 메타 정보 조회 예제:** ReflectionClassInfo.java
- **생성자 (Constructor) 접근 예제**: ReflectionConstructor.java
- **메서드(Method) 접근 예제**: ReflectionMethod.java
- **필드(Field) 접근 예제**: ReflectionField.java
- **메서드(Method) 파라미터 포함 호출 예제**: ReflectionMethodParam.java
- **리플렉션 어노테이션 (Annotation) 읽기 예제**: ReflectionAnnotation.java

실행해보기

```
> javac -d bin *.java
> java -cp bin  ReflectionClassInfo
```



Annotation

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

Annotation 이란?

자바의 Annotation은 클래스, 메소드, 필드에 Hint 붙이는 속성 (실제 실행은 Reflection등을 통해 별도 실행)

특징

주석(// comment)은 사용자 읽기용, Annotation은 내부 프로그램이 읽고 동작을 바꾸기 위한 정보

- 해석하는 주체 (외부시스템): 컴파일러, JVM, spring 같은 Framework
- 역할은 주체에 Reflection을 이용해서 구조화된 Meta data를 제공

Annotation 유지 범위 (Retention)

```
1) SOURCE RETENTION
 (소스 코드에만 존재, 컴파일 시 제거)
        ↓
2) CLASS RETENTION
 (바이트코드에 포함, 런타임 접근 불가)
        ↓
3) RUNTIME RETENTION
 (런타임에 리플렉션으로 접근 가능)
```

```java
@Override
public String toString() { ... }

@Deprecated
public void oldMethod() { ... }

@SpringBootApplication
public class DemoApplication { ... }
```

Annotation 동작 방식

동작 흐름

1. Annotation 정의 (@interface 사용)
2. 클래스/메서드/필드 등에 Annotation을 적용
3. 적용 범위
   - 컴파일러가 해석하는 경우 (@Override, @Deprecated 등)
   - 프레임워크가 런타임으로 리플렉션으로 읽는 경우 (Spring의 @Service, @GetMapping 등)
   - 우리가 직접 리플렉션/프로세서로 읽고 동작을 구현

1) 선언

```
@interface RunMe { }
```

2) 적용

```
class MyTask {
    @RunMe
    public void task1() { ... }
}
```

3) 실행 코드

```
RunMe ann = clazz.getAnnotation(RunMe.class);
```

Annotation : @interface

- 새로운 Annotation을 만들 때 @interface 키워드를 사용
- 메서드 모양으로 "속성(요소)"을 정의

```java
// 값이 없는 마커(Marker) Annotation
public @interface RunMe {
}
```



```java
@RunMe
public class JobRunner {
    public void run() {
        System.out.println("실행 대상 클래스");
    }
}
```

```java
// 값이 1개 있는 Annotation
public @interface Author {
    String name();    // 필수 속성
}
```



```java
@Author(name = "홍길동")
public class MyService {
}
```

```java
// 값과 기본값이 있는 Annotation
public @interface Hint {
    String value() default "no-hint";
    int level() default 1;
}
```



```java
// 사용 방법
@Hint(value = "강한 힌트", level = 5)
public class MyClass {
}
```

Annotation : @Retention, @Target

@Target – 어디에 붙일 것인가?

```
import java.lang.annotation.*;

@Target(ElementType.METHOD)      // 메서드에만 사용
@Retention(RetentionPolicy.RUNTIME)
public @interface RunMe { }
```

- ElementType.TYPE : 클래스, 인터페이스, enum에만 붙일수 있음
- ElementType.FIELD : 필드에만 붙일 수 있음
- ElementType.METHOD : 메서드만
- ElementType.PARAMETER : 메서드 파라미터에만
- ElementType.CONSTRUCTOR : 생성자에게만

@Retention – 언제까지 살아남는가?

```
@Retention(RetentionPolicy.SOURCE)   // 소스까지만 (컴파일 후 사라짐)
@Retention(RetentionPolicy.CLASS)    // .class 까지 (기본값)
@Retention(RetentionPolicy.RUNTIME)  // 런타임까지 (리플렉션으로 읽을 수 있음)
```

- 실행 단계에서 코드에서 리플렉션으로 읽고 싶다면 RUNTIME으로 설정되어야 함
- RetentionPolicy.SORUCE 를 사용하기 위해서는 AbstractProcessor를 상속 후 process method로 구현

Annotation 사용법 : Class

Annotation 정의 : MyComponent.java

```
import java.lang.annotation.*;

@Target(ElementType.TYPE)                // 클래스, 인터페이스, enum
@Retention(RetentionPolicy.RUNTIME)
public @interface MyComponent {
    String value() default "";           // 이름 속성
}
```

Annotation 적용: OrderService.java

```
@MyComponent("orderService")  // value="orderService"와 동일
public class OrderService {
    // …
}
```

Reflection으로 Annotation MyCompoment 읽기 : Main.java

```
Class<?> clazz = OrderService.class;

if (clazz.isAnnotationPresent(MyComponent.class)) {
    MyComponent comp = clazz.getAnnotation(MyComponent.class);
    System.out.println("컴포넌트 이름 = " + comp.value());
}
```

[참고] Spring boot에서 사용하는 Annotation : Class

```
// ========================
// Class 레벨 Annotation 예시
// ========================
import org.springframework.stereotype.Service;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@RestController
public class UserController {
}

@Service
public class UserService {
}

@SpringBootApplication
public class DemoApplication {
}
```

[실습] Annotation : Class

01.traing-code/13.annotation/01.class

13.annotation/01.class 코드 내용을 검토하고 실행해보자

13.annotation/01.class를 컴파일하고 실행해보세요

```
javac -d bin -cp . *.java
java -cp bin Main
```

Annotation 사용법: Field

Annotation 정의

```
import java.lang.annotation.*;

@Target(ElementType.FIELD)
@Retention(RetentionPolicy.RUNTIME)
public @interface Inject {
}
```

적용 방법

```
public class OrderController {
    @Inject
    private OrderService orderService;  // 주입 대상
}
```

Reflection으로 코드 읽기

```
public class SimpleContainer {

    public static void injectDependencies(Object target) throws Exception {
        Class<?> clazz = target.getClass();

        for (Field field : clazz.getDeclaredFields()) {
            if (field.isAnnotationPresent(Inject.class)) {
                // 인스턴스를 만들어서 주입 (아주 단순한 예)
                Object dependency =
                        field.getType().getDeclaredConstructor().newInstance();

                field.setAccessible(true);
                field.set(target, dependency);
            }
        }
    }
}
```

[참고] Spring에서 적용하는 코드: Field

```
// ========================
// Field 레벨 Annotation 예시
// ========================
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.beans.factory.annotation.Value;

public class SampleFields {

    @Autowired
    private UserService userService;

    @Value("${server.port}")
    private int serverPort;
}
```

[실습] Annotation 사용법: Field

01.traing-code/13.annotation/02.field

13.annotation/02.field 코드 내용을 검토하고 실행해보자

```
javac -d bin -cp . *.java
java -cp bin Main
```

Annotation 사용법: Method

Annotation 정의

```
import java.lang.annotation.*;

@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
public @interface RunMe {
    int order() default 0;
}
```

적용 방법

```
public class TaskRunner {

    @RunMe(order = 2)
    public void taskB() { System.out.println("Task B"); }

    @RunMe(order = 1)
    public void taskA() { System.out.println("Task A"); }

    public void noAnnotation() { System.out.println("Ignore"); }
}
```

Reflection으로 코드 읽기

```
public static void main(String[] args) throws Exception {
    TaskRunner runner = new TaskRunner();
    Class<?> clazz = runner.getClass();

    Method[] methods = clazz.getDeclaredMethods();

    // @RunMe가 붙은 메서드만 추출 후 order 순으로 정렬
    Arrays.stream(methods)
          .filter(m -> m.isAnnotationPresent(RunMe.class))
          .sorted(Comparator.comparingInt(m -> m.getAnnotation(RunMe.class).order()))
          .forEach(m -> {
              try {
                  m.invoke(runner);  // 메서드 호출
              } catch (Exception e) {
                  e.printStackTrace();
              }
          });
}
```

[참고] Spring에서 적용하는 코드: Method

```
GET /users
POST /users
```

```
// ========================
// Method 레벨 Annotation 예시
// ========================
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.transaction.annotation.Transactional;

public class SampleMethods {

    @GetMapping("/users")
    public String getUsers() {
        return "users";
    }

    @PostMapping("/users")
    public void createUser() {
    }

    @Transactional
    public void save() {
    }
}
```

[실습] Annotation : Method

01.traing-code/13.annotation/03.method

13.annotation/03.method 코드를 확인하고 실행해보자

```
javac -d bin -cp . *.java
java -cp bin Main
```

Annotation 사용법: Parameter

Annotation 정의

```
import java.lang.annotation.*;

@Target(ElementType.PARAMETER)
@Retention(RetentionPolicy.RUNTIME)
public @interface ParamName {
    String value();
}
```

적용 방법

```
public class UserController {

    public void createUser(
            @ParamName("userName") String name,
            @ParamName("userAge") int age) {
        // ...
    }
}
```

Reflection으로 코드 읽기

```
Method method = UserController.class
        .getMethod("createUser", String.class, int.class);

Parameter[] params = method.getParameters();

for (Parameter p : params) {
    ParamName ann = p.getAnnotation(ParamName.class);
    if (ann != null) {
        System.out.println("실제 파라미터 이름: " + p.getName()
                + ", Annotation 이름: " + ann.value());
    }
}
```

[참고] Spring에서 적용하는 코드: Parameter

```
GET /users/10?name=hong

@PathVariable("id") → /users/{id} 경로 값
@RequestParam("name") → ?name=hong 쿼리 파라미터
```

```
POST /users HTTP/1.1
Host: example.com
Content-Type: application/json
Content-Length: 27

{
  "id": 10,
  "name": "hong"
}
```

```
// ========================
// Parameter 레벨 Annotation 예시
// ========================
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RequestBody;

public class SampleParameters {

    @GetMapping("/users/{id}")
    public void getUser(
            @PathVariable("id") Long id,
            @RequestParam("name") String name
    ) {
    }

    public void createUser(
            @RequestBody User user
    ) {
    }
}
```

[실습] Annotation : Parameter

01.traing-code/13.annotation/04.parameter

13.annotation/04.parameter 코드를 확인하고 실행해보자

```
javac -d bin -cp . *.java
java -cp bin Main
```

객체 생성 패턴

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

객체 생성 패턴(Creational Patterns)

- 객체의 생성 과정을 추상화하여, 시스템이 구체적인 클래스에 의존하지 않고 객체를 만들 수 있게 해주는 패턴

패턴명	주요 목적/역할	대표 예시 클래스
Singleton	단 하나의 인스턴스만 생성, 공유 - 전역에서 하나의 객체만 필요할 때	Runtime, Logger 등
Factory Method	객체 생성을 하위 클래스에서 담당 - 상위 클래스는 인터페이스만 제공, 하위 클래스가 생성 결정	Calendar.getInstance() 등
Abstract Factory	관련 객체 집합을 생성 - 객체 계열을 통일된 방식으로 생성	GUI Toolkit, DAO Factory 등
Builder	복잡한 객체의 생성 과정 분리 - 객체의 생성 절차를 단계별로 분리 및 체이닝 지원	StringBuilder, Lombok @Builder

객체 생성 패턴 – Singleton

- 시스템 전체에서 단 하나의 인스턴스만을 제공하며, 이를 어디서든 접근할 수 있도록 보장하는 패턴
  - **장점**: 메모리 절약, 전역 상태 공유, 인스턴스 생명주기 관리 용이.
  - **단점**: 테스트 어려움, 과도한 공유로 인한 결합도 증가, 멀티스레드 환경에서는 동기화 필요.

```
public class Singleton {
    private static final Singleton instance = new Singleton();
    private Singleton() {}
    public static Singleton getInstance() {
        return instance;
    }
 public void sayHello() {
        System.out.println("Hello, Singleton!");
    }
}
```

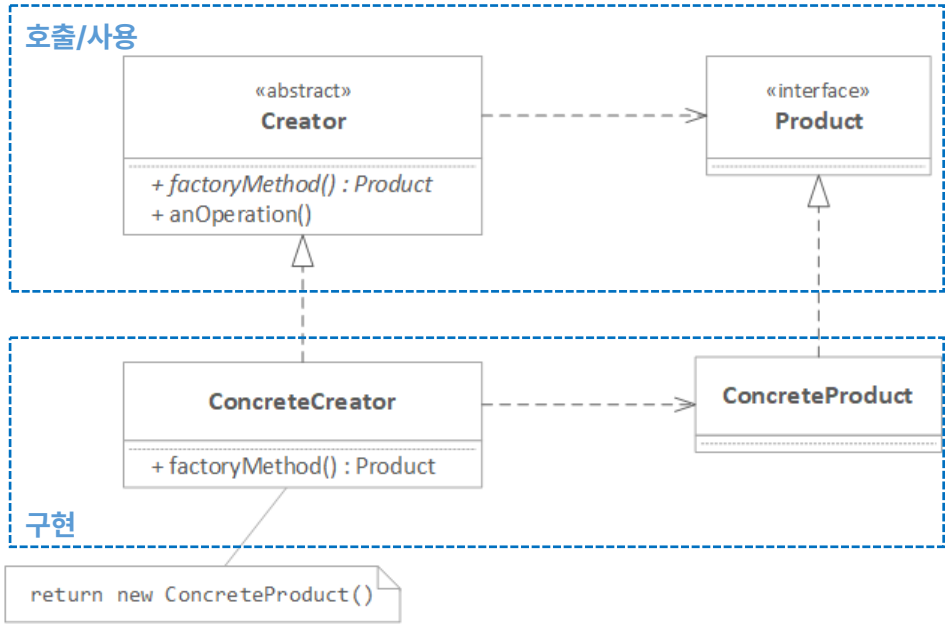
❖ **스프링에서 @Component, @Service, @Repository로 클래스를 선언 하면 싱글톤 객체로 관리 됨.**

```
public class Main {
    public static void main(String[] args) {
        Singleton s1 = Singleton.getInstance();
        Singleton s2 = Singleton.getInstance()
        s1.sayHello();
   }
}
```

객체 생성 패턴 – Factory Method (1/2)

객체 생성 코드를 별도의 factory Method에 위임하여, 객체 생성 로직과 처리 로직을 분리하는 패턴

- **장점**: 코드의 결합도 감소, 확장성 및 유지보수성 증가.
- **단점**: 클래스 수 증가, 코드 구조가 복잡해질 수 있음.




```java
abstract class Creator {
    public abstract Product factoryMethod();
}

class ConcreteCreator extends Creator {
    @Override
    public Product factoryMethod() {
        // 복잡한 사전 설정, 여러 구현체 중 선택,
        // 생성 제어 (캐싱, 폴링, 재사용, 권한 체크) 등
        return new ConcreteProduct();
    }
```

```java
public abstract class Product {
    public abstract void use();
}

public class ConcreteProduct extends Product {
    @Override
    public void use() {
        System.out.println("ConcreteProduct 사용!");
    }
}
```

객체 생성 패턴 – Factory Method(2/2)

처리 로직 (생성과 완전히 분리)

```
class ProductProcessor {

    // 구현고려하지 않고 사용 및 절차/프로세스만 구성 (구현 객체에 대한 명시적 호출이 없음)
    public void process(Creator creator) {
        Product product = crator.factoryMethod();   // How는 포함되어 있지 않음
        product.use();
    }
}
```

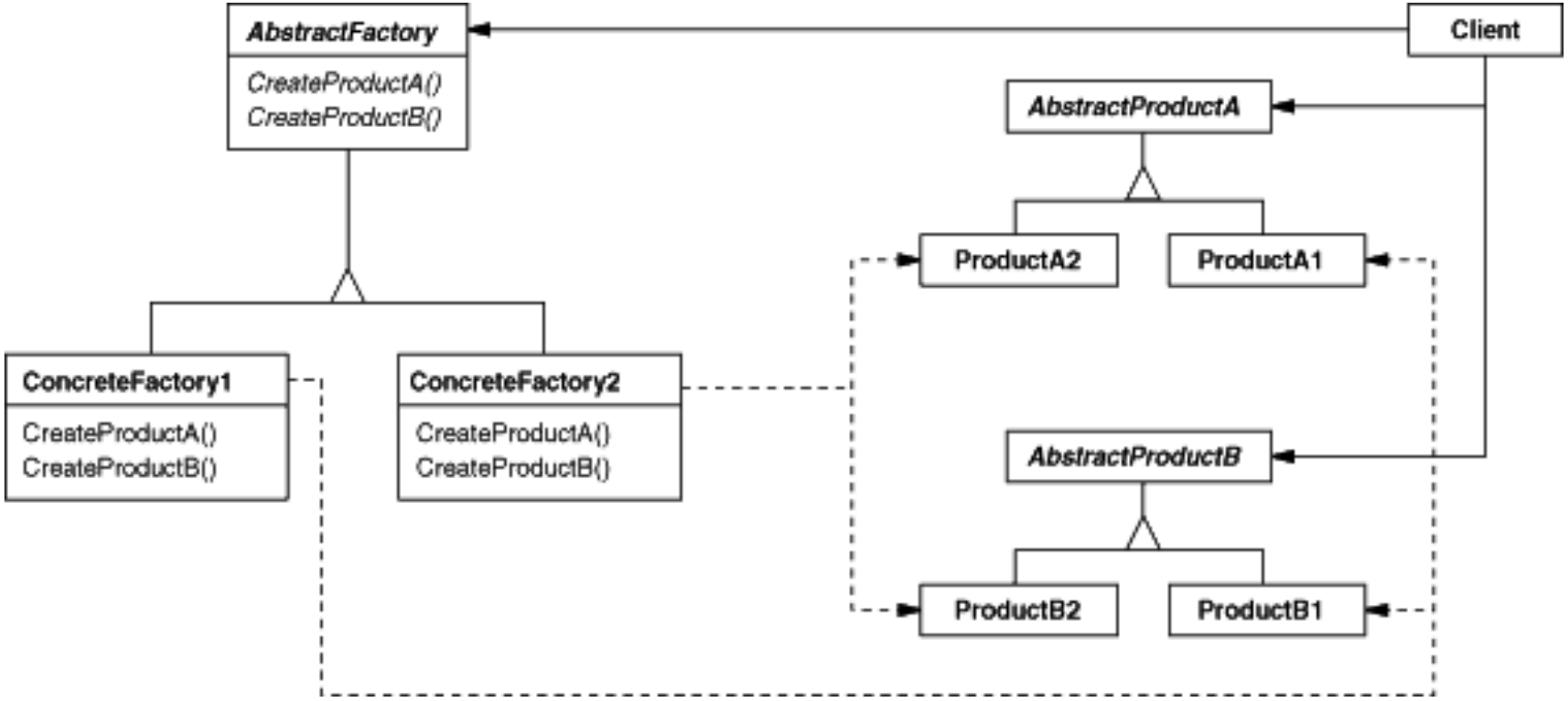
생성 로직 (구체적인 대상 생성 및 업케스팅)

```
ProductProcessor processor = new ProductProcessor();
processor.process(new ConcreteCreator());  //실제 구현제 전달
```

객체 생성 패턴 – Abstract Factory

관련성 있는 여러 종류의 객체군(패밀리)을 생성하는 인터페이스를 제공하는 패턴

- **장점**: 관련된 객체들을 일관되게 생성, 객체군 교체가 쉬움.
- **단점**: 새로운 제품 추가가 어려움, 구조가 복잡해질 수 있음.



객체 생성 패턴 – Abstract Factory

- 관련성 있는 여러 종류의 객체군(패밀리)을 생성하는 인터페이스를 제공하는 패턴
  - **장점**: 관련된 객체들을 일관되게 생성, 객체군 교체가 쉬움.
  - **단점**: 새로운 제품 추가가 어려움, 구조가 복잡해질 수 있음.

```
interface GUIFactory {
    Button createButton();
    Checkbox createCheckbox();
}
class WindowsFactory implements GUIFactory { ... }
class MacFactory implements GUIFactory { ... }
```

사용 예시

```
GUIFactory factory;
String osName = System.getProperty("os.name").toLowerCase();
if (osName.contains("mac")) {
  factory = new MacFactory();
} else {
  factory = new WindowsFactory();
}
Application app = new Application(factory);
app.renderUI();
```

객체 생성 패턴 – Builder

- 복잡한 객체를 단계별로 생성할 수 있게 도와주는 패턴(각 단계는 **메서드 체이닝 방식**으로 구현 가능)
  - **장점**: 객체 생성 과정 분리, 불변 객체 생성 가능, 가독성 향상.
  - **단점**: 클래스 구조 복잡해질 수 있음.

참고) 중첩 클래스 간에는 private 속성에 대해 서로 접근 가능

```
class User {
    private String name;
    private int age;

    // private 생성자 → Builder로만 객체 생성 가능
    private User(Builder builder) {
        this.name = builder.name;
        this.age = builder.age;
    }

    public static class Builder {
        private String name;
        private int age;
        public Builder setName(String name) { this.name = name; return this; }
        public Builder setAge(int age) { this.age = age; return this; }
        public User build() { return new User(this); }
    }
    private User(Builder builder) {
        this.name = builder.name;
        this.age = builder.age;
    }
}
// 사용: new User.Builder().setName("홍길동").setAge(20).build();
```

❖ **Lombok의 @Builder 어노테이션을 이용해서 구현 가능**

[실습] Builder를 직접 구성해보자

01.traing-code/14.pattern

14.pattern의 User 클래스 내에 아래의 Builder Class를 적용해보자

```
// 정적 내부 클래스(Static Inner Class) Builder
 public static class Builder {
     private String name;
     private int age;
     private String email;
     private String phoneNumber;

     // 빌더 생성자 (필수 값이 있다면 여기에 인자로 추가 가능, 현재는 기본 생성자)
     public Builder() {}

     public Builder setName(String name) {..}

     public Builder setAge(int age) {..}

     // 추가 필드에 대한 빌더 메서드
     public Builder setEmail(String email) {..}

     public Builder setPhoneNumber(String phoneNumber) {..}

     // 최종 객체를 조립하여 반환하는 build() 메서드
     public User build() {..}
 }
```

SOLID 원칙

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

SOLID 원칙 이란?

- 유지보수성, 확장성, 재 사용성을 높이고, 결합도 (coupling)을 낮추기 위한 OOP의 5대 설계원칙
- Clean Architecture(지속가능한 튼튼한 소프트웨어 구조)를 구성하기 위한 핵심적인 기초 원리

SOLID 원칙을 큰 그림에서 바라보면

- 객체 간의 결합도를 낮추고, 객체 내 응집도를 높이는 원칙
- 인터페이스와 추상화를 적극적으로 활용 (What(사용) 과 How(구현)를 분리)
- 변경이 필요할 때 기존 코드 수정 없이 확장이 가능하도록 설계 (상속/확장)
- 코드 유지보수성과 테스트 용이성을 향상

❖ Clean Architecture는 관심사 분리(separation of Concerns)를 기반으로 애플리케이션을 계층 별로 나누고 각 계층이 자신의 역할에만 집중하도록 설계하는 아키텍처.
비즈니스 규칙 과 기술 세부적 공통 사항을 분리

SOLID 원칙 이란?

객체지향에서 시작되었지만, 객체 뿐만 아니라 마이크로서비스와 API Level에까지 SOLID 원칙을 고려

원칙	이름	핵심 개념
S	**단일 책임 원칙 (SRP)** Single Responsibility Principle	하나의 클래스는 하나의 책임만 가져야 한다
O	**개방-폐쇄 원칙 (OCP)** Open/Closed Principle	시스템의 행위는 기존 구성을 수정하지 않고도 확장(Extension)할 수 있어야 하며, 확장할 때 기존 구성은 변경으로부터 폐쇄(Closure)되어야 합니다
L	**리스코프 치환 원칙 (LSP)** Liskov Substitution Principle	상위 호환성을 보장하는 원칙 시스템의 어떤 구성 요소를 호환 가능한 다른 요소로 교체하더라도, 시스템 전체의 행위가 일관되게 유지되고 오동작하지 않아야 합니다
I	**인터페이스 분리 원칙 (ISP)** Interface Segregation Principle	모듈이나 서비스는 자신이 사용하지 않는 불필요한 기능이나 데이터(의존성)에 강제로 엮이지 않아야 합니다
D	**의존 역전 원칙 (DIP)** Dependency Inversion Principle	고수준의 핵심 비즈니스 로직(정책)은 저수준의 구체적인 가변적 요소(기술, 프레임워크, 데이터베이스)에 의존해서는 안된다

1.SRP (단일 책임 원칙)

한 클래스는 하나의 책임만 가져야 한다. (구현자 관점)

- 한 클래스가 변경되어야 할 이유는 오직 하나여야 함
- 한 클래스가 여러 기능을 담당하면, 한 기능의 변경이 다른 기능에 부정적인 영향을 줄 수 있음

[나쁜 구조]

```
class Employee {
    public void getEmployeeInfo() { /* 직원 정보 조회 로직 */ }
    public void calculatePay() { /* 급여 계산 로직 */ }
}
```

[좋은 구조]

```
class Employee {
    public void getEmployeeInfo() { /* 직원 정보 조회 로직 */ }
}

class PayCalculator {
    public void calculatePay(Employee employee) { /* 급여 계산 로직 */ }
}
```

적용 예시

- Spring boot의 Controller, Service, Repository 분리

2. OCP (개방-폐쇄 원칙) (1/2)

행위의 본질을 '추상화(인터페이스, 규격)'하고, 구체적인 세부 기능은 '플러그인' 형태로 갈아 끼울 수 있게 만드는 것

- 기존 코드는 변경하지 않고(공통 흐름), 새로운 기능 (세부 구현) 을 추가할 수 있어야 함
- 인터페이스, 추상 클래스, 다형성을 활용한 구현

확장이 어려운 구조

```
// 결제 수단 종류
enum PaymentType { KAKAO_PAY, CARD }

// 결제 서비스
class PaymentService {
    public void processPayment(PaymentType type, int amount) {
        if (type == PaymentType.KAKAO_PAY) {
            System.out.println("카카오페이 " + amount + "원 결제 완료");
        } else if (type == PaymentType.CARD) {
            System.out.println("신용카드 " + amount + "원 결제 완료");
        }
        // 새로운 결제 수단(NAVER_PAY)이 추가되면?
        // 여기에 else if를 또 추가하고 이 클래스를 새로 컴파일해야 함!
    }
}
```

2. OCP (개방-폐쇄 원칙) (2/2)

확장이 용이한 구조

```
// 1. 결제라는 행위를 인터페이스로 추상화
interface Payment {
    void pay(int amount);
}

// 2. 구체적인 결제 구현체들을 분리
class KakaoPay implements Payment {
    public void pay(int amount) { System.out.println("카카오페이 " + amount + "원 결제 완료"); }
}

class CardPay implements Payment {
    public void pay(int amount) { System.out.println("신용카드 " + amount + "원 결제 완료"); }
}

// 3. 결제 서비스는 인터페이스에만 의존 (변경에 닫힘)
class PaymentService {
    public void processPayment(Payment payment, int amount) {
        payment.pay(amount); // 어떤 결제 수단이 들어오든 이 코드는 변하지 않음
    }
}
```

[실습] OCP

15.soild/02.ocp

[실습 1]
Payment Interface를 상속받은 Kakao.java CardPay.java, NaverPay.java를 생성하고,
Main.main() 에 3개의 Pay를 PaymentService.processPayment()를 호출해보자

```
class PaymentService {
    public void processPayment(Payment payment, int amount) {
        // 어떤 결제 수단 클래스가 들어오든 이 코드는 절대 변경되지 않습니다.
        payment.pay(amount);
    }
}
public class Main {
    public static void main(String[] args) {
        PaymentService paymentService = new PaymentService();

        // 1. 카카오페이 결제 시도
        Payment kakao = new KakaoPay();
        paymentService.processPayment(kakao, 15000);

        // 2. 신용카드 결제 시도
        Payment card = new CardPay();
        paymentService.processPayment(card, 53000);

        // 3. OCP를 증명하는 핵심: 기존 코드를 전혀 고치지 않고 새로 추가된 NaverPay를 바로 수용함
        Payment naver = new NaverPay();
        paymentService.processPayment(naver, 21000);

    }
}
```

3. LSP (리스코프 치환 원칙) (1/2)

자식 클래스는 언제나 부모 클래스의 역할을 대체할 수 있어야 한다.

- 자식 클래스는 부모 클래스가 정의한 계약을 위반하지 말아야 함
- 부모 클래스의 행동(method)을 호출하는 코드에서 자식 클래스로 대체하더라도 정상적으로 동작

LSP 위반 구조

```
// 직사각형 (부모) - 가로가 바뀌어도 새로는 유지될 것으로 이해
class Rectangle {
    protected int width;
    protected int height;

    public void setWidth(int width) { this.width = width; }
    public void setHeight(int height) { this.height = height; }
    public int getArea() { return width * height; }
}

class Square extends Rectangle {
    // 정사각형은 가로/세로가 같아야 하므로 부모의 메서드를 왜곡함 - LSP 위반
    @Override
    public void setWidth(int width) {
        this.width = width;
        this.height = width; // 가로를 바꿨는데 세로도 바뀜
    }

    @Override
    public void setHeight(int height) {
        this.width = height;
        this.height = height;
    }
```

3. LSP (리스코프 치환 원칙) (2/2)

LSP 준수 구조

```
// 1. 공통 규약 인터페이스 정의
interface Shape {
    int getArea();
}

// 2. 직사각형은 고유의 규칙대로 구현
class Rectangle implements Shape {
    private final int width;
    private final int height;

    public Rectangle(int width, int height) {
        this.width = width;
        this.height = height;
    }

    @Override
    public int getArea() { return width * height; }
}

// 3. 정사각형도 고유의 규칙대로 구현 (더 이상 직사각형의 규격 요청을 고려할 필요 없음)
class Square implements Shape {
    private final int length;

    public Square(int length) {
        this.length = length;
    }

    @Override
    public int getArea() { return length * length; }
}
```

[실습] LSP

15.soild/03.lsp

03.lsp 코드를 확인해보고 Main.java에
- Rectangle, Square 코드를 만들고, 아래의 Main 코드를 작성해서 실행해보자

```
class Main {
    public static void renderArea(Shape shape) {
        // 어떤 도형(Shape)이 들어오든, 개발자는 뒤통수 맞을 걱정 없이 getArea() 규칙만 믿고 쓰면 됨
        System.out.println("도형의 넓이: " + shape.getArea());
    }

    public static void main(String[] args) {
        Shape rect = new Rectangle(5, 10);
        Shape square = new Square(5);

        renderArea(rect);   // 출력: 50
        renderArea(square); // 출력: 25
    }
}
```

4.ISP(인터페이스 분리 원칙) (1/2)

클라이언트는 자신이 사용하지 않는 메서드에 의존하지 말아야 한다 (사용자/소비자 관점)

- 하나의 범용적인 인터페이스보다는 여러 개의 구체적인 인터페이스를 만들어야 함
- 한개의 인터페이스에 너무 많은 기능을 담고 있으면 이를 구현한 클래스는 불필요한 메서드를 강제로 구현

LSP 위반 구조

```
// 거대하고 뚱뚱한 인터페이스 (Fat Interface)
interface SmartPrinter {
    void print();
    void copy();
    void fax();
}

// ISP 위반! 보급형 프린터는 '복사'나 '팩스'를 못 하는데도 억지로 메서드를 구현해야 함
class BasicPrinter implements SmartPrinter {
    @Override
    public void print() {
        System.out.println("문서를 출력합니다.");
    }

    @Override
    public void copy() {
         throw new UnsupportedOperationException("복사 기능을 지원하지 않습니다.");
    }

    @Override
    public void fax() {
        throw new UnsupportedOperationException("팩스 기능을 지원하지 않습니다.");
    }
}
```

4.ISP(인터페이스 분리 원칙)

LSP 준수 구조

```
// 1. 역할을 잘게 쪼갠 작고 구체적인 인터페이스들
interface Printer {
    void print();
}

interface Copier {
    void copy();
}

interface Fax {
    void fax();
}

// 2. 보급형 프린터는 출력 인터페이스만 구현
class BasicPrinter implements Printer {
    @Override
    public void print() {  System.out.println("보급형 프린터: 문서를 출력합니다.");  }
}

// 3. 최신형 고급 복합기는 필요한 모든 인터페이스를 선택해서 구현
class AdvancedSmartPrinter implements Printer, Copier, Fax {
    @Override
    public void print() { System.out.println("고급 복합기: 출력합니다."); }

    @Override
    public void copy() { System.out.println("고급 복합기: 복사합니다."); }

    @Override
    public void fax() { System.out.println("고급 복합기: 팩스를 보냅니다."); }
}
```

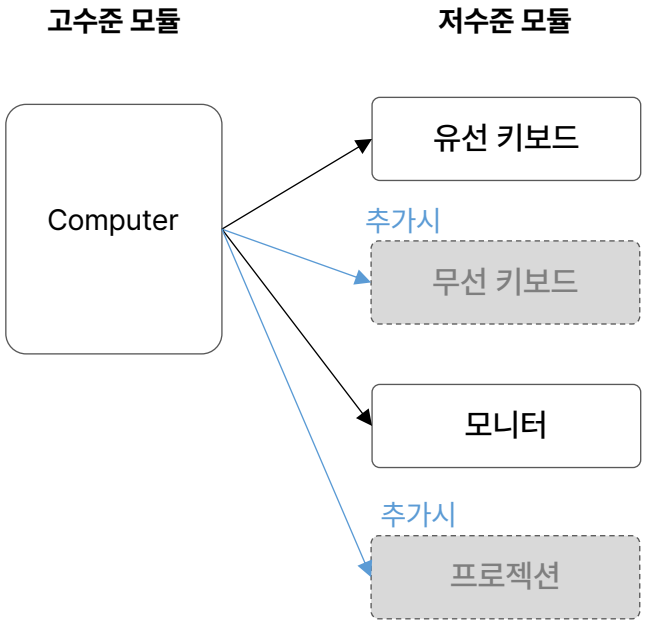
SOLID 원칙 – 5. DIP(의존 역전 원칙)

고수준 모듈은 저수준 모듈에 의존하면 안 되며, 둘 다 추상화에 의존해야 한다

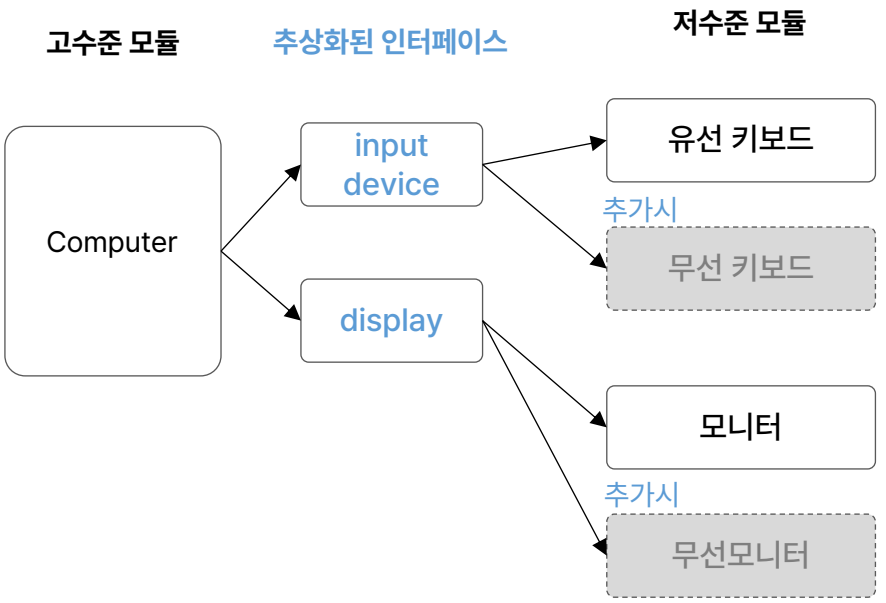
추상화는 구체적인 것에 의존해서는 안되고, 구체적인 것이 추상화에 의존해야 한다.

- **고수준 모듈**: 비즈니스 정책, 흐름 (process, service)
- **저수준 모듈**: 세부 구현, 기술, 라이브러리
- **추상화**: Interface, abstract class

나쁜 구조 **고수준 모듈은 저수준 모듈 구현체에 의존**



고수준 모듈은 저수준 모듈 구현체에 의존하지 않음 좋은 구조



SOLID 원칙 – 5. DIP(의존 역전 원칙) (1/2)

DIP 위반 구조

```
class Keyboard {
    public void type() {
        System.out.println("키보드를 사용하여 입력");
    }
}

class Monitor {
    public void display() {
        System.out.println("모니터에 화면을 출력");
    }
}

class Computer {
    private Keyboard keyboard;
    private Monitor monitor;

    public Computer() {
        this.keyboard = new Keyboard();
        this.monitor = new Monitor();
    }

    public void operate() {
        keyboard.type();
        monitor.display();
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Computer computer = new Computer();
        computer.operate();
    }
}
```

강한 결합 (Tight Coupling)

- Computer 클래스는 Keyboard와 Monitor의 구체적인 구현에 직접 의존
- 새로운 입력 장치(Mouse)가 추가된다면 Computer 클래스를 수정 필요

유지보수 및 확장성이 낮음

- Keyboard를 무선 키보드(WirelessKeyboard)로 변경하려면 Computer 클래스를 직접 수정 필요
- OCP(개방-폐쇄 원칙)도 위반

SOLID 원칙 – 5. DIP(의존 역전 원칙) (2/2)

DIP 준수 구조

```
interface InputDevice {
    void type();
}
interface OutputDevice {
    void display();
}

class Keyboard implements InputDevice {
    @Override
    public void type() {
        System.out.println("키보드를 사용하여 입력");
    }
}
class Monitor implements OutputDevice {
    @Override
    public void display() {
        System.out.println("모니터에 화면을 출력");
    }
}
```

```
class Computer {
    private final InputDevice inputDevice;
    private final OutputDevice outputDevice;

    public Computer( InputDevice inputDevice,
                     OutputDevice outputDevice) {
        this.inputDevice = inputDevice;
        this.outputDevice = outputDevice;
    }

    public void operate() {
        inputDevice.type();
        outputDevice.display();
    }
}

public class Main {
    public static void main(String[] args) {
        // 조립(Composition Root)에서만 구현체를 선택
        InputDevice keyboard = new Keyboard();
        OutputDevice monitor = new Monitor();

        Computer computer = new Computer(keyboard, monitor);
        computer.operate();
    }
}
```

새로운 입력 장치(마우스)나 출력 장치(프로젝터)를 추가해도 Computer 클래스를 수정 불필요

실행 시점에 원하는 구현체를 주입할 수 있으므로, 코드 수정 없이 다양한 장치를 사용 가능

[실습] DIP

15.soild/05.dip

[실습1]

앞서 배운 Computer, InputDevice, OutputDevice Class를 기반으로 DIP를 적용하기 위한

1. Keyboard, Monitor 클래스를 작성하고

2. Main.main() 함수를 완성해보자

[실습 2]

WirelessKeyboard.와 WirelessDisplay 를 추가해보자

인터넷과 웹

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

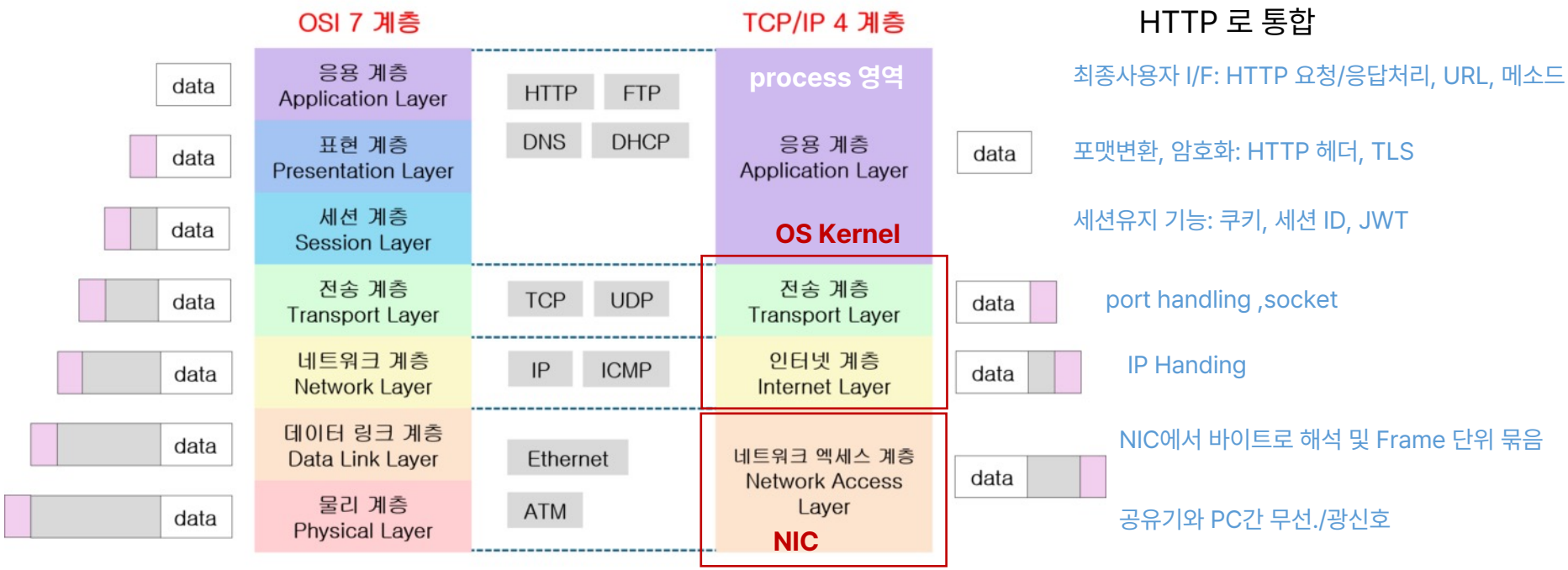
네트워크 참조 모델

OSI 7 Layer는 국제 표준화 기구(ISO)에서 발표한 네트워크 통신 기준이 되는 가장 대표적인 참조 모델

TCP/IP 4 Layer는 미 국방성 연구에서 시작되어 발전한 사실상의 업계 표준 참조 모델

프로토콜은 참조 모델 계층의 구현체

- IP(Internet Protocol): 데이터를 목적지(IP 주소)까지 전송
- TCP(Transmission Control Protocol): 데이터 전송의 신뢰성 보장 → 패킷 손실 시 재전송, 순서 보장
- UDP(User Datagram Protocol): 실시간 스트리밍 등의 빠른 데이터 전송
- 응용 계층 프로토콜: HTTP, DNS, FTP, SMTP 등

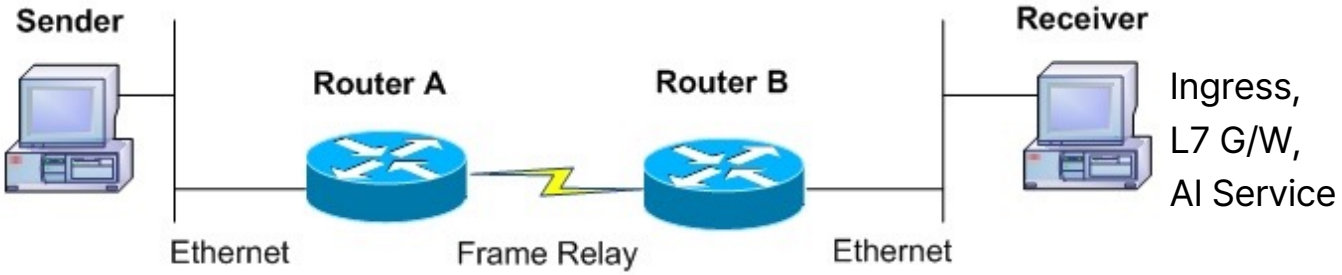


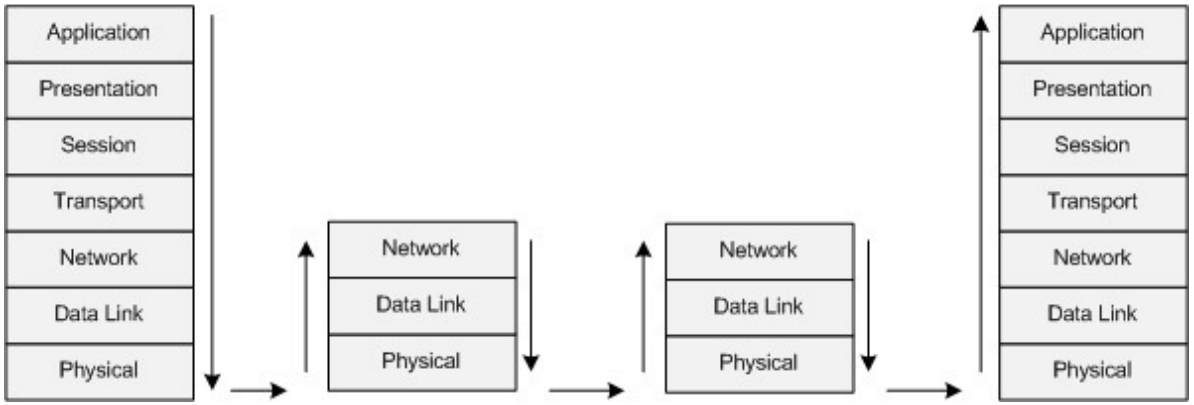
출처: https://westahn.com/osi-7-%EA%B3%84%EC%B8%B5%EC%9D%B4%EB%9E%80/

[참고] 레이버별 역할

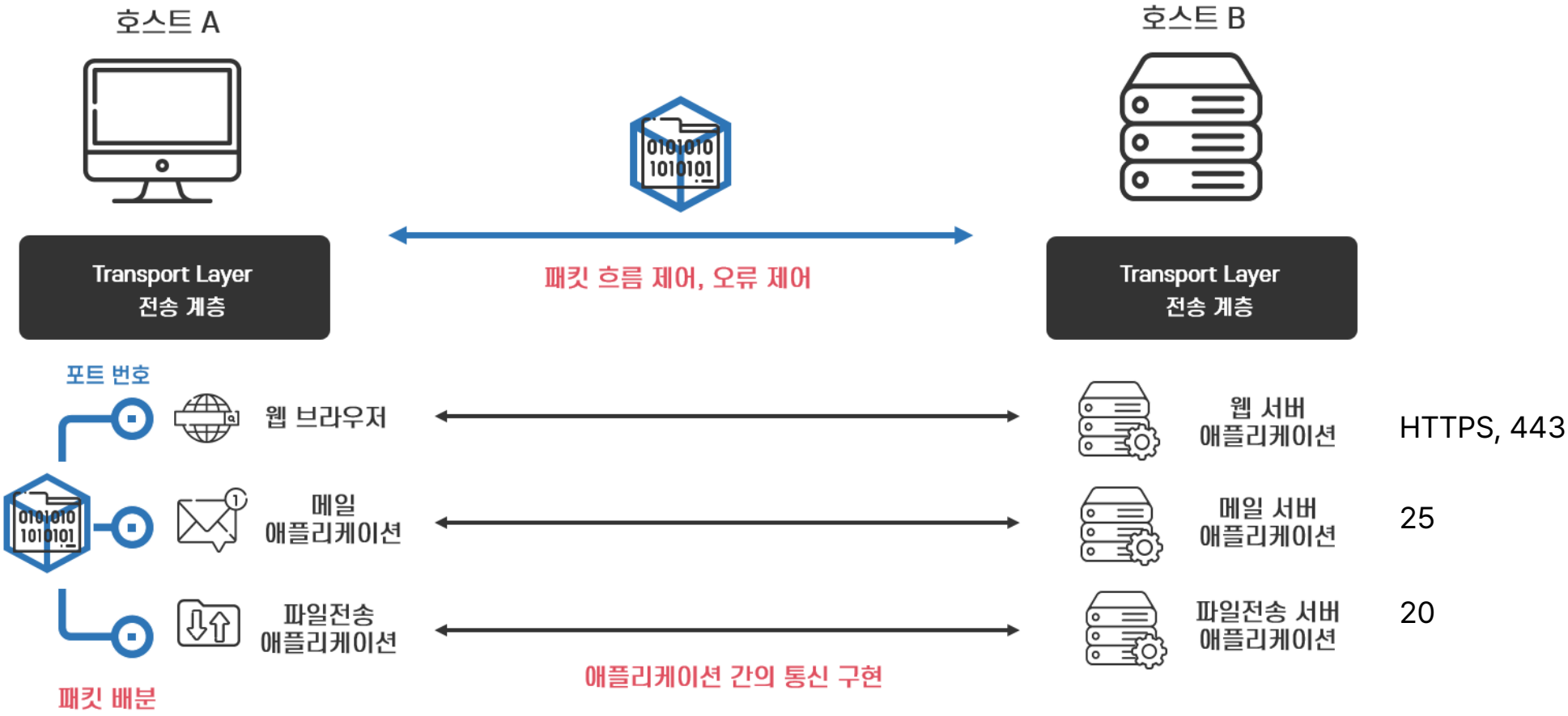
OSI 개층 별 서버 간 연결 방식과 장비

계층	주요 역할	주요 장비	설명
L1	신호 전송	허브, 리피터	물리적 신호를 단순 중계, 충돌 많음 (구시대 장비)
L2	같은 네트워크 내 MAC 기반 통신	**L2 스위치**	같은 네트워크 내 서버 MAC 주소 기반으로 패킷 전달
L3	서로 다른 네트워크 간 통신	**라우터 (L3 스위치)**	IP 기반 라우팅, 외부 네트워크로 트래픽 전달
L4	TCP/UDP 포트 기반 트래픽 처리	**L4 로드 밸런서**	포트/세션 기반 분산 처리 및 방화벽 기능 가능
L5L7	세션 유지, 컨텐츠 기반 분산	**L7 로드 밸런서 (G/W)**	HTTP 헤더, URL 등으로 로드 밸런싱, 보안 제어 등





[참고] Transport Layer (Port 기반)



웹(WWW - World Wide Web) 이란?

인터넷 상의 문서, 이미지, 비디오 등 하이퍼링크로 연결된 정보들의 집합이며,

웹 브라우저로 접근 가능한 전 세계 정보 공간을 의미

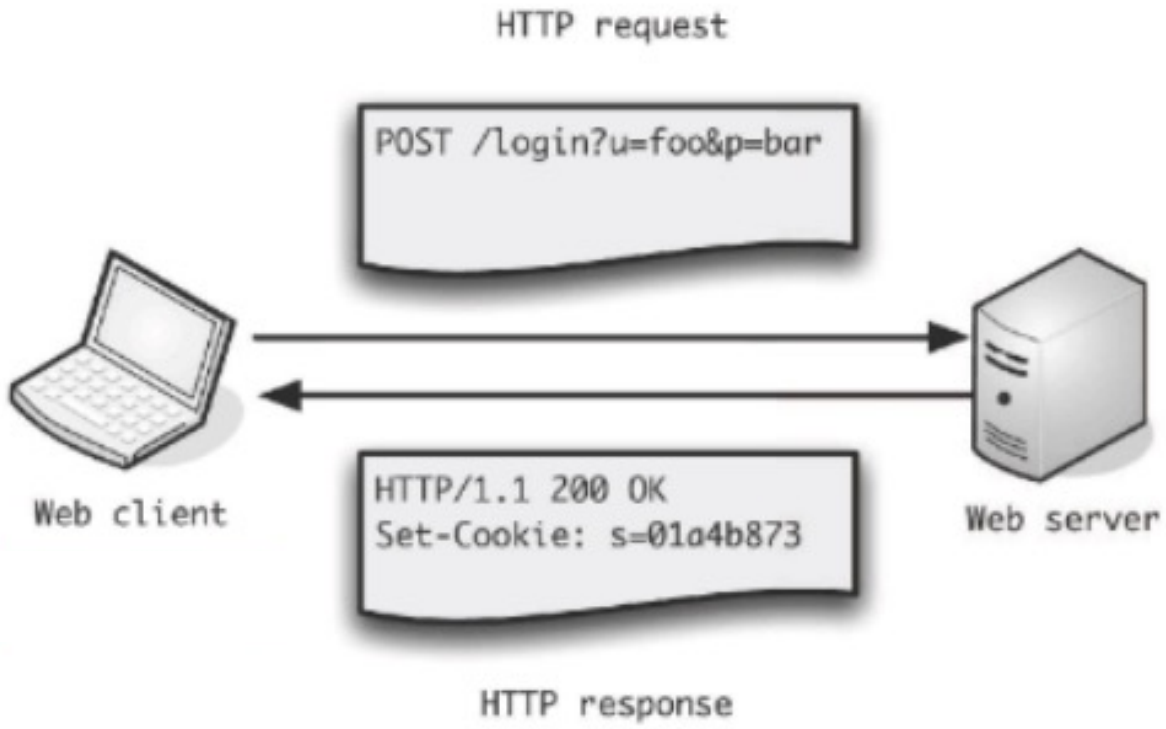
- **인터넷 = 네트워크 인프라** (하드웨어, 라우터, IP 등)
- **WWW = 인터넷 위에 구현된 정보 공간**
- 웹 문서는 **HTML**, 웹 서버는 **HTTP 프로토콜**, 사용자 인터페이스는 **브라우저**

❖ 인터넷과 웹의 차이점

구분	인터넷	웹(WWW)
정의	네트워크 인프라 (하드웨어, 라우터, IP 등)	인터넷 상의 문서, 이미지, 비디오 등 하이퍼링크로 연결된 정보들의 집합
역할	네트워크 연결과 데이터 통신을 담당	웹 브라우저로 접근 가능한 전 세계 정보 공간 접근
주요 프로토콜	(TCP, UDP)/IP 등	HTTP, HTTPS

응용 계층 프로토콜: HTTP(s)

웹에서 데이터를 주고받기 위한 통신 규약, 즉 프로토콜이며, 응용 계층 (Application 내부에서 처리)
클라이언트(웹 브라우저 등)가 서버에 요청을 보내면 서버는 해당 요청에 대한 응답을 돌려주는 방식
HTTP는 웹 페이지, 이미지, 영상 등 다양한 형태의 데이터를 전송하는 데 사용



HTTPS (~ Secure)

- HTTP의 보안 버전으로 **SSL(Secure Sockets Layer)/TLS(Transport Layer Security)** 암호화를 사용하여 데이터 보호

❖ HTTPS는 현재 웹사이트의 기본 표준이며, Google 크롬 같은 브라우저는 HTTPS를 사용하지 않는 웹사이트에 대해 "**보안 위험 경고**"를 표시

https://makarand-khiste.medium.com/understanding-the-osi-model-all-7-layers-explained-with-architecture-afd42c7b7483

URL

URL = Uniform Resource Locator

⇒ 네트워크 상에서 통합 자원의 "위치"를 나타내기 위한 규약

⇒ 웹사이트 주소 + 컴퓨터 네트워크 상의 자원

```
<scheme>://<user>:<password>@<host>:<port>/<path>?<query>#<fragment>
```

프로토콜 (Scheme)	포맷 예시	기본 포트	설명
HTTP	http://example.com	80	웹에서 일반적으로 사용하는 프로토콜
HTTPS	https://example.com	443	보안 웹 프로토콜 (TLS/SSL)
FTP	ftp://user:pass@example.com:21/path/file	21	파일 전송 프로토콜 (파일 업로드/다운로드)
FILE	file:///C:/Users/example/file.txt	없음	로컬 파일 시스템 접근
WS / WSS	ws://example.com/socket, wss://...	80 / 443	웹소켓 (실시간 통신)

user/pwd 예시

```
git clone https://himang10:xxxxx@github.com/himang10/ywyi-skala-edu-code.git
```

Request / Response 예시

GET Request Message

https://jsonplaceholder.typicode.com/todos/1

```
GET /api/user/12345 HTTP/1.1                          ← Start Line
Host: api.example.com                                 ← Header
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5...     ← JWT 토큰 등
Accept: application/json                              ← JSON 응답을 요청
User-Agent: Mozilla/5.0
Connection: keep-alive
```

GET Response Message

```
HTTP/1.1 200 OK                                       ← Status Line
Content-Type: application/json                        ← Header
Content-Length: 85
Date: Fri, 26 Jul 2025 04:40:00 GMT
Connection: keep-alive

{                                                     ← Body (JSON)
  "id": 12345,
  "name": "홍길동",
  "email": "hong@example.com",
  "status": "active"
}
```

HTTP Method

CRUD 작업

⇒ Create, Read, Update, Delete 를 나타내며 기본적인 데이터 처리 기능을 의미함

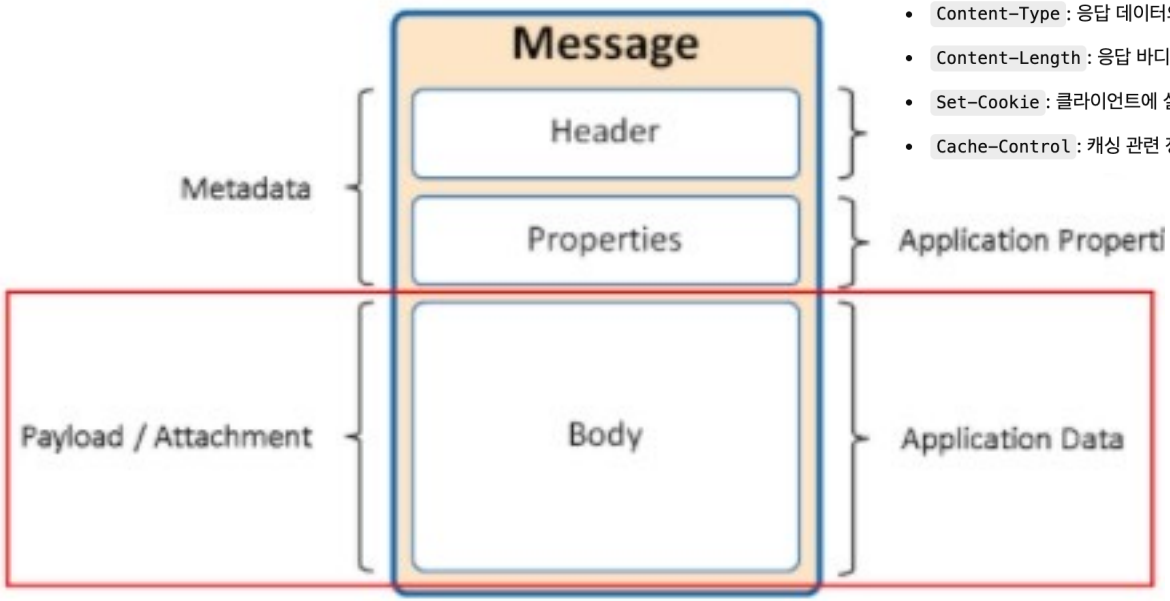
HTTP Method 와는 다음과 같이 연결됨

- GET => Read
- POST => Create
- PUT=> Update
- DELETE => Delete

HTTP Message

Payload = 페이로드 = 전송되는 데이터

⇒ 함께 전달되는 헤더나 속성은 포함되지 않음



① HTTP Request Headers (요청 헤더)

- `Host` : 요청을 처리할 서버의 도메인 (예: `example.com`)
- `User-Agent` : 클라이언트 정보 (브라우저, OS 정보 등)
- `Accept` : 클라이언트가 원하는 응답의 MIME 타입 (예: `application/json`)
- `Authorization` : 인증 정보 (예: `Bearer token`)
- `Content-Type` : 요청의 본문이 어떤 데이터 형식인지 지정 (예: `application/json`)

② HTTP Response Headers (응답 헤더)

- `Content-Type` : 응답 데이터의 MIME 타입 (예: `text/html`)
- `Content-Length` : 응답 바디의 크기
- `Set-Cookie` : 클라이언트에 설정할 쿠키 정보
- `Cache-Control` : 캐싱 관련 정책

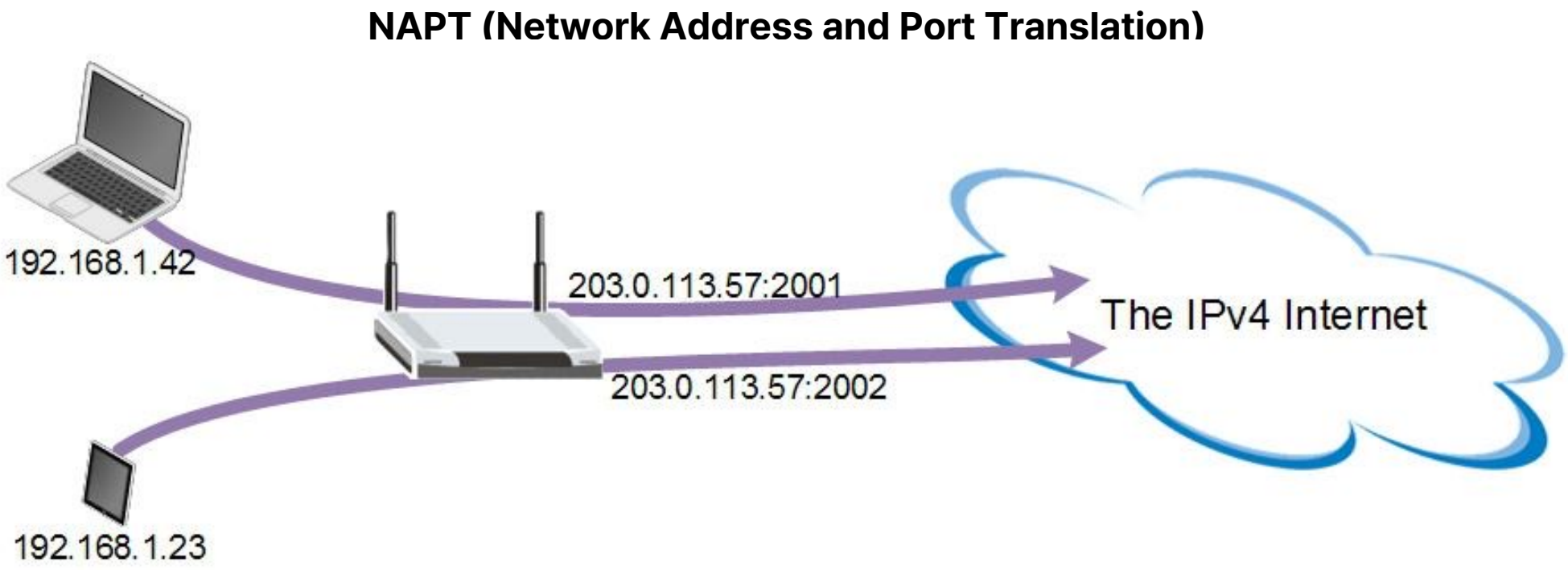
예시 (Custom Headers)

```http
X-Correlation-ID: 12345abcd
X-Request-ID: abcdefg-67890
X-App-Version: 1.0.3
```

- `X-Correlation-ID` : 분산 트랜잭션을 추적하기 위한 ID
- `X-Request-ID` : 특정 요청을 식별하기 위한 ID
- `X-App-Version` : 애플리케이션 버전 정보

응용 계층 프로토콜: 무선공유기 DHCP

공유기 (DHCP 서버)와 Client PC (DHCP Client) 사이에 PC의 내부 네트워크 상에서 사용 가능한 IP를 동적으로 할당받기 위한 프로토콜 (서버 67 번포트, 클라이언트 68 번 포트)



ipconfig / ifconfig로 나의 IP를 확인하고 다른 사람 네트워크로 접속해보기

Socket

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

Application layer 진입점: 소켓(socket)

- OS 커널이 제공하는프로세스가 네트워크(IP/Port)를 통해 통신하기 위한 추상화된 통신 엔드포인트
- OS Kernel 객체 + Socket API (유저 공간)
- 연결 유지, Socket file Descriptor 관리, 포트 바인딩 (kernel) + 응용 프로그램 socket 사용
- 소켓은 특정 포트 번호와 연결되어 있어서 TCP에서 데이터를 보낼 응용 프로그램을 식별

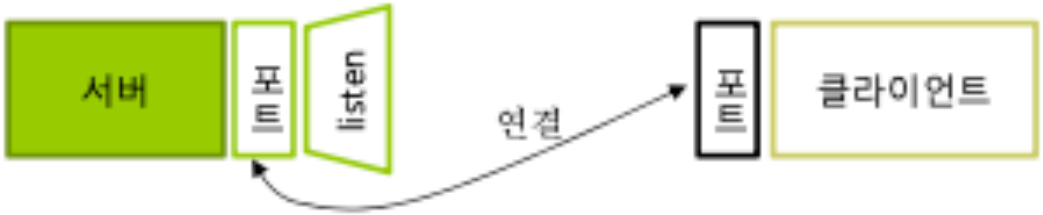
https://makarand-khiste.medium.com/understanding-the-osi-model-all-7-layers-explained-with-architecture-afd42c7b7483

클라이언트와 서버 연결

- 서버는 서버 소켓으로 들어오는 연결 요청을 기다림

- 클라이언트가 서버에게 연결 요청

- 서버가 연결 요청 수락하고 새로운 소켓을 만들어 클라이언트와 연결 생성



https://makarand-khiste.medium.com/understanding-the-osi-model-all-7-layers-explained-with-architecture-afd42c7b7483

InetAddress Class

개념

- java.net.InetAddress
- IP 주소 + 호스트 이름(도메인)을 표현하는 클래스
- DNS 조회, 로컬 호스트 정보 조회 등에 사용

주요 역할

- 도메인 이름 → IP 주소 변환
- IP 주소 → 도메인 이름(역방향 조회)
- 로컬 머신의 IP/호스트명 조회

```
InetAddress address = clientSocket.getInetAddress();

String clientIp = address.getHostAddress(); // "192.168.1.100"

String hostname = address.getHostName();    // "client-pc.local"
```

주요 메소드

- **InetAddress.getByName(String host)** : "www.naver.com" → 해당 서버의 IP 정보
- **InetAddress.getLocalHost()**: 현재 실행 중인 로컬 컴퓨터의 InetAddress Instance 정보
- **getHostAddress()** : String 형태의 IP 주소 리턴 (예: "192.168.0.10")
- **getHostName()**: 호스트 이름(도메인명) 리턴

참고: InetAddress 를 통해 Domain Name을 IP를 찾는다. 실제는 OS Kernel glib resolver에게 IP를 찾도록 요청 (/etc/reolv.conf)

https://makarand-khiste.medium.com/understanding-the-osi-model-all-7-layers-explained-with-architecture-afd42c7b7483

[실습] InetAddress Class

16.socket/InetAddressExample.java

harbor registry의 domain을 기반으로 IP를 추출해보자

```
        // 도메인 이름으로 IP 조회
        InetAddress harbor = InetAddress.getByName("skala-registry.skala-ai.com");
        System.out.println("Host Name : " + harbor.getHostName());
        System.out.println("Host Address : " + harbor.getHostAddress());

        // 로컬 호스트 정보 조회
        InetAddress local = InetAddress.getLocalHost();
        System.out.println("Local Host Name : " + local.getHostName());
        System.out.println("Local IP Address : " + local.getHostAddress());
```

컴파일:

javac -d bin -cp . InetAddressExample.java

실행:

java -cp bin InetAddressExample

실행결과:

Host Name : skala-registry.skala-ai.com

Host Address : 3.39.139.241

Local Host Name : 09442ui-MacBookPro.local

Local IP Address : 127.0.0.1

Socket Class

개념

- java.net.Socket
- TCP 클라이언트를 표현하는 클래스
- 특정 서버(IP, Port)에 접속해서 데이터 송수신

주요 역할

- 서버에 TCP 연결 시도 (new Socket(host, port))
- 연결된 후 입출력 스트림을 통해 데이터 송·수신
- 연결 종료 (close())

주요 메소드

- **생성자: new Socket(String host, int port)**
- **getInputStream() : 서버에서 오는 데이터 읽기**
- **getOutputStream() : 서버로 데이터 쓰기**
- **close() : 소켓 종료**

https://makarand-khiste.medium.com/understanding-the-osi-model-all-7-layers-explained-with-architecture-afd42c7b7483

ServerSocket Class

개념

- java.net.ServerSocket
- TCP 서버를 표현하는 클래스
- 특정 포트에서 클라이언트 연결을 기다리고(리스닝), 연결되면 Socket을 하나 만들어 반환

주요 역할

- 서버 포트 바인딩 (new ServerSocket(port))
- accept() 호출로 클라이언트 연결 대기
- 연결이 들어오면 Socket 인스턴스를 반환 → 이 Socket으로 실제 데이터 송·수신

주요 메소드

- **생성자:** new ServerSocket(int port)
- **accept() :** 클라이언트 접속을 기다렸다가, 연결되면 Socket 반환
- **close() :** 서버 소켓 종료 (더 이상 새로운 연결 받지 않음)

https://makarand-khiste.medium.com/understanding-the-osi-model-all-7-layers-explained-with-architecture-afd42c7b7483

[코드 리뷰] ServerSocket 클래스 (1/3)

16.socket/SimpleEchoServer.java

ServerSocket 객체를 이용해서 TCP 통신 (IP/PORT)을 위한 Server Listening 기능을 검토하고 실행해보자

```
try (ServerSocket serverSocket = new ServerSocket(port)) {
            System.out.println("Echo Server started on port " + port);

            while (true) {
                // 클라이언트 접속 대기
                Socket clientSocket = serverSocket.accept();
                System.out.println("Client connected: " + clientSocket.getInetAddress());

                // 간단히 한 번만 Echo 처리 후 종료하는 버전
                try (
                    BufferedReader reader = new BufferedReader(
                        new InputStreamReader(clientSocket.getInputStream())
                    );
                    PrintWriter writer = new PrintWriter(
                        clientSocket.getOutputStream(), true
                    )
                ) {
                    System.out.println("Waiting for message...");
                    String line = reader.readLine();
                    System.out.println("Received: " + line);

                    writer.println("Echo: " + line);
                }

                clientSocket.close();
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
```

[코드 리뷰] Socket 클래스 (2/3)

16.socket/SimpleEchoClient.java

Socket 객체를 이용해서 ServerSocket으로
1. TCP/IP 세션을 연결하고,
2. 메시지를 전달하고
3. 연결을 끊는 기능을 검토하고 실행해보자

```
String host = "localhost"; // 또는 서버 IP
int port = 8080;

InetAddress serverAddr = InetAddress.getByName(host);

try (
        Socket socket = new Socket(serverAddr, port);
        BufferedReader reader = new BufferedReader(new InputStreamReader(socket.getInputStream()));
        PrintWriter writer = new PrintWriter( socket.getOutputStream(), true);
)
{
        // 서버로 메시지 전송
        writer.println("Hello Server!");

        // 서버에서 Echo 응답 수신
        String response = reader.readLine();
        System.out.println("Response: " + response);
}
```

[실행하기] Simple Socket 통신 (3/3)

16.socket/SimpleEchoClient.java

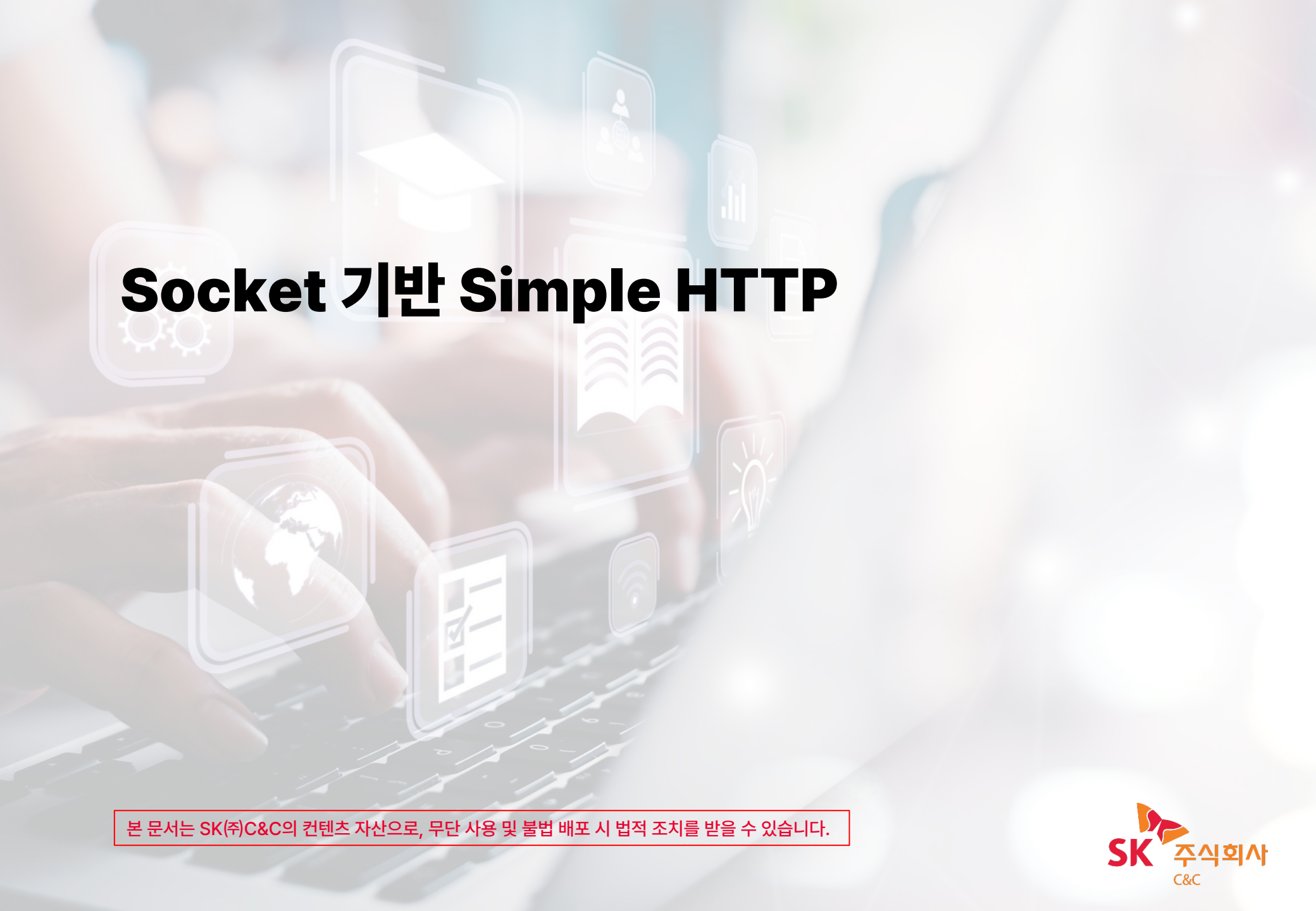
SimpleEchoClient와 SimpleEchoServer를 실행하고 통신이 되는 것을 확인한다.

1. Server Socket 실행

```
> java SimpleEchoServer.java
Echo Server started on port 8080



Client connected: /127.0.0.1
Waiting for message...
Received: Hello Server!
```

2. Client Socket 실행

```
> java SimpleEchoClient.java



Response: Echo: Hello Server!
```



Socket 기반 Simple HTTP

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

HTTP 프로토콜 이해하기

HttpEchoServer.java, HttpEchoClient.java

- 아래의 실습 코드는 ServerSocket을 이용해서 HTTP 기반 API Server를 구성하고
- Socket을 이용해서 HTTP Client를 구성하고 있음

HTTP는 Transport Layer 위의 Application Layer에서 동작하는 프로토콜이다.

즉 Application 프로그램 내에서 HTTP 프로토콜을 처리하는 구조

프로그램 내용을 검토해보고 이 개념을 이해해보자

16.socket 아래 코드

HttpEchoServer.java

- 연결 대기 (ServerSocket.accept())
- TCP Socket으로 입력되는 ByteStream을 HTTP Request 전환
- 응답 Text Response 생성 및 ByteStream으로 Socket으로 전달

HttpEchoClient.java

- HttpEchoServer로 연결 요청 및 연결
- 연결된 세션으로 HTTP Request를 전달 및 HTTP Response 수신
- 연결 종료

[코드 리뷰] HTTP Server/Client (2/3)

HttpEchoServer.java, HttpEchoClient.java

ServerSocket과 Socket을 이용해서 HTTP Server와 Client를 구성한 HttpEchoServer.java와 HttpEchoClient.java를 검토해보자

HttpEchoClient가 보내는 HTTP Request 메시지 (HTTP 프로토콜 메시지)

```
GET /users HTTP/1.1
Host: localhost
Connection: close

```

일반적으로 Connection: keep-alive

HttpEchoServer가 보내는 HTTP Response 메시지 (HTTP 프로토콜 메시지)

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=UTF-8
Content-Length: 83
Connection: close

{"users":[{"id":1,"name":"Alice"},{"id":2,"name":"Bob"},{"id":3,"name":"Charlie"}]}
```

[코드 리뷰] HTTP Server/Client (3/3)

HttpEchoServer.java, HttpEchoClient.java

SimpleEchoClient와 SimpleEchoServer를 실행하고 통신이 되는 것을 확인한다.

1. Server Socket 실행

```
> java HttpEchoServer.java
HTTP Echo Server started on port 8080
접속 경로 예시:
  GET /          → 서버 정보
  GET /users     → 사용자 목록
  GET /time      → 현재 시간
  GET /other     → 404 Not Found
```

2. Client Socket 실행

```
> java HttpEchoClient.java

================================================
▶ GET / 요청
[전송 요청]
  GET / HTTP/1.1
  Host: localhost
  Connection: close
  (빈 줄 - 헤더 끝)

[수신 응답]
  상태 라인: HTTP/1.1 200 OK
  헤더: Content-Type: application/json; charset=UTF-8
  헤더: Content-Length: 59
  헤더: Connection: close
  (빈 줄 - 헤더 끝)
  본문(JSON):
{"service":"HttpEchoServer","status":"running","port":8080}

[결과] 상태코드=200 ✓ 성공

================================================
```

수고하셨습니다.

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C